BIBLIOTECA IA, SOFTWARE & COMPUTAÇÃO · LIVRO 3

Programando com IA

Ferramentas de programação com IA, hardware e IA local, o conceito AI-Driven Engineer, desenvolvimento seguro, Pull Requests e segurança cloud-first.

Edição de outubro de 2026

Sobre esta coleção

> **Para começar**
> Esta coleção explica inteligência artificial, desenvolvimento de software e computação do zero. Cada capítulo começa em linguagem simples, com analogias do dia a dia, e aprofunda aos poucos. Os livros podem ser lidos separadamente.

O conteúdo reorganiza o que ensinam centenas de aulas e vídeos (do CS50 de Harvard a canais como IBM Technology, Anthropic e muitos outros), artigos técnicos de referência e o blog AkitaOnRails, complementados com pesquisa atualizada. Os vídeos aparecem pelo título, com link; cada capítulo termina com a tabela de fontes. Caixas *Você sabia?* trazem curiosidades e grandes nomes da tecnologia.

Os livros da coleção

1. Livro 1 — Fundamentos de IA e Modelos de Linguagem
2. Livro 2 — Construindo com LLMs
3. Livro 3 — Programando com IA
4. Livro 4 — Engenharia de Software e Ciência da Computação
5. Livro 5 — IA, Negócios e Sociedade
6. Livro 6 — Cultura, Comunidade e História

> **Datas**
> IA muda depressa. Preços, planos, benchmarks e nomes de modelos refletem outubro de 2026. Figuras marcadas como "ilustrativas" mostram a ideia, não medições.

Sumário

1. Programando com IA: Claude Code, Codex, Cursor e cia 4
2. Hardware, GPUs e IA Local 19
3. AI-Driven Engineer 31
4. Desenvolvimento seguro com IA e o futuro do Pull Request 48
5. Segurança cloud-first: o app-casca e a nova camada de proteção 61

CAPÍTULO 1 ·

Programando com IA: Claude Code, Codex, Cursor e cia

> *Em menos de dois anos, programar com IA deixou de ser "aceitar sugestões de autocomplete" e virou coordenar agentes que leem o repositório, rodam comandos, escrevem testes e abrem pull requests. Este capítulo explica o que é um harness, compara as principais ferramentas, apresenta os fluxos de trabalho que funcionam (plan mode, Spec-Driven Development, TDD, hooks, worktrees, CI), discute custos e separa "vibe coding" de engenharia. A tese que atravessa tudo vem do Akita: os modelos ficaram "competentes o suficiente"; o diferencial voltou a ser disciplina de engenharia.*

Para começar

Pense num estagiário muito rápido que leu quase todo o código já publicado, mas não conhece o seu projeto e às vezes inventa coisas com total confiança. Claude Code, Codex, Cursor e similares dão a esse estagiário acesso ao seu computador: ele lê arquivos, roda comandos e propõe mudanças. Os conceitos de LLM e de contexto estão no Livro 1, Capítulo 2 (LLMs e Transformers) e no Livro 2, Capítulo 2 (Engenharia de Contexto); agentes e MCP, no Livro 2, Capítulo 4 (Agentes e MCP).

De autocomplete a agente

Para começar

O GPS primeiro sugeria a próxima curva; hoje você diz o destino e ele planeja a rota. As ferramentas de IA para código seguiram o mesmo caminho: começaram completando palavras e hoje recebem um objetivo e executam vários passos sozinhas.

Gerar código a partir de linguagem natural é mais um capítulo da história da **tradução** em computação: do compilador A-0 de Grace Hopper (1952), passando pelas IDEs com IntelliSense, até o GitHub Copilot (2021). O vídeo What is an AI Code Generator? LLM Coding, Productivity, & Risk, da IBM, resume: *a IA não aprendeu a escrever código, aprendeu a traduzi-lo*, inclusive no sentido inverso, explicando uma função legada ou migrando COBOL para Java.

Na conversa Designing Claude Code, a designer-chefe do Claude Code descreve quatro degraus de abstração: **linha** (autocomplete), **tarefa** (chat na IDE que edita arquivos), **PR** (agente que planeja, edita, testa e entrega uma mudança revisável) e **projeto** (vários agentes em paralelo, tarefas longas, execução em CI). O vídeo Agentic Engineering vs Software Engineering: Beyond Vibe Coding, da IBM, mede o mesmo espectro pela agência delegada, terminando na **engenharia agêntica**: projetar o ambiente — regras, sen-

sores, permissões — em que agentes operam. É legítimo prototipar no modo vibe e migrar para engenharia agêntica conforme o risco sobe.

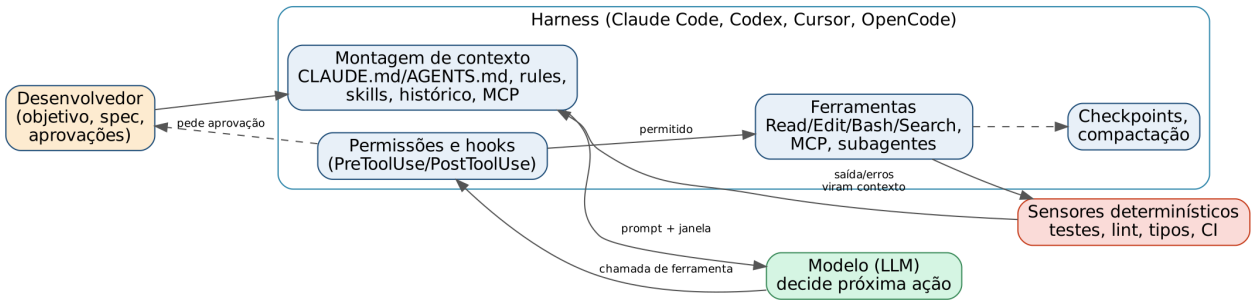
O Claude Code nasceu como CLI por pragmatismo, e a escolha virou vantagem: o terminal já está no fluxo de todo desenvolvedor e "texto entra, texto sai" é a forma natural de um LLM. Hoje o mesmo motor roda em terminal, IDE, desktop, web e celular.

> **Você sabia?**
>
> Programar "conversando" com a máquina é um sonho antigo: em 1955 a equipe de Grace Hopper começou o FLOW-MATIC, linguagem com comandos em inglês que inspirou o COBOL. Os prompts em linguagem natural levam a ideia ao extremo.

Anatomia de um agente de código: modelo + harness

> **Para começar**
>
> Um cavalo forte não lavra um campo sozinho: precisa de arreio e de quem o guie. O **modelo** (o LLM) é o cavalo; o **harness** ("arreio") é o programa ao redor que lê arquivos, roda comandos e devolve resultados. Essa divisão explica por que a mesma IA rende tão diferente em times diferentes.

O modelo só decide o próximo passo: qual ferramenta chamar, com quais argumentos, ou que texto responder. Todo o resto é harness: ler e escrever arquivos, rodar comandos, aplicar permissões, salvar checkpoints, compactar a conversa, disparar hooks, conectar servidores MCP e montar a janela de contexto a cada volta do loop. Trocar de modelo muda as *decisões*, não as *ferramentas*.



O loop agêntico: o harness monta o contexto, o modelo escolhe uma ação, permissões e hooks filtram, ferramentas executam e sensores determinísticos devolvem evidências ao contexto. Fonte: elaboração própria.

No workshop App do Zero com IA: Harness, Sub Agents, Spec-Driven, Novas Skills e Workflows, Waldemar Neto amplia o termo: em sentido amplo, harness é tudo o que você põe ao redor do modelo — **guias** (AGENTS.md, skills, exemplos) e **sensores** determinísticos (testes, lint, tipos, CI, hooks). O Akita, em "LLMs + Agentes = competente o suficiente", dá o pano de fundo: depois do o1 (2024) e do ano da eficiência (2025: MoE, *prompt caching*, janelas de 1M), o ganho recente veio do harness, não de modelos maiores. A distância entre o 1º e o 10º colocado no Chatbot Arena caiu de 12% para 5% em um ano, enquanto a METR mede que o horizonte de tarefas que agentes completam com

50% de confiabilidade dobra a cada ~7 meses. O corolário: **sem um "oráculo" (documentação, testes, compilador), o loop gira no vazio**.

Os mecanismos de extensão

Os harnesses convergiram para mecanismos quase idênticos. Na nomenclatura do Claude Code (Codex e OpenCode têm equivalentes):

- **CLAUDE.md / AGENTS.md**: instruções sempre carregadas, como o manual do novato no primeiro dia. Ex.: "rode os testes com `npm test`". Ideal: até ~200 linhas.
- **Rules por caminho**: regras que só valem em certas pastas (ex.: `api/**`). Ex.: uma regra para o site, outra para o servidor.
- **Skills** (`SKILL.md`): uma receita passo a passo que o agente só lê quando precisa. Se tem passos numerados, é skill.
- **Subagentes**: um ajudante com contexto próprio que devolve só a conclusão. Ex.: procurar onde um erro aparece no projeto.
- **Hooks**: comandos que rodam sozinhos em certos momentos, sem gastar contexto. Ex.: formatar o arquivo depois de cada edição.
- **MCP**: protocolo para ligar ferramentas externas (GitHub, Figma, navegador). Ex.: o agente abre uma issue sozinho.

Plugins empacotam skills, agentes, hooks e MCPs. Uma skill mínima:

```
---
name: summarize-changes
description: Use quando o usuário pedir um resumo do que mudou no branch atual.
---
1. Rode `git diff main...HEAD --stat` e depois o diff completo.
2. Agrupe as mudanças por módulo; aponte migrations e mudanças de API pública.
3. Liste riscos de regressão e testes que faltam.
```

> **Atenção**
> Baixar "500 skills" do GitHub tem três problemas: cada descrição ocupa contexto permanentemente, skills genéricas não conhecem o seu negócio e fora da sua área você não reconhece "como é o bom". Corey McClain, em How To Build Claude Skills Better Than 99% of People, propõe um teste: se você não consegue desenhar num guardanapo o fluxo da skill, não sabe o que ela faz. Leia toda skill de terceiros antes de instalar: é texto que entra no prompt com poder de executar comandos.

Comparativo das ferramentas

> **Para começar**
> Escolher uma ferramenta de IA para programar lembra escolher um carro: todos andam, mas mudam conforto, consumo e personalização. O objetivo aqui não é eleger uma vencedora, e sim mostrar critérios que continuam valendo quando a liderança muda.

As ferramentas copiam recursos umas das outras em semanas. O quadro em outubro de 2026:

- **Claude Code** (terminal, IDE, desktop, web; modelos Claude): o ecossistema mais completo de skills, hooks e plugins. Ressalva: os limites do plano acabam rápido em uso intenso.
- **Codex** (OpenAI; terminal, desktop, nuvem): rende mais horas pelo mesmo preço. Ressalva: menos extensões.
- **Cursor** (editor baseado no VS Code; vários provedores): plano visual e navegador embutido para conferir a tela. Ressalva: o custo varia com o modelo.
- **OpenCode 2.0** (open source; qualquer provedor, inclusive modelos locais): a mesma sessão no terminal, no desktop e no celular. Ressalva: as travas de segurança não cobrem tudo o que o próprio app faz.
- **GitHub Copilot** (extensões de IDE): barato e presente em quase todo editor, com modo agente. Ressalva: os modelos gratuitos são limitados.

O mesmo harness serve para além do código: o método CHIA, mostrado em transformei o Claude Code no SISTEMA OPERACIONAL da minha empresa (e mostrei como criar o teu), roda um negócio inteiro "em pastas e arquivos", com dezenas de CLAUDE.md aninhados e skills.

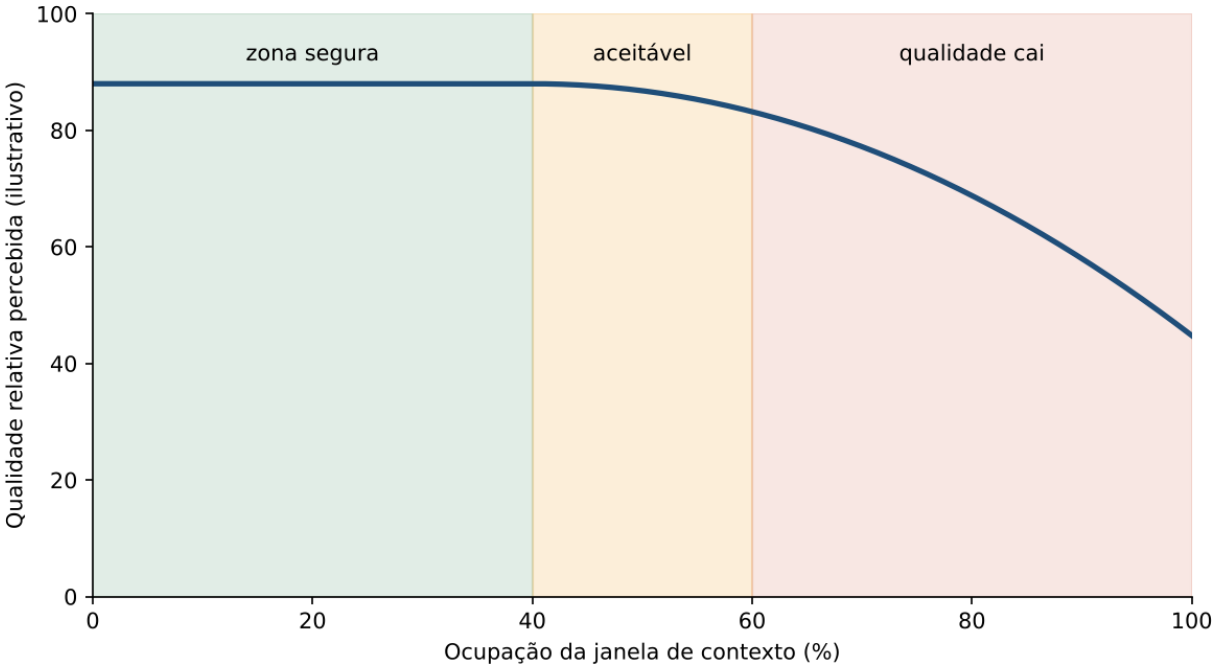
> **Dica**
> O critério que mais importa não é o benchmark do mês, e sim: (1) quanto você controla o contexto, (2) se há hooks e permissões para fechar o loop com sensores, (3) custo previsível. Um AGENTS.md bem escrito e uma boa suíte de testes migram com você de ferramenta. Desconfie de harnesses desconhecidos que pedem credenciais ou permissões amplas.

Contexto: o recurso mais escasso

> **Para começar**
> O **contexto** de um LLM é como uma mesa de trabalho: tudo o que o modelo "vê" de uma vez. É medido em **tokens** (pedaços de palavra; mil tokens ≈ 750 palavras em inglês) e tem tamanho limitado. Mesa cheia custa mais e piora o resultado.

LLMs são **stateless**: a cada volta do loop, o harness reenvia a janela inteira — system prompt, ferramentas, CLAUDE.md, histórico, saídas de comandos. No vídeo Como EXTRAIR o MÁXIMO do seu plano Claude Code!, um simples "oi" numa sessão nova consumiu 38 a 48 mil tokens só com skills, plugins e MCPs carregados.

E quanto mais cheia a janela, pior a atenção do modelo (*context rot*). Os praticantes convergem: até ~40% de ocupação é seguro, até ~60% aceitável; acima disso paga-se em qualidade. Os sintomas: o agente esquece instruções, repete erros corrigidos e relê arquivos.



Faixas de ocupação da janela relatadas por praticantes (curva ilustrativa, não medida). Fonte: elaboração própria.

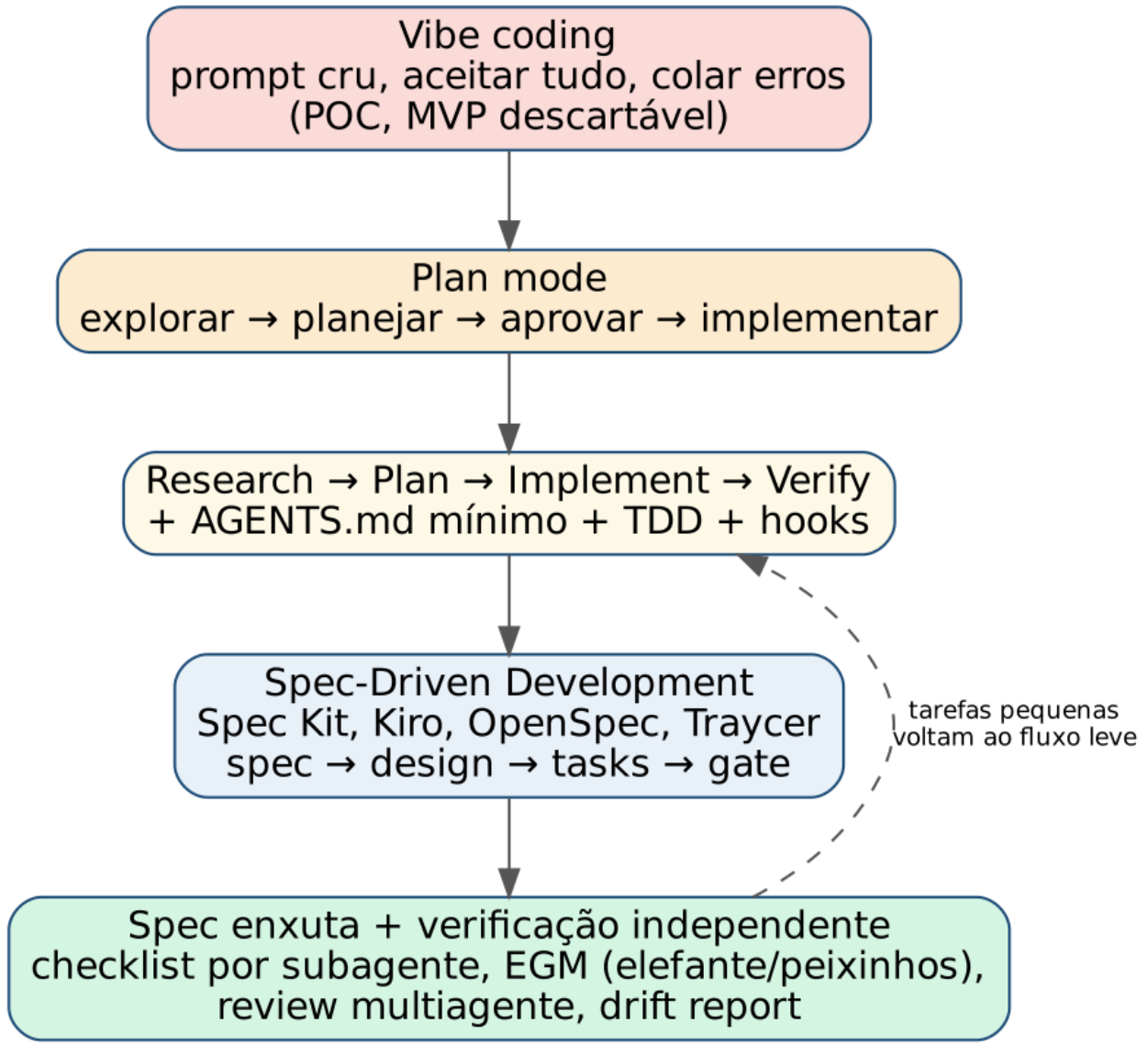
- `/clear` **entre tarefas não relacionadas**; o CLAUDE.md garante que a orientação básica sobrevive.
- **Subagentes para exploração**: leem dezenas de arquivos e devolvem só a conclusão.
- **Handoff em vez de** `/compact`: muitos preferem abrir sessão nova com objetivo, decisões e próximo passo escritos.
- **Comece no menor diretório possível** (só o backend de um monorepo, por exemplo) e adicione outros quando precisar.
- **Desligue o que não usa** (plugins, MCPs ociosos) e **filtre saídas**: logs de 300 linhas no contexto são desperdício.

"Mais contexto" não é "melhor contexto": o objetivo é o contexto *certo* — o padrão a imitar, o arquivo relevante, o teste que falha.

Fluxos de trabalho

> **Para começar**
> Um bom pedreiro olha a planta antes de assentar tijolo e confere o prumo a cada fiada. Com agentes é igual: planejar, especificar, testar e automatizar verificações. Começamos pelo fluxo mais simples e subimos degrau a degrau.

A figura a seguir organiza os fluxos numa escada de maturidade. Não é progressão obrigatória: o erro é aplicar o degrau errado — burocracia de SDD num bug de float, ou vibe coding num módulo de pagamentos.



Escada de fluxos de trabalho com agentes, do vibe coding à spec enxuta com verificação independente. Fonte: elaboração própria.

Plan mode: o fluxo-base

O núcleo é **Research → Plan → Implement → Verify**. O plan mode deixa o agente apenas ler até você aprovar um plano, porque *"o erro mais caro não é o código errado, é o código certo para o problema errado"*. Em ESCREVA PROMPTs MELHORES | Dicas simples para melhorar a qualidade do código gerado pela IA, Fernanda Kipper mostra a diferença: "refatore para ficar mais organizado" só moveu arquivos de pasta; nomear o padrão (Strategy), citar arquivos e rotas e usar plan mode gerou perguntas, um plano em tarefas e a interface de pagamento esperada.

- Planeje num contexto zerado, com o modelo mais forte; implemente com um mais barato.
- Peça que o agente pergunte **uma coisa por vez** e comente o plano antes de aprovar.
- Explique o **porquê** e mostre em vez de descrever: *o modelo imita bem o que você mostra e inventa mal o que só descreve*.

Para features maiores, Augusto Galego, em O Workflow Correto para Desenvolver Software com IA, propõe sete fases: ideia → pesquisa → protótipo → spec/PRD → tickets pequenos → execução → revisão. Sobre tamanho, os números se repetem: PRs de ~300 linhas; acima de ~500 linhas de lógica, quebre a tarefa com fronteiras explícitas.

Spec-Driven Development (SDD)

SDD responde às dores do vibe coding — intenção perdida no chat, "pronto" como opinião da IA — tornando a **especificação versionada no repositório** o contrato do qual derivam código, testes e validação. Ferramentas como GitHub Spec Kit (constitution → specify → plan → tasks → implement), AWS Kiro, OpenSpec e Traycer seguem variações desse fluxo. Uma boa spec tem critérios de aceite em **Dado/Quando/Então**, suposições e perguntas em aberto explícitas e tamanho proporcional à feature. Erros típicos, listados em SPEC Driven: A MELHOR FORMA De Usar IA em 2026!: critério vago, suposições escondidas, tarefa concluída sem teste passando e spec sem auditor.

A crítica vem do workshop de Waldemar Neto: spec → design → tasks atômicas ajuda modelos fracos, mas para modelos fortes vira *babysitting*, e a spec sofre **drift**. A alternativa "Spec Lean" funde spec e design num plano único, extrai dele um **checklist de verificação** rodado por subagente e deixa o modelo implementar.

O defeito estrutural de SDD com um único agente é a **autoconcordância**: o mesmo modelo escreve e revisa, e a bajulação deixa buracos. O **Elephant Goldfish Model**, publicado por engenheiros do Google (apresentado em Spec Driven Development 2.0: O método do Google para engenharia com Agentes de IA), separa os papéis: o "elefante", com contexto total, escreve a spec; "peixinhos-dourados", subagentes de contexto zero, a revisam por ângulos diferentes (simplicidade, escala, ambiguidade). É burocrático para tarefas pequenas, mas o princípio vale sempre: **quem implementa não valida o próprio trabalho**.

Nas demonstrações publicadas, a revisão pelos "peixinhos" pegou bloqueios reais que o autor da spec não viu: um professor que só enxergava as aulas do dia e não conseguia cancelar a de quinta; lojistas que podiam apagar produtos já vendidos quando a regra era apenas ocultá-los. O EGM também gera um **drift report**, que compara spec e código depois das mudanças, atacando o problema de a especificação envelhecer em silêncio.

> **Dica**
>
> Mesmo sem nenhuma ferramenta de SDD, um subagente revisor com contexto limpo e instruções adversariais ("procure o que falta, não o que está certo") custa poucos tokens e quebra a autoconcordância.

TDD com agentes

A prática mais recomendada é dar ao agente um **método de verificação**; testes são o oráculo que faz o loop convergir:

1. Peça os testes a partir dos critérios de aceite e confirme que falham pelo motivo certo.
2. Faça commit dos testes — isso impede o atalho mais comum: alterar o teste para passar.
3. Peça a implementação com "não modifique os testes".
4. Refatore com a suíte como rede de segurança.

Para UI, valide comparando screenshot com o design (Playwright MCP, navegador embutido). E cada bug corrigido vira teste de regressão.

> **Atenção**
>
> Teste verde não prova correção. Em Rust é a melhor linguagem pra IA?, Daniel Bergholz mostra a troca de contagem de caracteres por `.len()`, que em Rust conta **bytes** ("ç" ocupa 2 em UTF-8): compila, passa no linter e nos testes, e está errado. Casos de borda continuam sendo responsabilidade humana.

Hooks: tornando o processo determinístico

Instruções em CLAUDE.md são *probabilísticas*; hooks são *determinísticos*. Regra prática: **se algo precisa acontecer toda vez, não peça — configure um hook**. Os eventos principais são `PreToolUse` (pode bloquear), `PostToolUse` (formatar e lintar) e `Stop` (exigir testes verdes ao encerrar).

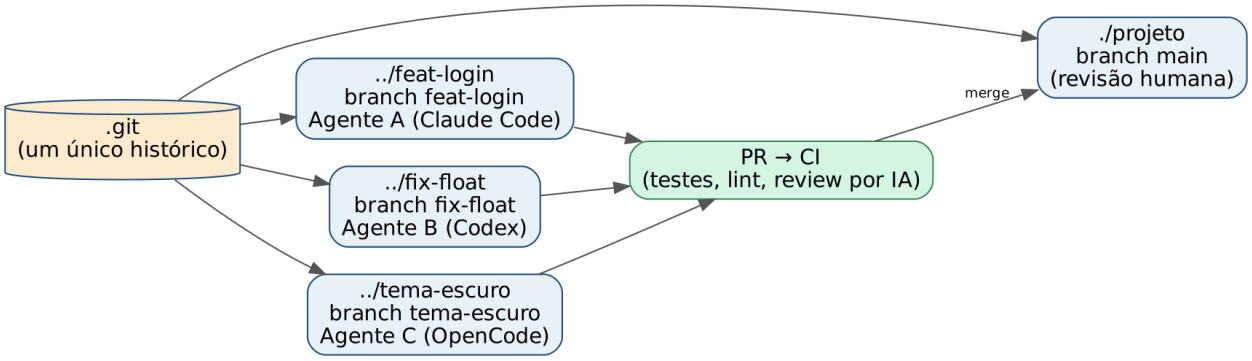
```
// .claude/settings.json (trecho)
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{ "type": "command", "command": "npx prettier --write \"$CLAU-
DE_FILE_PATHS\"" }]
    }],
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{ "type": "command", "command": ".claude/hooks/block-destructive.sh" }]
    }]
  }
}
```

Fora do harness, os equivalentes clássicos são ainda mais robustos: pre-commit hooks, branch main protegida e CI obrigatória. Comandos úteis do Claude Code: `/init` (gera o CLAUDE.md), `/context` (uso da janela), `/clear` e `/rewind` (gestão de contexto), `/model` (modelo e esforço), Shift+Tab (plan mode) e `/code-review`.

Worktrees e paralelismo

Dois agentes editando a mesma pasta se atropelam. O `git worktree`, tema de Você não pode se dar ao luxo de ignorar Git-Worktree, resolve: um histórico, várias pastas de trabalho, cada uma numa branch.

```
git worktree add ../feat-login -b feat-login main   # nova pasta + branch
claude --worktree feat-x                            # Claude Code cria a sua
git worktree remove ../feat-login
```



Um repositório, várias worktrees, um agente por worktree, e a CI como funil antes do merge. Fonte: elaboração própria.

O limite não é técnico, é cognitivo: **1 agente é bom, 2 ótimo, 3 às vezes; com 4 ou mais, o humano vira o gargalo**. A reescrita do Bun de Zig para Rust com dezenas de agentes mostra o outro extremo e seu preço: regressões, milhares de blocos `unsafe` e críticas de código não revisado.

> **Você sabia?**
>
> O `git worktree` existe desde o Git 2.5, de 2015, muito antes dos agentes de código. Ficou anos como recurso de nicho e virou peça central quando passou a ser comum rodar vários agentes no mesmo repositório.

Headless e CI

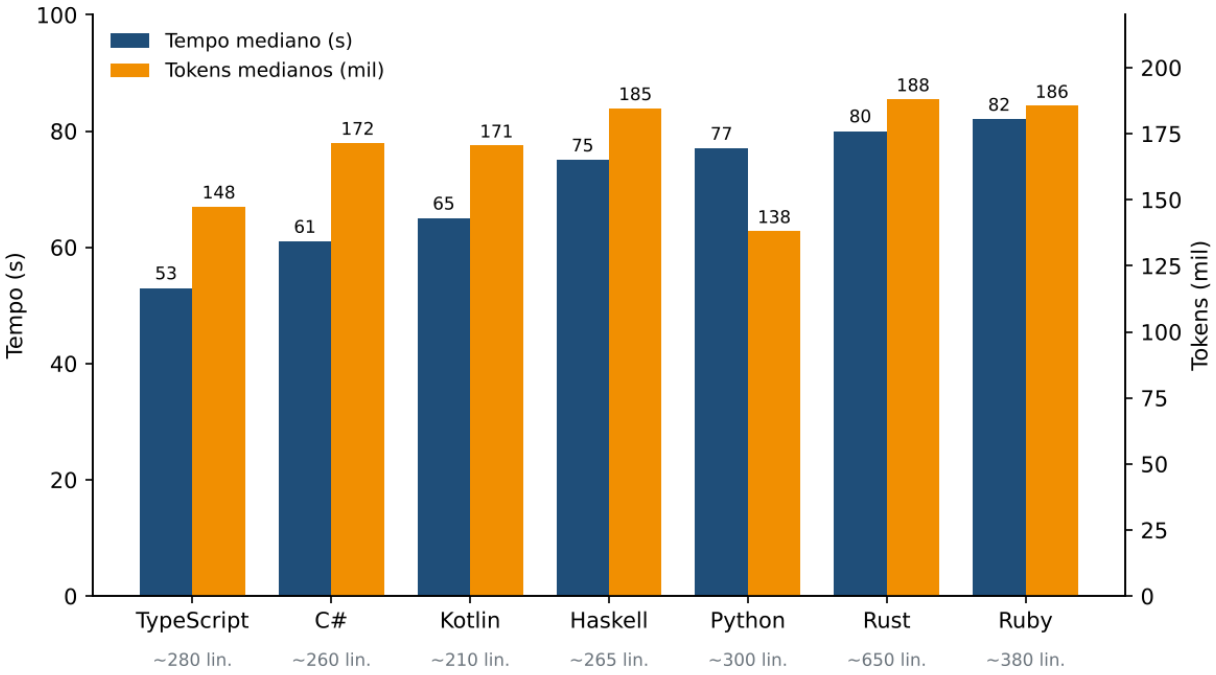
O mesmo agente roda sem interface (`claude -p`, `codex exec`, GitHub Actions), com permissões restritas a ferramentas pré-aprovadas. Isso abre revisão automática de PR, triagem de issues e migrações em lote. O vídeo What Is AI Code Refactoring? Agentic AI & Safe Code Changes, da IBM, descreve o loop seguro: plano → relatório → **aprovação humana** → snapshot → patch → verificar → rollback se quebrar. Na revisão, o padrão emergente é o **review multiagente**: subagentes especializados em segurança, requisitos, testes e arquitetura, com severidades *blocker / should-fix / nit*. Há times que rodam um loop em que um agente revisor comenta o PR e o agente autor corrige até não restarem bloqueios, com o humano entrando só no final. A IBM, em What Is AI Code Review? Fixing Slow PRs & Broken Workflows with AI, lembra que a revisão automatizada combina camadas (análise estática, testes dinâmicos, linters, LLM contextual) e que se deve medir sinais — taxa de defeitos, tempo de revisão, vulnerabilidades — em vez de confiar na sensação.

Linguagem importa? O benchmark do Akita

> **Para começar**
> Para quem cozinha bem, o idioma da receita pouco importa. O experimento a seguir comparou sete linguagens e chegou a uma lição parecida: o que mais pesa é ter testes que digam ao agente se ele acertou.

No post Existem linguagens melhores pra programar com LLMs?, o Akita rodou 7 linguagens × 4 modelos × 2 condições (sem testes e com 27 testes públicos) × 2 repetições num mesmo problema com 43 histórias de usuário.

Linguagem	Tempo mediano (s)	Tokens medianos (mil)	Linhas (~)
TypeScript	53	147,5	280
C#	61	171,6	260
Kotlin	65	170,6	210
Haskell	75	184,7	265
Python	77	138,2	300
Rust	80	188,1	650
Ruby	82	185,6	380



Tempo e tokens medianos por linguagem e tamanho aproximado da solução. Fonte: Akita, "Existem linguagens melhores pra programar com LLMs?".

- **Não há ranking geral.** TypeScript foi o mais rápido, Python o mais econômico em tokens, Rust gerou mais que o dobro de linhas.
- **Tipagem forte não reduziu reparos.** Os erros restantes eram de **invariantes de domínio**, que nenhum sistema de tipos comum captura.
- **A suíte de testes fez a diferença**: 24 artefatos corretos com testes contra 16 sem. Síntese: *"disciplina vence compilador e vence linguagem"*.

Compiladores com mensagens claras, como o de Rust, ajudam o loop (“para um agente, não é erro, é instrução”), mas cobram em tempo de build e código difícil de revisar. Note a distinção: o Cursor migrou seu agente de terminal para Rust por motivos de distribuição (instalar sem Node.js, menos memória) — isso é “Rust para construir o agente”, não “agente escrevendo Rust”. Ao escolher a linguagem, pergunte: o erro aparece rápido e claro? existe um caminho padrão (framework opinativo)? o modelo conhece bem a linguagem e suas APIs atuais? e, antes de tudo, ela resolve o seu problema?

Custos, tokens e escolha de modelo

> **Para começar**
> Usar um modelo é como andar de táxi: você paga pelo que entra e pelo que sai, contado em tokens. Como o agente relê a conversa inteira a cada passo, o taxímetro corre mais rápido do que parece.

Como cada turno reenvia o histórico, o custo de uma sessão cresce aproximadamente com o **quadrado** do número de turnos se nada for limpo. Duas consequências: **uma mensagem completa é mais barata que várias picadas**; e o **cache de prompt** muda a conta — dentro da janela de cache (5 minutos por padrão), o prefixo custa ~10% do preço de entrada. Com Opus a US$ 5 / US$ 25 por milhão de tokens (entrada/saída), uma sessão que relê 200k tokens 50 vezes custa ~US$ 50 sem cache e ~US$ 5 com cache, antes da saída.

Assinaturas (US$ 20, 100, 200) têm limites por janelas de 5 horas e 7 dias; relatos convergem que o Codex rende mais horas pelo mesmo valor. Do lado aberto, o LLM Benchmark v4 do Akita mostra modelos abertos (DeepSeek, GLM, Qwen) com 92,5 pontos, contra 100 dos fechados de topo. As lições: tamanho não garante qualidade, a configuração do *serving* pesa mais que a quantização e o ruído do benchmark é de ~10 pontos.

Papel	Modelo típico	Esforço
Arquitetura, plano grande, depuração difícil	O mais forte disponível	high
Implementação, subagentes, headless	Modelo médio ou aberto bom	medium
Buscas, resumos, tarefas mecânicas	Modelos pequenos ou locais	low

Regra de ouro: **corrija o contexto antes de aumentar o esforço**. Se o agente erra porque não viu o arquivo certo, esforço máximo só erra mais caro. Escrever menos código também economiza: skills como a Ponytail, testada em LANÇOU Ponytail 5! Agora tua IA vai economizar MUITO TOKEN! (teste completo), forçam perguntas antes de codar (precisa existir? já existe no projeto?) e, num teste único, cortaram custo em 42% e linhas em dois terços. Um teste é anedótico; o princípio — YAGNI aplicado ao agente — é sólido.

> **Você sabia?**
> O primeiro “bug” documentado era literal: em 9 de setembro de 1947, a equipe de Grace Hopper achou uma mariposa presa num relé do Harvard Mark II e a colou no diário de bordo. A palavra já existia no jargão de engenharia; a mariposa só lhe deu a foto mais famosa.

Vibe coding vs. engenharia

> **Para começar**
> Fazer bolo na intuição funciona em casa, não numa confeitaria. *Vibe coding* é programar com IA aceitando o que ela gera sem ler o código. Aqui vemos os erros típicos quando ninguém revisa e por que um agente pode ser enganado.

O termo, cunhado por Andrej Karpathy em fevereiro de 2025, descreve "esquecer que o código existe". A definição estrita importa: **vibe coding é não ler o código**. Tem lugar em POCs, protótipos e scripts descartáveis; é armadilha em produção e para quem está aprendendo.

Em Revisei código gerado por I.A..... PREOCUPANTE, Augusto Galego revisa um app de tarefas (to-do) gerado sem guia:

Erro da IA	Por que importa	Correção
Sem login: cada pessoa vê as tarefas de todas	Falha de autorização	Requisitos na spec
SELECT * sem paginação (traz tudo de uma vez)	Lentidão e vazamento de dados	Paginação obrigatória
Router (o "roteador" de páginas) feito do zero	Perde proteções prontas	Usar um framework conhecido
Sem migrations	Quebrou ao adicionar autenticação	Ferramenta de migrations
JWT (o "crachá" de login) com senha padrão	Crachás falsificáveis	Fixar algoritmo, secret via ambiente

A IBM, em How AI Coding Agents Understand Your Codebase & Developer Tools, mostra o mesmo em escala corporativa: um endpoint que consulta o banco direto funciona, mas pula a camada de serviço. *"O código funciona, mas o sistema piorou."* Os números de segurança apontam na mesma direção: a Veracode (2025) encontrou vulnerabilidades em ~55% do código gerado nos testes — código limpo, bem nomeado e sutilmente errado, o que a IBM chama de **ilusão de correção**. Os exemplos são os de sempre: SQL por concatenação, senha em texto puro, segredos no código. Para uso corporativo, somam-se perguntas de governança: de onde vieram os dados de treino (risco de contaminação por licenças como GPL)? para onde vai o meu código? posso auditar? há opção on-premises para exigências regulatórias?

Um agente com **dados sensíveis**, **exposição a conteúdo externo** e **capacidade de enviar dados para fora** reúne a "tríade letal" (Simon Willison): basta um PR, issue ou página com instruções maliciosas (*prompt injection*) para vazar credenciais. Defesas: modo de permissão adequado, com o modo sem confirmações **apenas em contêiner ou VM descartável**; ações irreversíveis (push, deploy, apagar dados) sempre com aprovação; segredos fora do alcance do agente; MCPs e skills de terceiros revisados como dependências.

Produtividade real

> **Para começar**
> “Parecer mais rápido” e “chegar antes” são coisas diferentes. Esta seção separa os números de produtividade pelo tipo de evidência.

Afirmação	Fonte	Tipo de evidência
+35% de produtividade	IBM	Autorrelato
66% se frustram com soluções “quase certas”	Stack Overflow Developer Survey	Pesquisa de opinião
Airbnb: 3.500 arquivos de teste migrados em 6 semanas, e não em 1,5 ano	Relato da empresa	Tarefa mecânica
Testes: 24 vs. 16 artefatos corretos	Benchmark do Akita	Experimento (n pequeno)
Devs experientes ~19% **mais lentos**, achando-se ~20% mais rápidos	METR (jul/2025)	Estudo controlado

Os modelos melhoraram desde o estudo da METR, mas a lição permanece: **percepção de produtividade não é produtividade**. Meça lead time, defeitos e retrabalho. A síntese honesta: *IA acelera, mas não facilita*. O gargalo migrou da escrita para a **verificação**, por isso o agente deve produzir evidências (testes, análise de impacto) e não só código. O ganho é grande em tarefas mecânicas e bem especificadas; em código maduro, depende de quanto das invariantes você tornou explícito.

“Competente o suficiente”: um checklist

> **Para começar**
> Os modelos já são “competentes o suficiente”; o que separa resultado bom de ruim é a disciplina ao redor deles. Eis uma lista que cabe na parede.

A tese do Akita amarra o capítulo: os modelos de fronteira convergiram (diferenças dentro do ruído dos benchmarks), modelos abertos chegaram ao tier A e os ganhos dos últimos 18 meses vieram do harness. Logo, o que separa resultado bom de ruim é a engenharia ao redor: testes, processo, invariantes explícitas.

O contraexemplo do Akita reforça a tese: em Treinei meu próprio modelo de IA, um modelo de 337M parâmetros fez 58/100 em problemas isolados e **zero** em tarefas agênticas. É preciso escala de modelo *e* um bom loop; o ganho marginal agora está no loop.

1. **Contexto antes de prompt**: AGENTS.md curto, só com o que o modelo não descobre sozinho.
2. **Regras concretas**: “nunca use `any`; nunca delete testes sem aprovação” em vez de “use clean code”.
3. **Plano antes de código** em tudo que não for trivial.
4. **Tarefas pequenas**, com fronteiras e o “porquê”.
5. **Verificação**: testes commitados antes, hooks, CI obrigatória, branch protegida.

6. **Revisão independente**: quem implementa não valida; humano lê o que vai para produção.
7. **Higiene de contexto e modelo por alavancagem**: `/clear`, subagentes, modelo forte só para planejar.
8. **Paralelismo com moderação** e **segurança**: 2 ou 3 agentes; permissões amplas só em sandbox.
9. **Meça** tempo até merge, defeitos e custo por tarefa, não a impressão de velocidade.

> **Dica**
>
> Comece pequeno: um CLAUDE.md, um hook de testes e o hábito do plan mode resolvem a maior parte. Acrescente SDD e review multiagente quando a dor aparecer.

Fontes deste capítulo

Título	URL	Canal / autor	Ponto principal
What is an AI Code Generator? LLM Coding, Productivity, & Risk	https://www.youtube.com/watch?v=oanQrXEiCy4	IBM Technology	IA como tradução; ilusão de correção
Designing Claude Code	https://www.youtube.com/watch?v=vLIDHi-1PVU	Anthropic	Terminal como produto; escada de abstração
Agentic Engineering vs Software Engineering: Beyond Vibe Coding	https://www.youtube.com/watch?v=FgaBdwSvOGM	IBM Technology	Espectro de agência delegada
App do Zero com IA: Harness, Sub Agents, Spec-Driven, Novas Skills e Workflows	https://www.youtube.com/watch?v=yKLedmyUDMA	Waldemar Neto - Dev Lab	Harness amplo; crítica ao SDD clássico
How To Build Claude Skills Better Than 99% of People	https://www.youtube.com/watch?v=aRVqTkyx4IY	Corey McClain	Diagramar skills revela erros
transformei o Claude Code no SISTEMA OPERACIONAL da minha empresa (e mostrei como criar o teu)	https://www.youtube.com/watch?v=7btZsF-9UDA	Ratos de IA	Empresa inteira operada via Claude Code
Como EXTRAIR o MÁXIMO do seu plano Claude Code!	https://www.youtube.com/watch?v=4vWd6z9zuCQ	Dev Aprender | Jhonatan de Souza	Gestão de contexto, cache e modelo
ESCREVA PROMPTs MELHORES | Dicas simples para melhorar a qualidade do código gerado pela IA	https://www.youtube.com/watch?v=fRoRAe8j8eo	Fernanda Kipper	Prompt específico + plan mode vence vago
O Workflow Correto para Desenvolver Software com IA	https://www.youtube.com/watch?v=LmUsdFWuaGg	Augusto Galego	Sete fases do desenvolvimento com IA
SPEC Driven: A MELHOR FORMA De Usar IA em 2026!	https://www.youtube.com/watch?v=ZM8WW8_HEeE	O Novo Programador	Spec versionada como contrato central
Spec Driven Development 2.0: O método do Google para engenharia com Agentes de IA	https://www.youtube.com/watch?v=xJwegOxj2sc	IA em 6 minutos	Elephant Goldfish Model contra autoconcordância
Rust é a melhor linguagem pra IA?	https://www.youtube.com/watch?v=ydQ7HhnXUZk	Daniel Bergholz	Compilador ajuda o loop, mas não basta
Você não pode se dar ao luxo de ignorar Git-Worktree	https://www.youtube.com/watch?v=qZQwxeCLdN0	Augusto Galego	Worktrees para agentes em paralelo
What Is AI Code Refactoring? Agentic AI & Safe Code Changes	https://www.youtube.com/watch?v=f84XbvASkk4	IBM Technology	Refatoração agêntica com aprovação e rollback
LANÇOU Ponytail 5! Agora tua IA vai economizar MUITO TOKEN! (teste completo)	https://www.youtube.com/watch?v=RcnnOS0YXak	Matheus Battisti - Hora de Codar	Menos código, menos custo
Revisei código gerado por I.A..... PREOCUPANTE	https://www.youtube.com/watch?v=u1nhcio8_iQ	Augusto Galego	Defeitos de um app gerado sem guia
How AI Coding Agents Understand Your Codebase & Developer Tools	https://www.youtube.com/watch?v=zAe-sau06io	IBM Technology	Código funciona, mas o sistema piora
What Is AI Code Review? Fixing Slow PRs & Broken Workflows with AI	https://www.youtube.com/watch?v=0-YCleDw-Po	IBM Technology	Revisão automatizada em camadas; medir sinais

Título	URL	Canal / autor	Ponto principal
LLMs + Agentes = competente o suficiente (12/10/2026)	https://akitaonrails.com/2026/10/12/llms-agentes-competente-o-suficiente/	AkitaOnRails (blog)	Ganho recente veio do harness
Existem linguagens melhores pra programar com LLMs? (11/10/2026)	https://akitaonrails.com/2026/10/11/existem-linguagens-melhores-pra-programar-com-llms/	AkitaOnRails (blog)	Testes importam mais que tipagem
LLM Benchmark v4 — veredito open source (22/10/2026)	https://akitaonrails.com/2026/10/22/llm-benchmark-v4-o-veredito-de-open-source-ate-out-2026/	AkitaOnRails (blog)	Abertos no tier A; serving pesa mais
Treinei meu próprio modelo de IA (14/10/2026)	https://akitaonrails.com/2026/10/14/treinei-meu-proprio-modelo-de-ia-spoiler-e-so-pra-estudo/	AkitaOnRails (blog)	Modelo pequeno zera tarefas agênticas
Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity (jul/2025)	https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/	METR	Devs ~19% mais lentos, percebendo-se mais rápidos

CAPÍTULO 2 ·

Hardware, GPUs e IA Local

> *Quem roda IA na própria máquina descobre logo que o gargalo quase nunca é "poder de computação" abstrato. O que limita é memória: quanta cabe na placa e com que velocidade ela entrega bytes ao processador. Este capítulo parte dessa ideia: a conta que diz se um modelo cabe, a física que diz quão rápido ele gera tokens, a quantização que encolhe modelos, os runtimes que os servem e o ecossistema além da NVIDIA. No fim, lembramos que uma GPU é um processador paralelo genérico, útil muito além dos LLMs.*

Para começar

Pense num modelo de IA como um livro enorme que o computador relê inteiro a cada palavra que escreve. A **GPU** é o leitor; a **VRAM** (memória da placa) é a mesa onde o livro fica aberto. Se o livro não cabe na mesa, nada funciona; se cabe, a velocidade depende de quão rápido o leitor vira as páginas. A decisão "assinatura ou hardware em casa?" está no Livro 5, Capítulo 1 (Negócios, Carreira e Governança); aqui fica a parte técnica.

Por que GPU? Um pouco de arquitetura

Para começar

Uma CPU é um pequeno grupo de engenheiros espertos que resolvem qualquer problema, um de cada vez. Uma GPU é um estádio cheio de pessoas com calculadora, todas fazendo a mesma conta sobre números diferentes. Para IA, que é basicamente multiplicar **matrizes** gigantes, o estádio vence.

A CPU tem poucos núcleos otimizados para latência, com previsão de desvios e caches grandes. A GPU tem milhares de unidades simples, organizadas em grupos (SMs na NVIDIA, CUs na AMD, Xe-cores na Intel) que executam a mesma instrução sobre muitos dados (modelo SIMT). Ela esconde latência com paralelismo em massa, não com cache. Desde 2017 as GPUs NVIDIA trazem **Tensor Cores**, unidades de produto de matrizes em precisões reduzidas (FP16, BF16, FP8, INT8 e, na Blackwell, FP4); AMD e Intel têm equivalentes. O vídeo AI Accelerators: Transforming Scalability & Model Efficiency, da IBM, acrescenta um padrão de sistemas: um modelo pequeno num acelerador próximo do processador decide a maioria dos casos em microssegundos, e só os duvidosos sobem para um modelo grande (cascata de modelos).

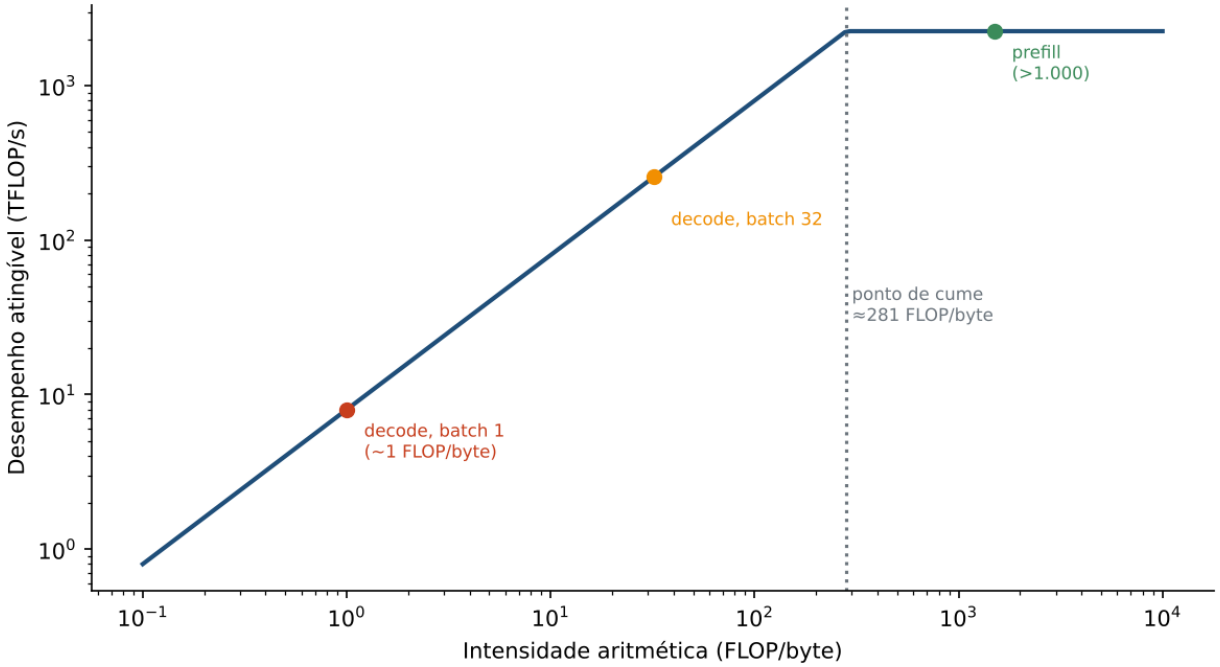
A parede de memória

Em cerca de vinte anos, a capacidade de cálculo por chip subiu milhares de vezes, enquanto a banda de memória cresceu bem menos. Esse descompasso é a **memory wall**, tema de The AI Memory Wall: Why LLMs & AI Accelerators Hit a Bottleneck. A ferramen-

ta para raciocinar sobre isso é o **modelo roofline**, baseado na **intensidade aritmética** (FLOPs por byte lido):

```
desempenho = min( pico_de_computação , banda_de_memória × intensidade )
ponto de cume (ridge) = pico_de_computação / banda_de_memória
```

Num B200, ~2,25 PFLOP/s ÷ 8 TB/s ≈ 281 FLOP/byte. Abaixo disso o trabalho é limitado por memória; acima, por computação.



Roofline de um B200. O decode com batch 1 fica à esquerda, limitado pela banda; o batching empurra o ponto para a direita, e o prefill já é limitado por computação. Fonte: elaboração própria com dados de "The AI Memory Wall: Why LLMs & AI Accelerators Hit a Bottleneck".

Um LLM (ver Livro 1, Capítulo 2 (LLMs e Transformers)) trabalha em duas fases:

- **Prefill** (processar o prompt): todos os tokens de entrada juntos, em multiplicações matriz×matriz. Limitado por computação; determina o **TTFT** (*time to first token*).
- **Decode** (um token por vez): cada token novo exige ler *todos* os pesos e o KV cache, com intensidade perto de 1 FLOP/byte. Limitado por banda; determina a latência entre tokens.

Em resumo

Num modelo denso com batch 1, o teto de geração é **tok/s ≈ banda de memória (GB/s) ÷ bytes lidos por token (GB)**, e os bytes por token são, grosso modo, o tamanho do arquivo do modelo. Uma placa com ~1,8 TB/s e um modelo de 16 GB tem teto de ~110 tok/s; na prática se obtém 60–80% disso.

Por isso o **batching** é crucial em servidores: pesos lidos uma vez servem dezenas de requisições. E por isso modelos **MoE** (Mixture of Experts) são atraentes localmente: um modelo com 30B parâmetros totais e 3B ativos precisa de memória para 30B, mas gera na velocidade de um de 3B.

Você sabia?

A NVIDIA lançou a GeForce 256 em 1999 chamando-a de “a primeira GPU do mundo” e, em 2006, apresentou o CUDA, que abriu a placa de vídeo para computação genérica. Foi essa aposta em software, mais que o silício, que pôs a empresa no centro da IA.

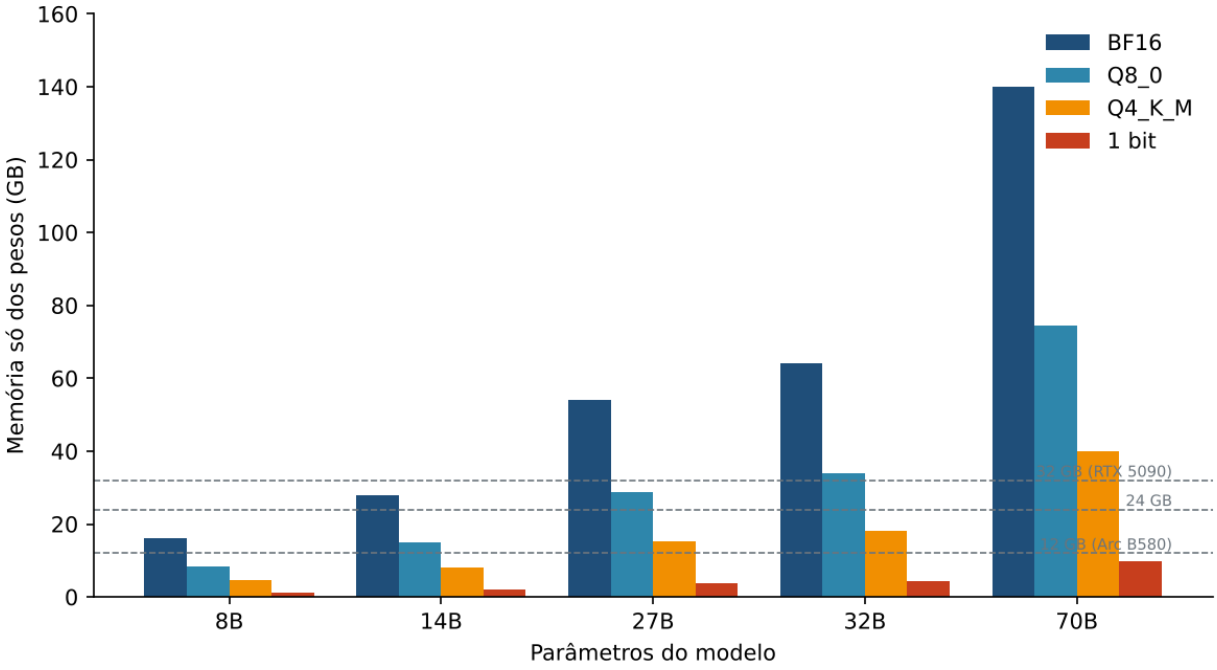
VRAM e tamanho de modelo: a conta de guardanapo

Para começar

Antes de comprar uma mala, você confere se ela cabe no bagageiro. Cada **parâmetro** de um modelo ocupa alguns bytes, e há bilhões deles; multiplicando, sabemos se o modelo cabe na VRAM.

```
memória dos pesos (GB) ≈ parâmetros (bilhões) × bytes por parâmetro
FP32 = 4 B   BF16/FP16 = 2 B   FP8/INT8 = 1 B   INT4/FP4 ≈ 0,5 B
memória total ≈ pesos + KV cache + buffers + overhead do runtime (≈ 0,5–1,5 GB)
```

Um modelo de 70B ocupa ~140 GB em BF16, ~70 GB em 8 bits e ~35 GB em 4 bits (What Is Quantization? How AI Models Get Smaller & Faster); um de 1 trilhão, ~2 TB em BF16. Na prática, some de 10% a 20% à conta ingênua: formatos reais guardam escalas por bloco (o Q4_K_M do llama.cpp usa ~4,8 bits por peso) e mantêm algumas camadas em precisão maior.



Memória ocupada só pelos pesos, por tamanho de modelo e formato. As linhas tracejadas marcam capacidades típicas de VRAM. Fonte: elaboração própria com a fórmula params × bytes.

O KV cache: a parte que todo mundo esquece

Para não recalcular a atenção sobre todo o histórico a cada token, o modelo guarda as chaves (K) e valores (V) de cada camada para cada token já processado. É trocar cálculo por memória:

```
KV (bytes) = 2 × camadas × cabeças_KV × dim_cabeça × tokens × bytes_por_elemento × usuá-
rios
```

Um modelo de ~8B com 32 camadas, 8 cabeças KV e dimensão 128, em FP16, gasta 128 KB por token: 4 GB com 32 mil tokens de contexto e 16 GB com 128 mil — mais que os próprios pesos quantizados. Localmente, reduza o contexto ao que você usa (`--ctx-size`, `num_ctx`) ou quantize o cache. Num servidor, a VRAM se divide entre pesos e cache: um modelo de 13B ocupa ~26 GB de uma placa de 40 GB, e só o resto sobra para os usuários e seus contextos. Os comandos são simples (`--cache-type-k q8_0` no llama.cpp; `--kv-cache-dtype fp8` no vLLM). Arquiteturas novas (MLA, janelas deslizantes, camadas híbridas) reduzem o cache por construção.

> **Dica**
>
> Regra de bolso: **pesos quantizados + 20–30% de folga** para KV cache e buffers. Placas de 12 GB rodam com conforto modelos densos de até ~14B em Q4 com 8–16k de contexto; 16 GB chegam a ~20–24B; 24 GB, a ~32B em Q4; 32 GB permitem ~32B em Q6/Q8 ou MoEs maiores com descarregamento.

Descarregamento e memória unificada

Quando o modelo não cabe, há duas saídas. O **descarregamento** (*offload*) deixa parte das camadas na RAM e as roda na CPU (`-ngl` no llama.cpp), mas a velocidade cai ao ritmo da RAM (60–100 GB/s em DDR5). Para MoE, uma variante esperta mantém atenção e experts compartilhados na GPU e manda os experts roteados para a RAM; motores mais novos usam a VRAM como *cache* dos experts mais acessados.

A outra saída são as máquinas de **memória unificada** (Apple Silicon, AMD Strix Halo com até 128 GB, NVIDIA DGX Spark). Elas resolvem o “cabe?”, mas não o “quão rápido?”: a banda é de 4 a 8 vezes menor que a de uma GPU topo de linha. O benchmark do Akita mostra isso: o mesmo modelo denso de 27B gera 195 tok/s numa GPU de datacenter e **7,7 tok/s** no Strix Halo, com qualidade praticamente igual. Pela fórmula, ~29 GB por token ÷ ~256 GB/s ≈ 9 tok/s de teto: a física bate com a medição.

Quantização

> **Para começar**
>
> Quantizar é trocar uma régua milimetrada por uma em centímetros: menos exata, mas menor e mais rápida de ler. Guardar cada número com 4 bits em vez de 16 encolhe o modelo para cerca de um quarto e, feito com cuidado, quase não piora as respostas.

Não basta truncar bits: é preciso **mapear** a faixa de valores de cada grupo de pesos para a faixa do formato menor com um fator de escala e arredondar. Se os valores vão de 0 a 100 e o destino comporta 0 a 10, 53 vira 5,3, que vira 5; na reconstrução, 5 × 10 = 50, com erro de 3.

```
q = round(w / s) + z        # quantizar (s = escala, z = zero point)
ŵ = s · (q − z)             # dequantizar na hora da conta
```

Quanto menor o bloco que compartilha uma escala, menor o erro e maior o overhead (gasto extra); daí os bits "fracionários" dos formatos reais. Nomes como **W4A16** ou **W8A8** dizem os bits dos pesos (W) e das ativações (A). Segundo LLM Compression Explained: Build Faster, Efficient AI Models, em cargas **online** (chat, agentes) quantizar só os pesos acelera o decode, limitado por memória; em **lote**, vale quantizar também as ativações para usar Tensor Cores de 8 bits. A maioria usa **PTQ** (pós-treino), com algoritmos como GPTQ, AWQ e as *importance matrices* do mundo GGUF; **QAT** (durante o treino) é mais cara, mas resiste melhor a níveis agressivos.

Formato	Bits efetivos	Onde aparece	Comentário
BF16/FP16	16	checkpoints originais	referência de qualidade
FP8	8	vLLM, TensorRT; GPUs recentes	praticamente sem perda
GGUF Q8_0	8,5	llama.cpp/Ollama	indistinguível do BF16 na prática
GGUF Q4_K_M	~4,8	padrão do Ollama	ponto ótimo custo/qualidade
GGUF IQ3 / IQ2	3,4 / 2,x	llama.cpp com imatrix	viáveis em modelos grandes
GPTQ / AWQ	~4,1–4,25	vLLM	W4A16 para servidores
NVFP4 / MXFP4	~4,25–4,5	Blackwell; padrão OCP	calcula nativamente em 4 bits
Ternário / 1 bit	~1,6 / ~1,1	BitNet, Bonsai	exige treino ou destilação dedicada

O **NVFP4** é a novidade importante de 2025–26: diferente do INT4 só de pesos, permite *calcular* em 4 bits nos Tensor Cores da Blackwell, com escalas finas (bloco de 16 valores) que preservam a qualidade. No benchmark do Akita, modelos em NVFP4 numa única B300 empataram no topo dos open source com outro que precisou de quatro.

Quanto se perde? Em 8 bits, praticamente nada; em 4 bits bem feitos, de décimos de ponto a ~1%. No LLM Benchmark v4 do Akita, Q8 empatou com BF16 e 3 bits custaram ~1,5 ponto. Mas a perda não é uniforme: no teste de Bonsai: IA local de 27 bilhões em menos de 4GB, a versão de 1 bit (3,9 GB, contra 16,8 GB em Q4) manteve o acerto num bug de concorrência, mas errou todas as vezes uma pergunta factual de literatura. A compressão apaga primeiro o **conhecimento factual raro** e preserva o raciocínio sobre código, muito mais redundante nos dados de treino.

> **Atenção**
>
> Quantização pesa *menos* que configuração. No benchmark v4 do Akita, o **mesmo checkpoint** tirou 94 num provedor e 27 num servidor mal configurado. Template de chat errado, parser de tool calls ausente ou contexto truncado derrubam um modelo muito mais do que ir de Q8 para Q4.

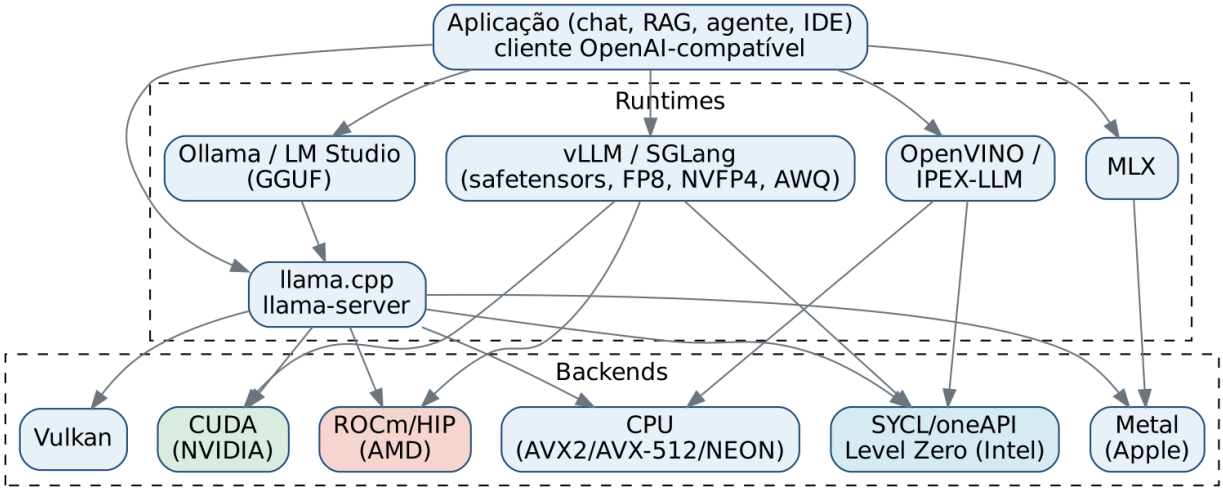
Você sabia?

O llama.cpp foi escrito por Georgi Gerganov em março de 2023, poucos dias após o vazamento dos pesos do LLaMA, e fez um LLM rodar num MacBook sem GPU dedicada. O formato GGUF nasceu no mesmo ecossistema — o "GG" vem das iniciais dele.

Runtimes: llama.cpp, Ollama, vLLM e companhia

Para começar

O arquivo do modelo sozinho não faz nada, assim como uma música precisa de um player. O **runtime** é esse player: carrega o modelo, recebe perguntas e devolve respostas. Há players para uso pessoal (Ollama, LM Studio) e para atender centenas de pessoas (vLLM).



Pilha de inferência local: aplicações falam uma API compatível com OpenAI; runtimes traduzem para backends de hardware. Fonte: elaboração própria.

Um motor de inferência recebe requisições, normalmente num endpoint compatível com a API da OpenAI (o que permite trocar de modelo sem reescrever código), tokeniza, agrupa o trabalho e executa prefill e decode, como detalha What Is LLM Inference? How AI Models Generate Responses.

O **llama.cpp** (What Is Llama.cpp? The LLM Inference Engine for Local AI) é C/C++ sem dependências. Usa o **GGUF**, um único arquivo com pesos, tokenizer e template de chat; oferece quantização ampla, execução em CPU e backends para Metal, CUDA, ROCm, Vulkan e SYCL. Ollama, LM Studio e Jan o usam por baixo; o **Ollama** acrescenta registro de modelos e um daemon, com padrões conservadores — o contexto padrão pequeno (`num_ctx`) é a causa número um de agentes "burros".

```
# Servidor OpenAI-compatível com llama.cpp
llama-server -m model.gguf --port 8080 -ngl 99 -c 16384 --jinja
```

Atenção

Servidores locais de inferência devem escutar só em `127.0.0.1`. Expor a porta na internet sem autenticação deixa qualquer um usar sua GPU e apagar modelos; varreduras já encontraram milhares de instâncias abertas assim. Para acesso remoto, use VPN ou proxy reverso com autenticação, como recomenda DIY AI Infrastructure: Build Your Own Privacy-Preserving AI at Home.

O **vLLM**, da UC Berkeley, é a referência para **servir muitos usuários** (What is vLLM? Efficient AI Inference for Large Language Models):

- **PagedAttention**: em vez de reservar KV cache contíguo pelo tamanho máximo (desperdiçando 60–80% da memória), divide-o em páginas pequenas alocadas sob demanda, como a memória virtual de um sistema operacional.
- **Continuous batching**: assim que uma sequência termina, outra entra no lote, como numa chapa de panquecas.
- **Prefix caching**: prefixos comuns (system prompts, documentos de RAG) são compartilhados entre requisições.
- **Decodificação especulativa**: um modelo rascunho propõe tokens e o grande verifica todos numa passada, com qualidade idêntica.

Critério	llama.cpp / Ollama	vLLM / SGLang	MLX	OpenVINO
Alvo	1 usuário, hardware de consumo	muitos usuários, datacenter	Mac	CPU/iGPU/GPU Intel
Formato	GGUF	safetensors (BF16, FP8, AWQ…)	MLX	IR do OpenVINO
Concorrência	básica	excelente	básica	moderada

Quando nem uma GPU comporta o modelo, divide-se o trabalho (How AI Models Scale Beyond a Single GPU Across LLM Workloads): réplicas inteiras (dados), blocos de camadas em GPUs diferentes (pipeline), cada camada fatiada entre GPUs com interconexão rápida (tensor), experts espalhados (MoE) e pools separados para prefill e decode (desagregação). O paralelismo de tensor exige uma operação coletiva por camada e, portanto, interconexão rápida (NVLink ou similar), normalmente dentro do mesmo servidor; o de pipeline precisa de fluxo contínuo de requisições para não deixar estágios ociosos. Em produção combina-se tudo: tensor dentro do servidor, pipeline entre servidores, dados entre réplicas.

CUDA e as alternativas

Para começar

Ter uma GPU não basta: é preciso um “idioma” para conversar com ela. A NVIDIA criou o **CUDA**, e quase todo software de IA foi escrito nele. AMD, Intel e Apple têm idiomas próprios, por isso usar placas delas exige escolher ferramentas que os falem.

O domínio da NVIDIA vem mais do software que do silício: quinze anos de bibliotecas (cuBLAS, cuDNN, NCCL, TensorRT) sobre as quais PyTorch e vLLM foram escritos. O ou-

tro lado é a **obsolescência**: cada GPU tem uma *compute capability*, e versões novas do CUDA e do PyTorch abandonam as antigas.

Ecossistema	Pilha	Situação para LLMs (out/2026)
NVIDIA CUDA	CUDA, cuBLAS, TensorRT-LLM	suporte universal; referência de desempenho
AMD ROCm/HIP	ROCm, HIP (similar ao CUDA)	PyTorch e vLLM oficiais; suporte a placas de consumo irregular
Intel oneAPI	SYCL, PyTorch XPU, OpenVINO	roda em llama.cpp, vLLM e OpenVINO; muita VRAM pelo preço
Apple Metal	Metal, MLX	ótimo para modelos grandes via memória unificada; banda limita velocidade
Vulkan	compute shaders	backend "universal" do llama.cpp; às vezes vence os nativos

> **Dica**
>
> Fora da NVIDIA, prefira formatos e runtimes "neutros": GGUF no llama.cpp (Vulkan, SYCL ou ROCm) ou OpenVINO, e teste mais de um backend, porque o mais rápido varia por modelo e versão. Projetos que dependem de kernels CUDA específicos costumam não ter porte, ou ter porte atrasado. Antes de comprar, confira se os runtimes que você pretende usar suportam oficialmente a placa.

Mesmo com o hardware certo, dá para deixar o acelerador ocioso. O vídeo The One-Line Slowdown: Why AI Models Get Slower mostra o caso clássico: ler um escalar com `.item()` dentro de um `if` ou log num modelo compilado causa um **graph break** (o `torch.compile` interrompe a otimização) e uma **sincronização** (a CPU espera o resultado e a fila do chip esvazia). Mantenha valores no dispositivo e registre a cada N passos.

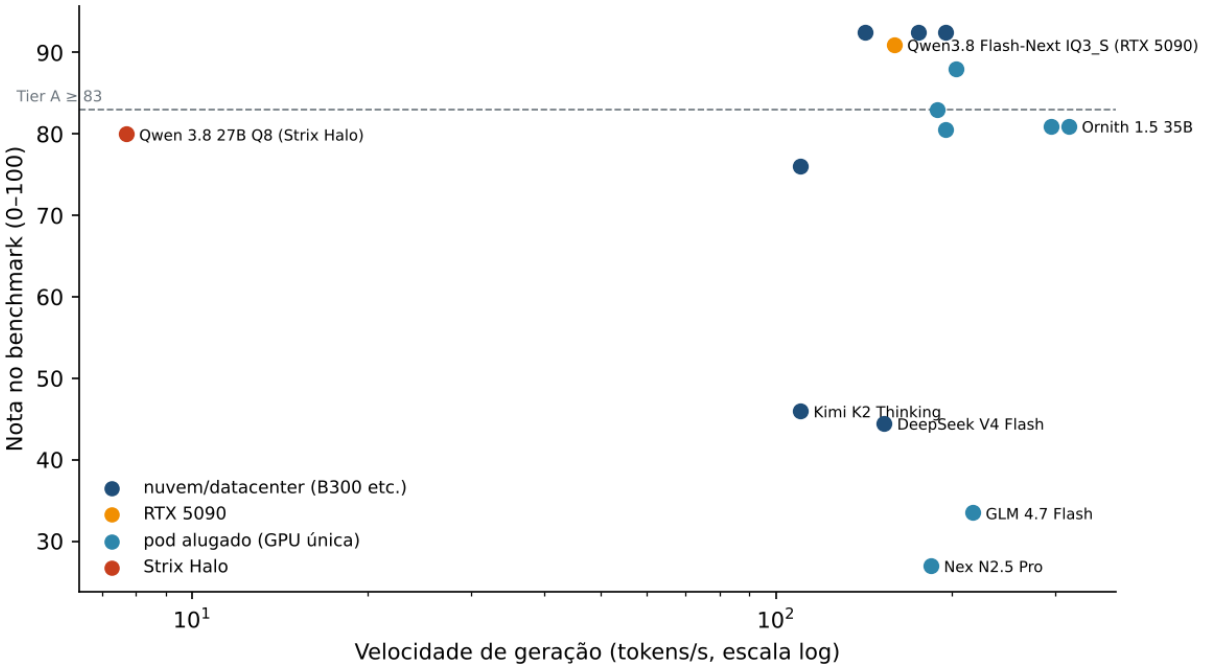
Medindo: tok/s, TTFT e o benchmark do Akita

> **Para começar**
>
> O velocímetro de um modelo mostra **tokens por segundo**. Também importa quanto ele demora para começar a responder, como o tempo até o carro arrancar. Definimos essas medidas e vemos números de um benchmark independente.

- **TTFT**: tempo até o primeiro token; cresce com o prompt e domina em RAG e agentes.
- **Tok/s de prefill e de geração**: o `llama-bench` mostra ambos; não compare um com o outro.
- **Vazão agregada**: soma de todos os usuários; é a métrica de servidor.

Para leitura interativa, ~20 tok/s já é confortável. Para agentes de código, que leem e escrevem dezenas de milhares de tokens por tarefa, pense em mais de 80–100 tok/s de geração.

No LLM Benchmark v4, o modelo constrói um app de chat em 7 sprints e um revisor injeta 14 defeitos; tier A é ≥ 83, com ruído de ~10 pontos.



Nota no benchmark × velocidade de geração dos modelos open source testados. A melhor configuração em GPU de consumo fica a 1,5 ponto do topo; a memória unificada tem boa qualidade, mas velocidade impraticável para agentes. Fonte: Akita, LLM Benchmark v4 (22/10/2026).

1. **Tamanho não garante qualidade**: um modelo de 1 trilhão de parâmetros tirou 46.
2. **Configuração pesa mais que quantização**: o mesmo checkpoint foi de 94 a 27.
3. **Uma GPU de consumo de topo chega a 1,5 ponto dos melhores abertos**, com um MoE em 3 bits.
4. Os fechados de ponta fazem 100; os melhores abertos, 92,5.

Na decisão entre alugar, comprar ou pagar por uso, a ocupação manda: Akita estima (LLM Benchmark v4: meu time cabe em 4x B300/H200?) que 4× B300 a ~US$ 36/h atendem ~32 sessões de agente, pouco mais de US$ 1 por usuário-hora — mas um pod ocioso custa o mesmo que um cheio, enquanto uma API escala a zero.

Opção	Custo fixo	Custo marginal	Quando faz sentido
API de fronteira	zero	por token	uso irregular; máxima qualidade
Pod GPU alugado	US$/hora ligado	~zero até saturar	equipe com uso contínuo; dados sensíveis
GPU própria	compra	energia	uso diário, privacidade, estudo
Memória unificada	compra	energia baixa	modelos grandes onde velocidade não é crítica

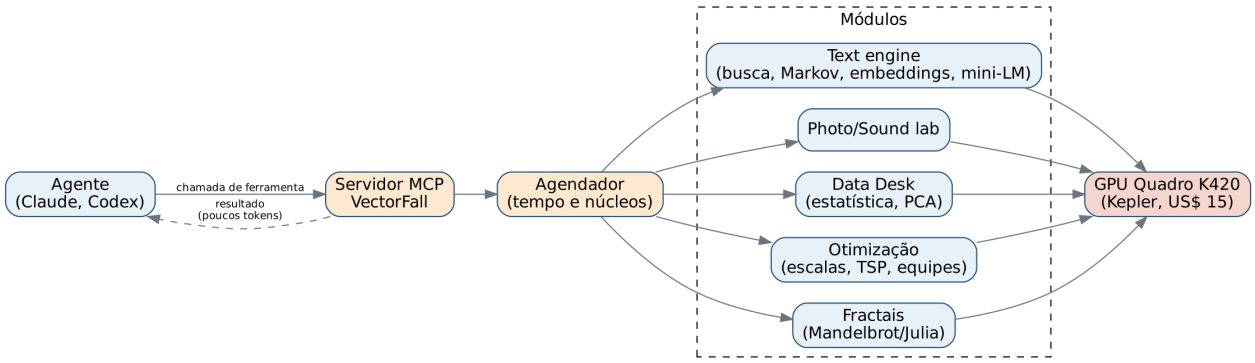
Para aprender sem gastar, o Google Colab (Build Anything with Google Colab, Here's How) oferece notebooks com GPU gratuita, com sessões que expiram. A armadilha clássica: o valor de uma variável é o da última célula *executada*, não o da ordem no arquivo.

GPGPU além da IA: a série VectorFall

> **Para começar**
> Uma GPU não serve só para IA. **GPGPU** é usar a placa para qualquer tarefa divisível em muitos pedaços independentes: filtrar imagens, analisar planilhas, testar rotas.

A série VectorFall, do canal Deep Tech with Amir Husain (Forget the $7,000 RTX 5090: Real AI and Text Analysis on a $19 GPU; A $15 GPU That Does Your AI Agent's Heavy Lifting | VectorFall Ep. 2; A $15 GPU Solves the Traveling Salesman Problem (and Your Staff Roster) | VectorFall Ep. 3), monta uma plataforma de computação sobre placas antigas de US$ 15–20. Como elas não têm suporte nas bibliotecas atuais, o código CUDA é escrito com ajuda de LLMs sobre versões antigas do toolkit; a IA tira o trabalho chato, não a necessidade de entender a arquitetura. A plataforma oferece análise de texto (busca, cadeias de Markov, embeddings), filtros de imagem e áudio, estatística sobre planilhas e solvers de escalas e de caixeiro-viajante.

A ideia mais interessante é a GPU como “músculo” de agentes: em vez de pedir a um LLM caro que processe uma imagem ou resolva uma escala (gastando tokens e alucinando soluções que violam restrições), o agente chama uma ferramenta MCP que executa o trabalho determinístico na GPU barata e devolve um resultado verificável. A GPU não torna o caixeiro-viajante “fácil” — ele continua NP-difícil (ver Livro 4, Capítulo 2 (Algoritmos)) —, mas acelera heurísticas paralelas.



Arquitetura do VectorFall: agentes chamam ferramentas MCP; um agendador reparte tempo e núcleos da GPU entre os módulos. Fonte: elaboração própria com dados de “A $15 GPU That Does Your AI Agent's Heavy Lifting | VectorFall Ep. 2”.

```
// kernel de Mandelbrot em CUDA: um thread por pixel
__global__ void mandel(unsigned char* img, int w, int h, int maxIt) {
  int x = blockIdx.x*blockDim.x + threadIdx.x, y = blockIdx.y*blockDim.y + threadIdx.y;
  if (x >= w || y >= h) return;
  float cr = -2.0f + 3.0f*x/w, ci = -1.5f + 3.0f*y/h, zr = 0, zi = 0; int i = 0;
  while (zr*zr + zi*zi < 4.0f && i < maxIt) {
    float t = zr*zr - zi*zi + cr; zi = 2*zr*zi + ci; zr = t; ++i; }
  img[y*w + x] = (unsigned char)(255 * i / maxIt);
}
```

O mesmo raciocínio chega ao SQL (How GPUs Accelerate Data & Analytics with AI): filtros, joins e agregações repetem a mesma operação sobre milhões de linhas. Motores he-

terogêneos decidem por operador onde executar: em volumes pequenos a CPU vence, porque copiar dados pelo PCIe não se paga; em volumes grandes, a GPU.

> **Você sabia?**
> O conjunto de Mandelbrot é o exemplo clássico de problema "embaraçosamente paralelo": cada pixel é calculado de forma totalmente independente dos outros. Por isso é usado há décadas para demonstrar e medir o desempenho de GPUs.

Checklist prático

> **Para começar**
> Como a lista de um piloto antes de decolar, siga os passos na ordem: cada um usa a resposta do anterior.

1. **Defina o uso**: chat, agente de código, servidor para equipe ou lote offline.
2. **Calcule a memória**: params × bytes (+15%) + KV cache para o contexto real.
3. **Estime a velocidade**: banda ÷ bytes por token (em MoE, parâmetros ativos). Abaixo de 15 tok/s, repense.
4. **Escolha o runtime**: Ollama/LM Studio para começar; llama.cpp para controle fino e placas que não são NVIDIA; vLLM para servir várias pessoas; OpenVINO para Intel; MLX para Mac.
5. **Escolha a quantização**: Q4_K_M como padrão; Q6/Q8 se sobrar memória; abaixo de 4 bits só em modelos grandes.
6. **Configure direito**: template de chat, contexto, parser de tool calls, sampling recomendado.
7. **Meça** velocidade e qualidade com tarefas suas.
8. **Proteja**: nada de portas de inferência expostas sem autenticação.

Fontes deste capítulo

Título	URL	Canal / autor	Ponto principal
Forget the $7,000 RTX 5090: Real AI and Text Analysis on a $19 GPU	https://www.youtube.com/watch?v=S6_5Wfn2BkM	Deep Tech with Amir Husain	Análise de texto em GPU antiga
A $15 GPU That Does Your AI Agent's Heavy Lifting | VectorFall Ep. 2	https://www.youtube.com/watch?v=uH6ekrpWIrg	Deep Tech with Amir Husain	GPU barata como ferramenta MCP
A $15 GPU Solves the Traveling Salesman Problem (and Your Staff Roster) | VectorFall Ep. 3	https://www.youtube.com/watch?v=2ILc4WY86us	Deep Tech with Amir Husain	Estatística, escalas e TSP na GPU
The AI Memory Wall: Why LLMs & AI Accelerators Hit a Bottleneck	https://www.youtube.com/watch?v=4b6KpFN7Rio	IBM Technology	Decode limitado por banda; roofline
AI Accelerators: Transforming Scalability & Model Efficiency	https://www.youtube.com/watch?v=KX0qBM-ByAg	IBM Technology	Aceleradores e cascata de modelos
What Is LLM Inference? How AI Models Generate Responses	https://www.youtube.com/watch?v=615-yWyS1n0	IBM Technology	Prefill, decode, TTFT e ITL
How AI Models Scale Beyond a Single GPU Across LLM Workloads	https://www.youtube.com/watch?v=qZBibWYcKH4	IBM Technology	Formas de paralelismo na inferência
What is vLLM? Efficient AI Inference for Large Language Models	https://www.youtube.com/watch?v=McLdlg5Gc9s	IBM Technology	PagedAttention e continuous batching

Título	URL	Canal / autor	Ponto principal
What Is Llama.cpp? The LLM Inference Engine for Local AI	https://www.youtube.com/watch?v=P8m5eHAyrFM	IBM Technology	GGUF, quantização e backends
What Is Quantization? How AI Models Get Smaller & Faster	https://www.youtube.com/watch?v=mmns33BsBcY	IBM Technology	Mapeamento por escala; PTQ vs QAT
LLM Compression Explained: Build Faster, Efficient AI Models	https://www.youtube.com/watch?v=wIXr22QTEHg	IBM Technology	Quantização online vs lote; pouca perda
Bonsai: IA local de 27 bilhões em menos de 4GB	https://www.youtube.com/watch?v=mCWUtEbmssM	ViktorKav	1 bit apaga fatos, preserva código
DIY AI Infrastructure: Build Your Own Privacy-Preserving AI at Home	https://www.youtube.com/watch?v=BvCOZrqGyNU	IBM Technology	IA doméstica com VPN e MFA
The One-Line Slowdown: Why AI Models Get Slower	https://www.youtube.com/watch?v=UgEA5ictENM	IBM Technology	Graph break e sincronização ociosa
How GPUs Accelerate Data & Analytics with AI	https://www.youtube.com/watch?v=zqowMgMgSjA	IBM Technology	SQL heterogêneo CPU/GPU por operador
Build Anything with Google Colab, Here's How	https://www.youtube.com/watch?v=JsCU1V7Hjpc	David Ondrej	Notebooks com GPU grátis
LLM Benchmark v4 — o veredito de open source (22/10/2026)	https://akitaonrails.com/2026/10/22/llm-benchmark-v4-o-veredito-de-open-source-ate-out-2026/	Fabio Akita (AkitaOnRails)	Serving pesa mais que quantização
LLM Benchmark v4: open source nível consumidor na RTX5090 ou Strix Halo (22/10/2026)	https://akitaonrails.com/2026/10/22/llm-benchmark-v4-open-source-nivel-consumidor-rtx5090-strix-halo-strata/	Fabio Akita (AkitaOnRails)	Memória unificada: cabe, mas é lenta
LLM Benchmark v4: meu time cabe em 4x B300/H200? (22/10/2026)	https://akitaonrails.com/2026/10/22/llm-benchmark-v4-can-my-team-fit-in-a-4x-b300-h200/	Fabio Akita (AkitaOnRails)	4× B300 ≈ US$ 36/h, ~32 sessões

CAPÍTULO 3 ·

AI-Driven Engineer

> *Este capítulo apresenta um conceito proposto por **Marcus B. Silva** (GitHub: marcusbsilva): o **AI-Driven Engineer**, o engenheiro que projeta, especifica, orquestra e verifica sistemas usando agentes de IA como força de trabalho. Veremos a definição, os doze princípios do conceito, os dados de adoção e produtividade, as dez competências centrais, o que se espera de cada nível de carreira e o que está mudando nas vagas e entrevistas. Números externos verificados em 30 e 31/10/2026.*

Um conceito proposto por Marcus B. Silva

"AI-Driven Engineer" não é um cargo padronizado nem um rótulo de fornecedor: é um **conceito proposto por Marcus B. Silva**, formulado a partir da prática de construir vários projetos em paralelo com agentes de IA. O que o diferencia de termos vizinhos, como "AI engineer" (quem constrói produtos *de* IA) ou "vibe coder" (quem aceita o que a IA gera), são os princípios apresentados adiante.

Definição

AI-Driven Engineer é quem constrói software soberano e verificável usando agentes de IA como equipe: decide o que construir e em qual linguagem cada parte vive, especifica, orquestra os agentes, exige que eles discordem, verifica cada entrega e entrega o resultado pronto para usar. O código pode ser escrito por máquinas; a responsabilidade continua humana.

Para começar

Pense numa obra com pedreiros incansáveis e baratos (os agentes), que às vezes erguem a parede no lugar errado e garantem que está tudo pronto. Nesse canteiro, vale mais quem sabe *o que* construir, desenha a planta, inspeciona cada parede e assina o laudo. Esse é o AI-Driven Engineer.

De onde vem o conceito: arquitetura modular

Parte da ideia de AI-Driven Engineer vem de um estilo antigo e testado de construir software: a **arquitetura modular**. Nela, o sistema é dividido em **módulos**, peças com uma única responsabilidade. Cada módulo conversa com os outros por uma **interface** (a "porta de entrada" do módulo) que segue um **contrato** claro: o que ele recebe, o que devolve e o que promete. Dois termos resumem o objetivo:

- **Baixo acoplamento**: mexer num módulo não obriga a mexer nos outros. É como trocar a lâmpada sem refazer a fiação da casa.

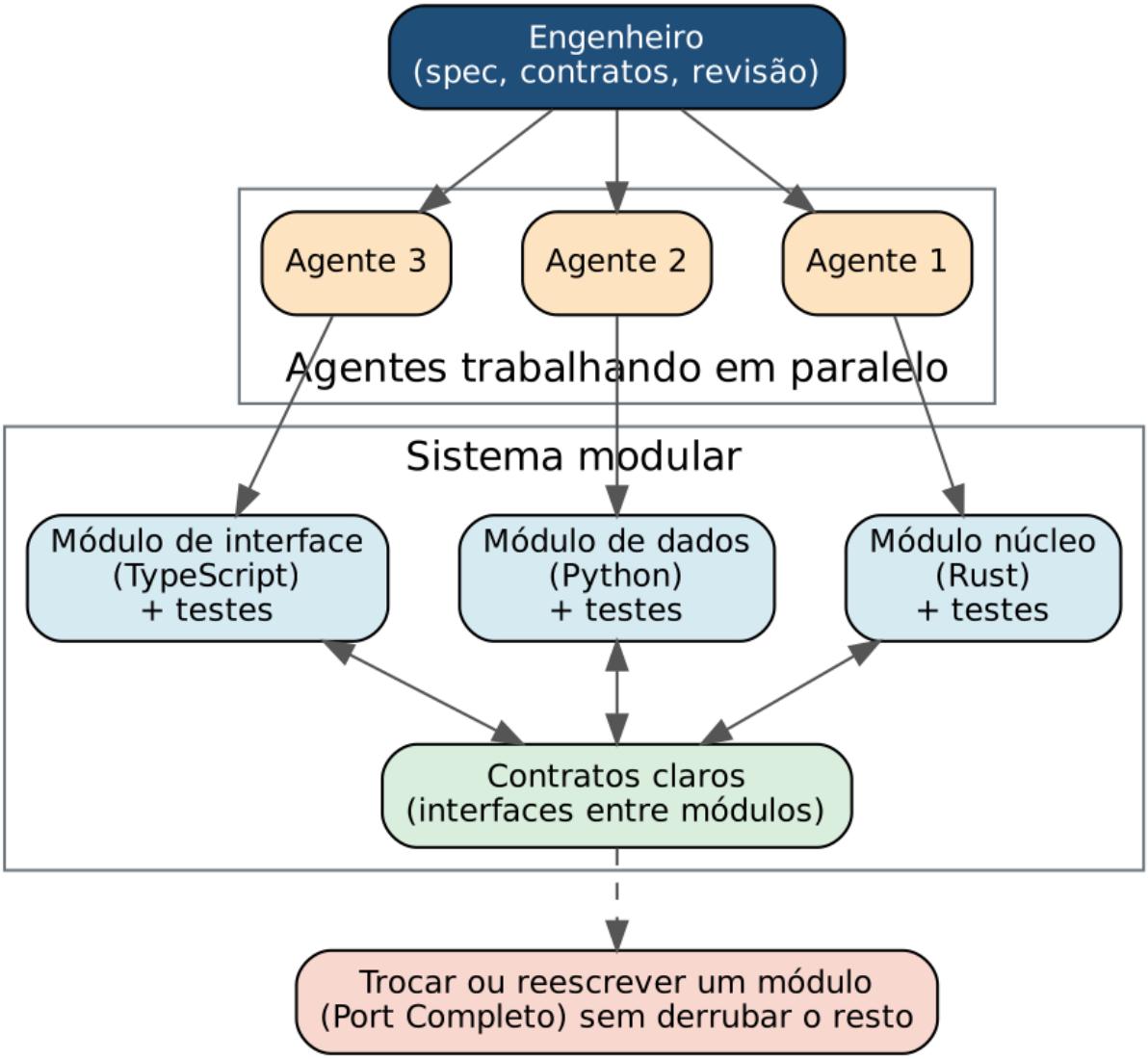
- **Alta coesão**: tudo o que está dentro de um módulo trata do mesmo assunto. É como uma gaveta só de talheres.

Exemplos conhecidos dessa forma de pensar:

- **Filosofia Unix**: "faça uma coisa e faça bem". Pequenos comandos, como `grep` e `sort`, combinados em sequência.
- **Plugins**: o editor de texto continua o mesmo; cada plugin acrescenta uma função sem mexer no núcleo.
- **Microsserviços × monólito modular**: no primeiro, cada módulo roda como um programa separado; no segundo, tudo roda junto, mas com divisões internas bem marcadas. As duas opções são modulares; muda o custo de operar.
- **Clean Architecture e Arquitetura Hexagonal**: a regra de negócio fica no centro, e banco de dados, telas e serviços externos ficam nas bordas, ligados por interfaces. Trocar o banco não muda a regra.

Por que isso combina com agentes de IA

- **Cada módulo cabe no contexto de um agente.** O agente não precisa entender o sistema inteiro, só o seu pedaço e o contrato dele.
- **Contratos e testes por módulo permitem verificar o que a IA produz.** Se o módulo cumpre o contrato e passa nos próprios testes, a entrega está conferida.
- **Cada módulo pode usar a linguagem que melhor serve ao seu papel**: o núcleo rápido em Rust, os dados em Python, a tela em TypeScript. É o princípio **Poliglota por Papel**, visto adiante.
- **Agentes trabalham em paralelo sem pisar uns nos outros**, cada um no seu módulo. É a base da **IA como Equipe Paralela**.
- **Trocar ou reescrever um módulo não derruba o sistema.** Enquanto o contrato for respeitado, dá para fazer um **Port Completo** de uma peça, ou unir projetos numa **Fusão de Repositórios**, aos poucos e com segurança.



Arquitetura modular com agentes: o engenheiro define contratos e revisa; cada agente cuida de um módulo, com linguagem e testes próprios; os módulos só se falam pelos contratos, por isso uma peça pode ser trocada sem derrubar as outras. Fonte: elaboração própria.

> **Dica**
>
> Antes de pedir código a um agente, pergunte: "qual módulo é este, qual é o contrato dele e qual teste prova que ele funciona?". Se não souber responder, a tarefa ainda não está pronta para ser delegada.

O que é um AI-Driven Engineer

Ele **projeta** (arquitetura e limites), **especifica** (transforma intenção em requisitos verificáveis), **orquestra** (distribui tarefas e monta o contexto e as ferramentas dos agentes) e **verifica** (prova com testes, revisão e medição que o resultado está correto, seguro e barato o bastante). **Agente** é um programa baseado num LLM que executa ações em ciclo: lê arquivos, roda comandos, observa o resultado e decide o próximo passo (Livro 2, Capítulo 4 (Agentes e MCP)). Sem um "oráculo" (testes, especificação, métricas) que diga se o resultado está certo, o agente gira no vazio.

Fabio Akita defende a mesma tese em LLMs + Agentes = competente o suficiente: os modelos de ponta convergiram em qualidade, e o salto recente veio do *harness* agêntico (ferramentas, buscas e loops de teste em volta do modelo). Por isso, **engenharia tradicional (testes, processo, invariantes) é o diferencial**.

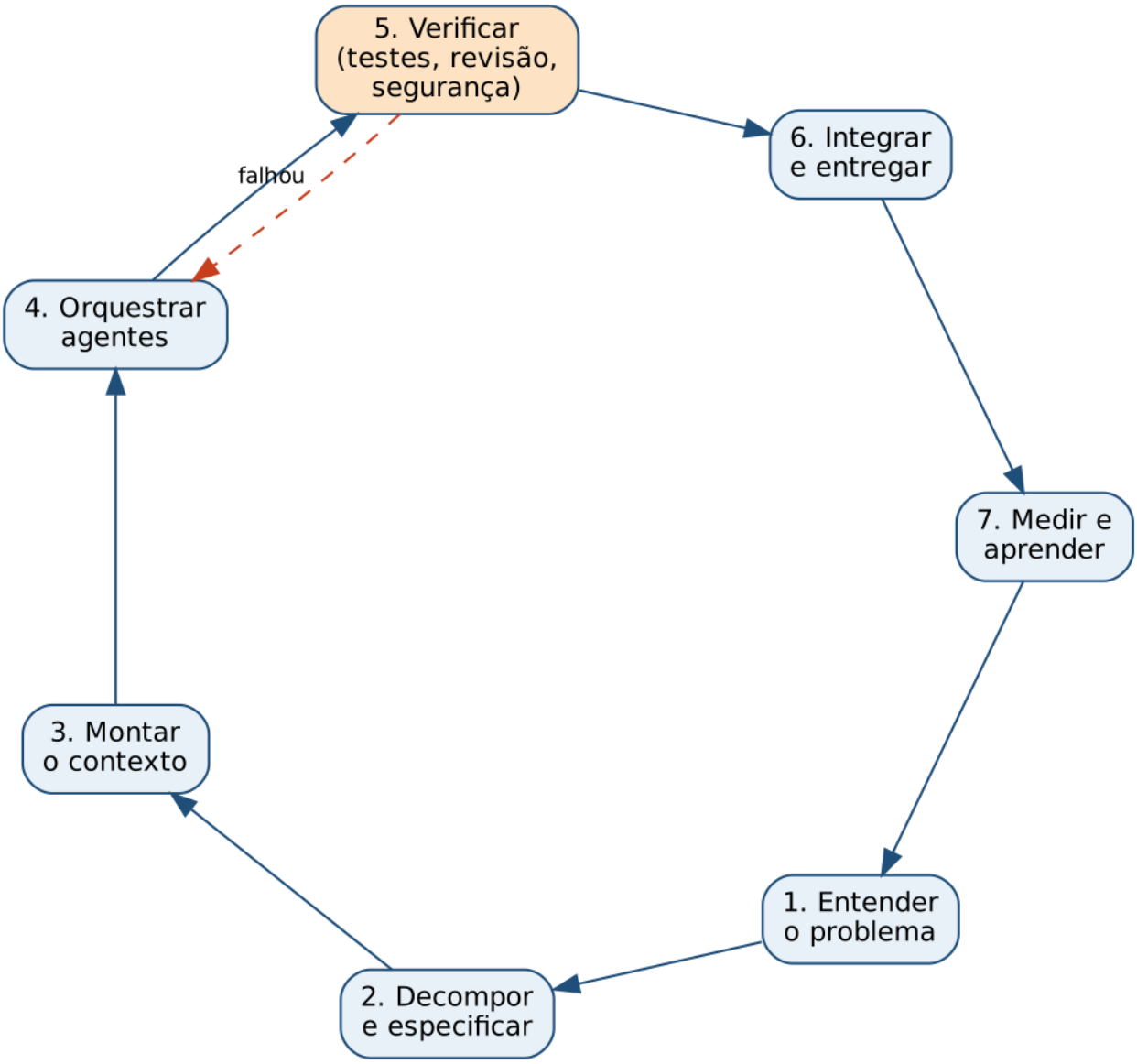
Três perfis

Vibe coding é descrever o que se quer, aceitar o código sem ler, colar o erro de volta e repetir até "funcionar". Serve para protótipos descartáveis; o problema surge quando o protótipo vira produto.

Dimensão	Programador tradicional	Vibe coder	AI-Driven Engineer
Quem escreve o código	Ele mesmo	A IA, sem revisão	Agentes, sob especificação e revisão
Como sabe que está pronto	Testa e revisa	"A IA disse que está pronto"	Testes automatizados, métricas, revisão humana
Gargalo	Velocidade de digitação	Entender o que foi gerado	Clareza do pedido e da conferência
Risco típico	Lentidão	Falhas de segurança escondidas	Confiar demais nas ferramentas; gastar muito

O canal Área Tech Brasil resume em COMO FICAR PERIGOSAMENTE BOM EM PROGRAMAÇÃO: o "codificador de prompt" terceiriza o pensamento; o "orquestrador" chega com hipótese e critério de aceite e terceiriza só a digitação. Em O jeito CERTO de usar IA para programar (ninguém te explica isso), o Sujeito programador reserva o vibe coding para POCs, usa fluxo agêntico (pesquisar, planejar, implementar) em tarefas médias e o GitHub Spec Kit em features complexas: "a IA é proporcional ao conhecimento do desenvolvedor".

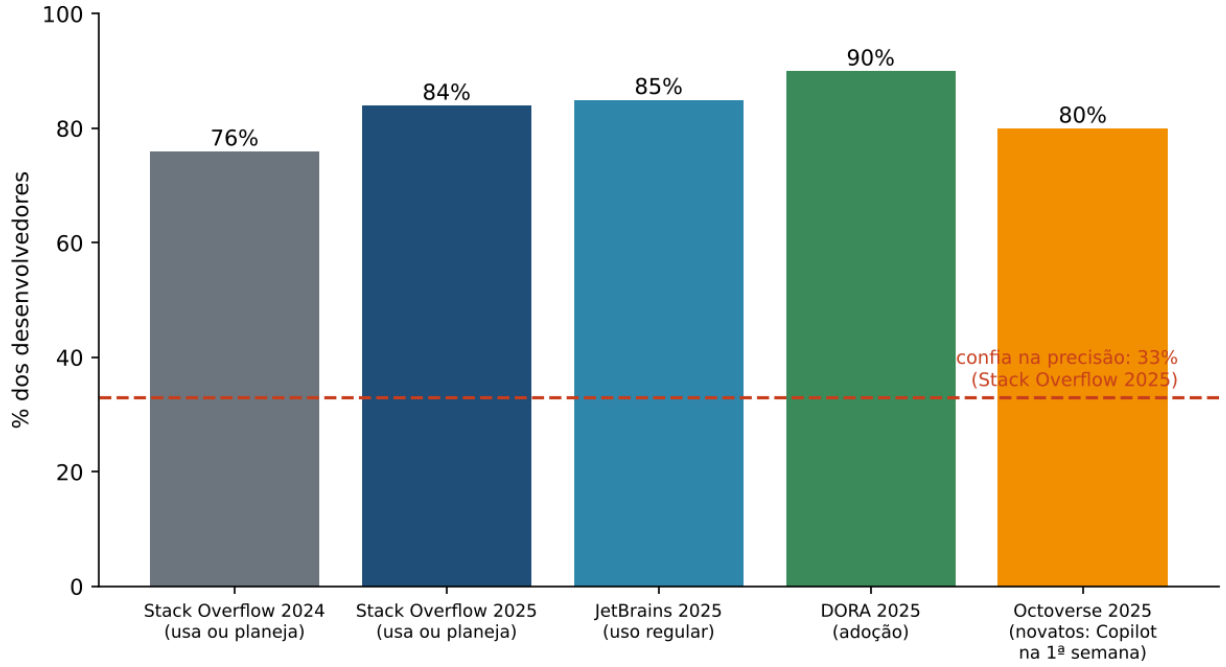
O que acontece sem guia aparece em Revisei código gerado por I.A..... PREOCUPANTE, de Augusto Galego: um app de tarefas gerado pelo Cursor sem isolamento entre usuários, `SELECT *` sem paginação, roteador reinventado sem cabeçalhos de segurança, sem migrations e com JWT sem restrição de algoritmo. Tudo "roda"; nada vai para produção.



O ciclo do AI-Driven Engineer: entender, especificar, montar o contexto, orquestrar, verificar (voltando aos agentes quando algo falha), integrar e medir. Fonte: elaboração própria.

Adoção: uso alto, confiança baixa

- **Stack Overflow 2025**: 84% usam ou planejam usar IA; 47% usam diariamente; só cerca de 33% confiam na precisão. Apenas cerca de 15% dizem praticar vibe coding.
- **Stack Overflow 2026** (segundo o IT Brief, dados preliminares): 66% usam assistentes ou agentes; só 24% têm diretrizes formais do empregador.
- **Google DORA 2025**: 90% de adoção, mas só 24% confiam muito; a IA é um "espelho e multiplicador" da organização.
- **GitHub Octoverse 2025**: mais de 180 milhões de desenvolvedores; o TypeScript virou a linguagem mais usada; o Brasil é o 4.º país.



Adoção de IA por desenvolvedores comparada à confiança nas respostas. As perguntas diferem entre pesquisas; as barras indicam ordem de grandeza. Fonte: elaboração própria com dados de Stack Overflow, JetBrains, Google DORA e GitHub Octoverse.

Se todos usam uma ferramenta em que poucos confiam, vale muito quem consegue *tornar o resultado confiável*.

O que dizem os estudos de produtividade

Estudo	Resultado	Leitura
Cui, Demirer et al. (4.867 devs, Copilot)	+26% de pull requests (propostas de mudança) por semana; ganho maior entre iniciantes	Autocompletar ajuda em tarefas rotineiras
METR (2025, 16 devs experientes em open source)	Achavam ter ganhado 20%; levaram 19% *mais* tempo	A percepção de ganho engana
Anthropic (2026, 52 engenheiros aprendendo uma biblioteca)	Compreensão de 50% com IA contra 67% sem	Delegar tudo prejudica o aprendizado

A IA acelera tarefas bem delimitadas, mas pode atrapalhar o aprendizado de quem só delega. No estudo da Anthropic, os melhores padrões foram "gerar e depois perguntar até entender" e "pedir código com explicação". Desconfie de números de fornecedor ("10x"); meça no seu contexto: tempo até o merge, defeitos, retrabalho.

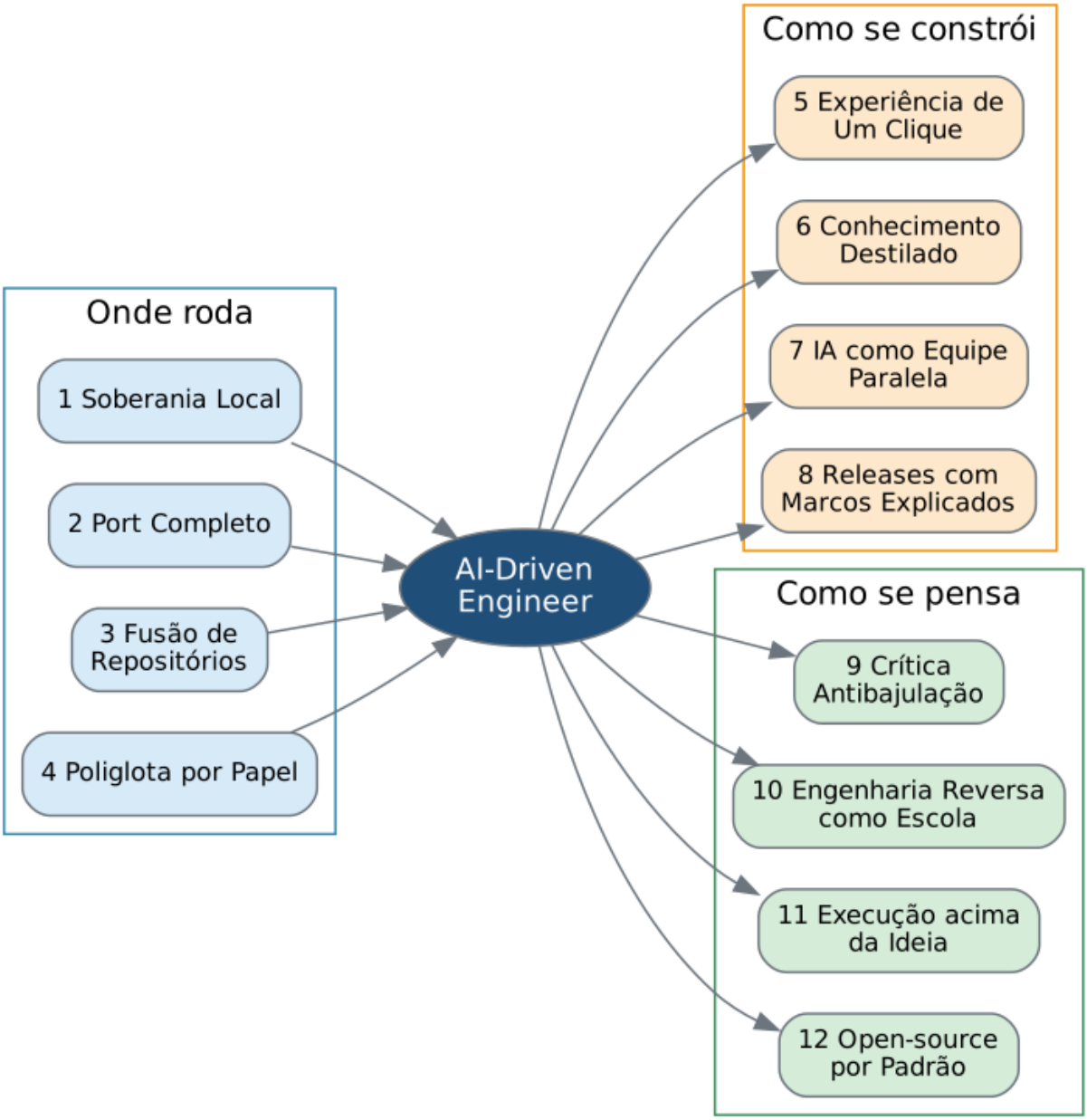
Você sabia?

No estudo do METR, os desenvolvedores previram antes que a IA os deixaria 24% mais rápidos e, ao final, ainda acreditavam ter ganhado 20%. A medição mostrou o contrário: 19% mais lentos.

Os princípios do AI-Driven Engineer

Para começar

Os princípios respondem a três perguntas: **onde** o software roda, **como** é construído e **como** o engenheiro pensa. Cada um traz uma prática comum na indústria e o anti-padrão que evita.



Os doze princípios do AI-Driven Engineer. Fonte: elaboração própria a partir do conceito apresentado neste capítulo.

Soberania Local (local-first)

Sempre que possível, o sistema e a IA que o constrói rodam na máquina de quem os usa. Modelos abertos servidos por llama.cpp ou Ollama eliminam a dependência de uma API,

o custo por token e o envio de código a terceiros. Isso exige conhecer os limites do hardware: memória da GPU, quantização adequada (Capítulo 2 (Hardware e IA local)). **Na prática:** usar um modelo pequeno no próprio PC para tarefas do dia a dia e a nuvem só quando ele não der conta. **Anti-padrão:** um produto cujo recurso central para quando a chave de API expira ou o provedor muda o preço.

Port Completo

Ferramentas de referência são estudadas e reescritas como versões próprias e abertas, com as funções que importam. O original serve de especificação viva. **Na prática:** como o LibreOffice diante do Microsoft Office ou o Inkscape diante do Illustrator, escrever uma alternativa aberta com a mesma experiência, sem copiar código. **Anti-padrão:** um "clone" que copia a tela e chama a mesma API fechada por baixo, ou código copiado sem licença compatível (Livro 5, Capítulo 3 (Propriedade intelectual)).

Fusão de Repositórios

Quando vários projetos resolvem pedaços do mesmo problema, eles são fundidos num sistema coeso, com uma configuração, um ponto de entrada e responsabilidades sem sobreposição. Agentes tornam viável ler e integrar bases que antes levariam meses. **Na prática:** juntar vários scripts soltos num único programa de linha de comando, mantendo licenças e créditos. **Anti-padrão:** cinco repositórios clonados lado a lado e um script que reza para que todos subam na ordem certa.

Poliglota por Papel

A linguagem é escolhida pelo papel da peça: Rust ou Go no núcleo que exige desempenho, Python em dados e ML, TypeScript na interface web, Lua para scripting embutido. Pontes como PyO3 ligam Rust a Python. **Na prática:** reescrever em Rust só a parte lenta de um programa Python, sem mudar o jeito de usá-lo (como fizeram o Ruff e o Pydantic v2). Akita mostra em Existem linguagens melhores pra programar com LLMs? que bons testes pesam mais que a linguagem. **Anti-padrão:** tudo na única linguagem que se conhece, ou trocar de linguagem a cada modismo.

Experiência de Um Clique

O usuário vem antes: instaladores que resolvem as dependências sozinhos, um **launcher único** e uma raiz de projeto limpa, com scripts auxiliares numa pasta própria. Toda release traz um pacote pronto. **Na prática:** um comando ou instalador (como os de VS Code e Docker Desktop) que verifica e baixa o que falta e abre a aplicação. **Anti-padrão:** um README de 40 passos ou uma raiz com `start.bat`, `start2.bat` e `run_final_v3.bat`.

Conhecimento Destilado

Documentação, vídeos e decisões viram um corpus organizado e com fontes, que alimenta um RAG consultado pelos agentes (Livro 2, Capítulo 3 (RAG)). **Na prática:** dar ao assistente de código do time acesso à documentação interna e às decisões já tomadas.

Anti-padrão: centenas de links "salvos para depois" ou um RAG de texto bruto, sem curadoria.

IA como Equipe Paralela

Agentes permitem várias frentes ao mesmo tempo, cada uma com contexto e especificação próprios; o engenheiro age como líder técnico. **Na prática:** um agente cria uma função nova, outro escreve os testes e um terceiro atualiza a documentação, cada um na sua branch (cópia separada do código), revisadas em horários fixos do dia. **Anti-padrão:** abrir dez frentes e não revisar nenhuma, ou misturar projetos no mesmo contexto.

Releases Incrementais com Marcos Explicados

O software evolui por versões pequenas e nomeadas, cada uma com um marco claro. Antes de cada passo, a IA explica o plano; depois, o changelog registra o que mudou. **Na prática:** "v0.5: login com a conta da empresa; pronto quando o usuário entra e sai sem problemas". O agente propõe, o engenheiro aprova e os testes provam. **Anti-padrão:** um commit gigante "versão nova" sem changelog.

Crítica Antibajulação

Modelos tendem a concordar com quem pergunta (*sycophancy*). O prompt de sistema **exige** discordância fundamentada: falhas, riscos, alternativas (Livro 2, Capítulo 2 (Engenharia de Contexto)). **Na prática:** regra no arquivo de instruções: "Antes de implementar, liste até três objeções técnicas. Se não houver, diga por quê." **Anti-padrão:** aceitar o "Excelente ideia!" como validação.

Engenharia Reversa como Escola

Estudar como sistemas existentes funcionam (protocolos, formatos, arquiteturas) é uma forma de aprender a projetar, como a indústria faz em interoperabilidade, auditoria e preservação (Livro 4, Capítulo 4 (Eng. Reversa e Segurança)). **Na prática:** documentar um formato de arquivo para que outro programa o leia, como fizeram projetos como o Samba. **Anti-padrão:** redistribuir código ou assets de terceiros ou contornar proteções sem base legal.

Execução acima da Ideia

Ideias valem pouco até virarem software usado. Com protótipos baratos, a pergunta passa a ser "o que a primeira versão real nos ensinou?". **Na prática:** publicar uma v0.1 útil em semanas, ouvir testadores e reorientar o escopo. **Anti-padrão:** meses polindo um plano, ou insistir nele quando o mercado já mudou.

Open-source por Padrão (em projetos pessoais)

Nos **projetos pessoais**, o código nasce público, com licença clara, releases e convite a testadores. Abrir obriga a manter documentação útil e build reproduzível, e retribui ao ecossistema (Livro 6, Capítulo 1 (Comunidades e Linux)). **Código de empresa privado**

nunca é publicado: pertence ao empregador ou cliente e só é aberto por decisão formal da empresa. **Na prática:** cada projeto pessoal tem licença, CHANGELOG e uma release. **Anti-padrão:** repositório público sem licença (que na prática não autoriza o uso), ou subir código do trabalho para um repositório pessoal.

Manifesto do AI-Driven Engineer

Rodo local antes de depender da nuvem de alguém. Reescrevo para entender. Uno o que está espalhado. Escolho a linguagem pelo papel. Entrego com um clique. Transformo o que estudo em memória para os agentes. Lidero a IA como equipe e exijo que ela discorde. Avanço por marcos explicados. Desmonto para aprender, nunca para copiar. Executo em vez de guardar ideias. Publico aberto o que é meu e protejo o código da empresa. E assino, como humano, tudo o que meus agentes escrevem.

Você sabia?

O llama.cpp foi criado por Georgi Gerganov em março de 2023 para rodar o LLaMA em CPU comum. Em agosto de 2023 o projeto introduziu o formato GGUF, hoje padrão de fato para distribuir modelos abertos para uso local.

As dez competências centrais

Para começar

Se a IA digita, ao humano sobra o que vem antes e depois: entender, decompor, especificar, conferir, proteger e explicar. Nenhuma competência é nova; mudou o peso de cada uma.

1. Decomposição. Quebrar um objetivo vago em partes pequenas e testáveis: um endpoint, um componente, uma migração. Regra prática: se o teste de aceitação não cabe em poucas linhas, a tarefa ainda está grande demais.

2. Especificação (SDD). No desenvolvimento orientado a especificação, a spec fica versionada e é o contrato do qual derivam código e testes. Em SPEC Driven: A MELHOR FORMA De Usar IA em 2026!, O Novo Programador propõe seis fases (especificar, projetar, quebrar, executar com teste antes, auditar, aprender), critérios **Dado / Quando / Então** e nenhuma implementação com suposição em aberto.

```
História: Como tutor, quero remarcar um banho para outro horário livre.
Critério 1
  Dado um agendamento futuro e um horário livre no mesmo dia
  Quando o tutor escolhe o novo horário
  Então o agendamento muda e o horário antigo fica livre
Critério 2
  Dado um agendamento a menos de 2 horas do início
  Quando o tutor tenta remarcar
  Então o sistema recusa com "Remarcação só até 2h antes"
Pergunta em aberto: remarcar para outro dia é permitido? (bloqueia a tarefa)
```

3. Engenharia de contexto. Decidir o que entra na janela do agente: arquivos de regras (CLAUDE.md, AGENTS.md), skills, ferramentas via MCP, um chat limpo por tarefa e subagentes para pesquisas (Livro 2, Capítulo 2 (Engenharia de Contexto)). Contexto demais também degrada a qualidade.

4. Verificação e testes. A competência que mais cresce: testes viram o sensor que diz ao agente se ele acertou. No experimento de Akita com 7 linguagens e 4 modelos, a linguagem não decidiu; com 27 testes públicos, os modelos fracos entregaram 24 artefatos corretos, contra 16 sem eles. Verificar também é separar quem implementa de quem valida, exigir o código de saída dos testes e revisar o diff.

5. Arquitetura. As decisões caras de mudar: divisão do sistema, dados, comunicação, falhas. A IA perde o fio em sistemas grandes e reinventa o que o framework já resolve; o engenheiro fixa padrões e limites. Renato Augusto, em Como Virar Um ARQUITETO DE SOFTWARE | 5 PASSOS Que Vão Mudar Sua Carreira de PROGRAMADOR, põe a comunicação como passo mais importante.

6. Segurança. Falhas comuns em código gerado: controle de acesso ausente, validação fraca, segredos no código, pacotes inventados registrados por atacantes e **prompt injection** (texto lido pelo agente com instruções que ele obedece). Ver Capítulo 4 (Desenvolvimento seguro e Pull Request).

7. Leitura de código. Com a máquina escrevendo dez vezes mais, ler vira a habilidade principal, inclusive perceber o que *falta*. Código gerado quase sempre *parece* correto.

8. Avaliação de modelos. O LLM Benchmark v4 de Akita mostra que tamanho não garante qualidade, que a forma de servir pesa mais que a quantização e que há cerca de 10 pontos de ruído entre rodadas. Avaliar é rodar um pequeno *eval* com tarefas reais antes de trocar de modelo.

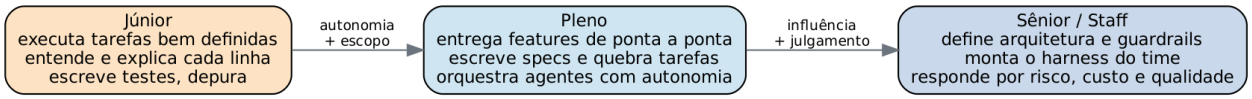
9. Custos. Uma tarefa de agente pode consumir mais de 100 mil tokens. Use cache de prompt, modelos menores para tarefas simples e caros para planejamento e revisão (Capítulo 2 (Hardware e IA local); Livro 5, Capítulo 1 (Negócios, carreira e governança)).

10. Comunicação. Specs, documentos de decisão, code review e escopo são comunicação escrita com humanos e máquinas ao mesmo tempo.

> **Em resumo**
> As competências formam três blocos: **pensar** (decomposição, especificação, arquitetura), **conduzir** (contexto, modelos, custos, comunicação) e **garantir** (verificação, segurança, leitura). Quem domina só "conduzir" é um vibe coder sofisticado.

Os níveis: de júnior a staff

> **Para começar**
> Níveis medem **autonomia e alcance**, não anos de casa: o júnior resolve tarefas definidas por outros, o pleno resolve problemas inteiros, o sênior resolve problemas ainda não definidos e o staff faz isso entre times. A IA mudou o que se espera em cada degrau.



A escada de níveis do AI-Driven Engineer: do júnior que explica cada linha ao sênior/staff que define arquitetura, guardrails e o harness do time. Fonte: elaboração própria.

Júnior

Princípios em foco: Crítica Antibajulação, Releases Incrementais, Experiência de Um Clique e, **nos projetos pessoais e de portfólio**, Open-source por Padrão (repositório público com licença). O código do empregador nunca vai para o portfólio. Um pequeno Port Completo, como reescrever do zero uma ferramenta de linha de comando conhecida, é ótimo exercício.

O que precisa fazer sozinho, mesmo com IA:

1. Explicar cada linha que submete; se não sabe explicar, não submete.
2. Depurar: reproduzir, isolar e confirmar uma hipótese antes de colar o erro na IA.
3. Escrever testes e reconhecer um teste fraco.
4. Dominar HTTP, SQL (joins, índices, paginação), Git e terminal Linux.
5. Implementar estruturas e algoritmos básicos e estimar complexidade (Livro 4, Capítulo 2 (Algoritmos)).
6. Reconhecer falhas comuns de segurança num diff gerado.
7. Escrever um bom pedido ao agente e uma boa descrição de PR.

O que estudar: fundamentos que não envelhecem (lógica, uma linguagem robusta, testes, design, OWASP Top 10) antes das ferramentas da moda. COMO APRENDER JAVA E CONSEGUIR UM EMPREGO EM 6 MESES sugere usar a IA para gerar exercícios, não para resolvê-los. Em 5 Erros que Todo Júnior comete NO CÓDIGO que o Impedem de virar PLENO (E Como Resolvê-los), O Novo Programador lista condicionais desnecessárias, falta de tratamento de erro, nomes ruins, desconhecer funções nativas e repetição; a IA comete os mesmos erros, e reconhecê-los já é revisar como pleno.

Armadilhas: não conseguir começar nada sem IA; ilusão de competência; e habilidades que nunca se formam (no estudo da Anthropic, a maior perda foi em depuração).

> **Dica**
> Use "gerar e depois entender": peça o código e depois peça à IA que *te faça perguntas* sobre ele ("onde isto pode falhar?").

Portfólio (projetos pessoais): uma API com autenticação, autorização, paginação, migrations, testes e CI; uma pasta `specs/` com as especificações que guiaram os agentes; um relatório de auditoria de um app gerado por IA; uma contribuição real a um projeto open source. Na entrevista, responda "como você sabe que está certo?" com testes, não com "a IA disse".

Pleno

Princípios em foco: Poliglota por Papel (justificar por escrito cada linguagem), Soberania Local (avaliar um modelo local para tarefas do time), Conhecimento Destilado e IA como Equipe Paralela.

O que se espera: entregar features de ponta a ponta: spec com Dado/Quando/Então, tarefas, orquestração, revisão, deploy e acompanhamento. Configurar o harness do repositório (regras, comandos de teste, skills, hooks); conduzir refatorações em passos reversíveis; diagnosticar produção com logs e métricas; controlar custo de tokens; integrar LLMs no produto com avaliação básica. Para IA aplicada, Essential Skills for Becoming an AI Engineer: RAG, AI Agents, & More, da IBM Technology, é um bom mapa.

Armadilhas: virar "gerente de agentes" e parar de programar; aprovar PRs enormes porque "os testes passam"; terceirizar o design ao modelo.

Sênior e Staff

Princípios em foco: Fusão de Repositórios (consolidar ferramentas internas num harness coeso), Soberania Local como decisão de custo e privacidade, Engenharia Reversa como Escola para avaliar legados dentro da lei e Crítica Antibajulação institucionalizada nas regras e revisões.

O que se espera: arquiteturas com fronteiras claras e sandbox para agentes; política de IA do time (ferramentas aprovadas, dados que podem ir a cada provedor, revisão obrigatória); evals com tarefas reais; modelagem de ameaças de agentes; medir impacto com métricas DORA, não com linhas de código; orçamento local × nuvem; e formar juniores num mundo em que as tarefas de aprendizado foram automatizadas.

Armadilhas: ceticismo paralisante ou entusiasmo acrítico; esquecer de formar a próxima geração; confiar na própria percepção de ganho.

Matriz de competências

E = esperado; **D** = desejável; **—** = ainda não.

Competência	Júnior	Pleno	Sênior/Staff
Decomposição	E: a própria tarefa	E: feature em tarefas para agentes	E: projeto entre times
Especificação	D: critérios de aceite	E: spec completa	E: padrão de spec do projeto
Contexto	E: bons pedidos	E: regras, skills, MCP	E: harness e políticas
Verificação	E: testes próprios	E: estratégia de testes, CI	E: pipeline de qualidade
Arquitetura	—/D: entender a existente	E: da feature	E: do sistema
Segurança	E: reconhecer falhas	E: revisar e corrigir	E: modelagem de ameaças
Avaliação de modelos	—/D	D/E: eval simples	E: evals contínuas
Custos	D: noção de tokens	E: custo por tarefa	E: orçamento
Comunicação	E: PRs, perguntas	E: specs, mentoria	E: influência entre times

Dica

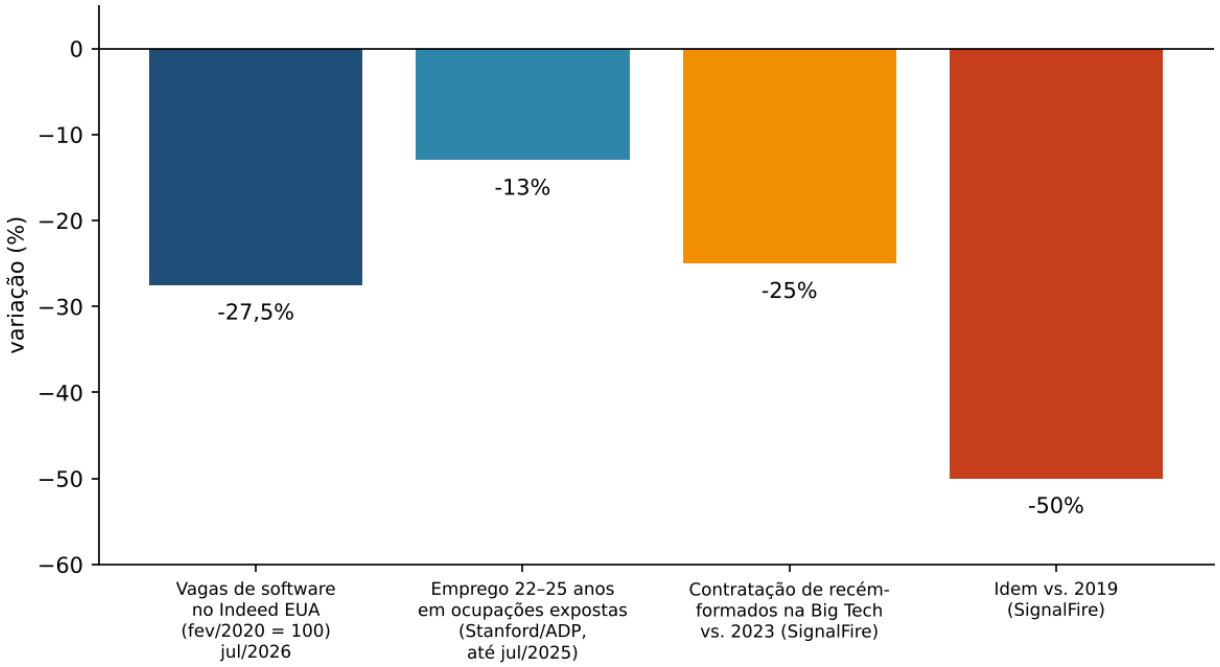
Rotina semanal enxuta: um problema resolvido sem IA, uma spec antes de pedir código, um diff gerado revisado à mão, um experimento comparando dois modelos e um "diário de decisões" anotando em uma linha por que aceitou ou rejeitou cada sugestão da IA.

O mercado: vagas e processos seletivos

Para começar

Mudanças tecnológicas mexem primeiro na porta de entrada. Com a IA, os primeiros sinais aparecem nas vagas de iniciante em software.

- **Emprego de jovens**: estudo de Brynjolfsson, Chandar e Chen (Stanford) encontrou queda de 13% no emprego de 22 a 25 anos nas ocupações mais expostas à IA nos EUA, até julho de 2025.
- **Recém-formados**: segundo a SignalFire, são só 7% das contratações das grandes empresas de tecnologia, mais de 50% abaixo de 2019.
- **Vagas**: no Indeed (EUA, julho de 2026), as vagas de software estão cerca de 27,5% abaixo de fevereiro de 2020, mas subiram quase 15% desde o início de 2025; 71% do aumento veio de vagas sêniores.



Indicadores do mercado de entrada em software nos EUA, de fontes diferentes, lado a lado só para dar a ordem de grandeza. Fonte: elaboração própria com dados de Indeed (via OfficeChai), Stanford (via LeadDev) e SignalFire.

Os dados combinam com a tese de que a IA automatiza tarefas de iniciante, mas há ressalvas: a queda começou em 2022, junto com juros altos e o fim das contratações da pandemia; as vagas voltaram a crescer; e quase tudo é dos EUA. No Brasil não há série pública confiável sobre vagas júnior. A conclusão honesta: **a porta ficou mais estreita e mais exigente**. O júnior que só produz código com IA compete com a IA; o que entende e verifica compete com os que não fazem isso.

As vagas pedem IA como requisito básico, Git, testes, CI/CD e APIs REST até no nível júnior, como mostram Fernanda Kipper e Rodrigo em O que VAGAS de MOBILE estão pedindo? - Avaliando vagas, além de RAG e agentes nas vagas de IA aplicada.

Nas entrevistas, a Meta passou em outubro de 2025 a usar uma rodada de cerca de 60 minutos **com IA permitida**, numa base de código com vários arquivos: corrigir um bug, implementar o algoritmo e otimizar. Avalia-se, entre outras coisas, a **verificação** do que a IA produziu. A tendência é trocar o algoritmo no quadro branco por trabalho em código existente, mostrar o processo e defender cada decisão.

Você sabia?

O Git foi escrito por Linus Torvalds em abril de 2005, em poucas semanas, depois que o kernel Linux perdeu o acesso gratuito ao BitKeeper. Meses depois, a manutenção passou a Junio Hamano, que a mantém até hoje.

Em resumo

O AI-Driven Engineer é definido mais pelo que garante do que pelo que digita. A adoção é quase universal, a confiança é baixa e os ganhos são reais, mas condicionais. Em todos os níveis cresce o valor de decompor, especificar, verificar, ler código, cuidar da segurança e comunicar.

Fontes deste capítulo

Vídeos

Título	URL	Canal	Ponto principal
COMO FICAR PERIGOSAMENTE BOM EM PROGRAMAÇÃO	https://www.youtube.com/watch?v=SI0Ie2zHTd8	Área Tech Brasil	Codificador de prompt × orquestrador
COMO APRENDER JAVA E CONSEGUIR UM EMPREGO EM 6 MESES	https://www.youtube.com/watch?v=lSHPYwyzFxk	Área Tech Brasil	Roteiro até a primeira vaga
5 Erros que Todo Júnior comete NO CÓDIGO que o Impedem de virar PLENO (E Como Resolvê-los)	https://www.youtube.com/watch?v=3ICKCyda8sE	O Novo Programador	Hábitos que minam a confiança dos revisores
SPEC Driven: A MELHOR FORMA De Usar IA em 2026!	https://www.youtube.com/watch?v=ZM8WW8_HEeE	O Novo Programador	SDD em seis fases
O jeito CERTO de usar IA para programar (ninguém te explica isso)	https://www.youtube.com/watch?v=UOKCuEzeOAo	Sujeito programador	Vibe coding, fluxo agêntico e Spec Kit
Revisei código gerado por I.A..... PREOCUPANTE	https://www.youtube.com/watch?v=u1nhcio8_iQ	Augusto Galego	Defeitos em app gerado sem guia
Como Virar Um ARQUITETO DE SOFTWARE	5 PASSOS Que Vão Mudar Sua Carreira de PROGRAMADOR	https://www.youtube.com/watch?v=mKlltWid_L0	Renato Augusto	Cinco passos até arquiteto
O que VAGAS de MOBILE estão pedindo? - Avaliando vagas	https://www.youtube.com/watch?v=Oa7ulYfl1pM	Fernanda Kipper	Requisitos que viraram padrão nas vagas
Essential Skills for Becoming an AI Engineer: RAG, AI Agents, & More	https://www.youtube.com/watch?v=XN3xNJvWXsc	IBM Technology	Pilha de habilidades do engenheiro de IA

Blog AkitaOnRails

Título	URL	Autor	Ponto principal
LLMs + Agentes = competente o suficiente	https://akitaonrails.com/2026/10/12/llms-agentes-competente-o-suficiente/	Fabio Akita	Ganho veio do harness; engenharia é diferencial
Existem linguagens melhores pra programar com LLMs?	https://akitaonrails.com/2026/10/11/existem-linguagens-melhores-pra-programar-com-llms/	Fabio Akita	Testes pesam mais que a linguagem
LLM Benchmark v4 — o veredito de open source até out/2026	https://akitaonrails.com/2026/10/22/llm-benchmark-v4-o-veredito-de-open-source-ate-out-2026/	Fabio Akita	Tamanho não garante qualidade; ruído alto

Autor do conceito e referências

Título	URL	Autor	Ponto principal
Perfil marcusbsilva no GitHub	https://github.com/marcusbsilva	Marcus B. Silva	Autor do conceito AI-Driven Engineer
llama.cpp	https://github.com/ggml-org/llama.cpp	Georgi Gerganov e colaboradores	Inferência local de LLMs; formato GGUF
A Short History of Git	https://git-scm.com/book/en/v2/Getting-Started-A-Short-History-of-Git	Pro Git (Chacon e Straub)	Origem do Git após o fim do BitKeeper

Pesquisas, estudos e reportagens (verificados em 30/10/2026)

Título	URL	Publicação	Ponto principal
Stack Overflow Developer Survey 2025 — AI	https://survey.stackoverflow.co/2025/ai	Stack Overflow	84% usam IA; ~33% confiam
Stack Overflow survey shows developers trust AI with caveats	https://itbrief.co.uk/story/stack-overflow-survey-shows-developers-trust-ai-with-caveats	IT Brief UK	Pesquisa 2026: 66% usam assistentes ou agentes
How are developers using AI? Inside our 2025 DORA report	https://blog.google/innovation-and-ai/technology/developers-tools/dora-report-2025/	Google	90% de adoção; IA como multiplicador
The State of Developer Ecosystem 2025	https://blog.jetbrains.com/research/2025/10/state-of-developer-ecosystem-2025/	JetBrains	85% usam IA regularmente
Octoverse: A new developer joins GitHub every second as AI leads TypeScript to #1	https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/	GitHub	180 mi de devs; Brasil em 4.º
Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity (comentário)	https://simonwillison.net/2025/Jul/12/ai-open-source-productivity/	Simon Willison	METR: 19% mais lentos com IA
Copilot developer productivity	https://infoq.com/news/2024/09/copilot-developer-productivity/	InfoQ	+26% de PRs por semana
How AI assistance impacts the formation of coding skills	https://www.anthropic.com/research/AI-assistance-coding-skills	Anthropic	Delegar tudo reduz a compreensão
Young devs see steepest losses in exposed roles, Stanford study finds	https://leaddev.com/hiring/young-devs-steepest-losses-exposed-roles-stanford-study-finds	LeadDev	−13% no emprego de 22–25 anos
The SignalFire State of Tech Talent Report 2025	https://www.signalfire.com/blog/signalfire-state-of-talent-report-2025	SignalFire	Recém-formados: 7% das contratações
US Software Job Postings Are Up 15% Since Launch Of Claude Code	https://officechai.com/ai/us-software-job-postings-are-up-15-since-launch-of-claude-code-while-overall-job-postings-are-down-7-indeed-data/	OfficeChai (dados Indeed)	Vagas em alta, concentradas em sêniores
Meta's AI-Enabled Coding Interview	https://hellointerview.com/blog/meta-ai-enabled-coding	Hello Interview	Entrevista com IA permitida avalia verificação

CAPÍTULO 4 ·

Desenvolvimento seguro com IA e o futuro do Pull Request

> *A IA passou a escrever uma fatia enorme do código do mundo: ótimo para a velocidade, um problema novo para a segurança, a revisão e o aprendizado. Este capítulo mostra por que código gerado por IA traz riscos próprios, apresenta as normas e um pipeline de segurança, propõe um novo modelo de Pull Request e compara, com dados e limitações, o que acontece com tempo, qualidade e aprendizado com e sem IA.*

Para começar

Imagine um ajudante de obra dez vezes mais rápido que qualquer pedreiro, mas que às vezes esquece a viga ou usa um material que não existe. A obra anda, mas o mestre de obras precisa conferir muito mais. O ajudante é o agente de código (Capítulo 1 (Programando com IA)); o mestre de obras é o revisor do **Pull Request** (PR), o pedido formal para juntar uma alteração ao código principal.

Por que código gerado por IA traz riscos próprios

Para começar

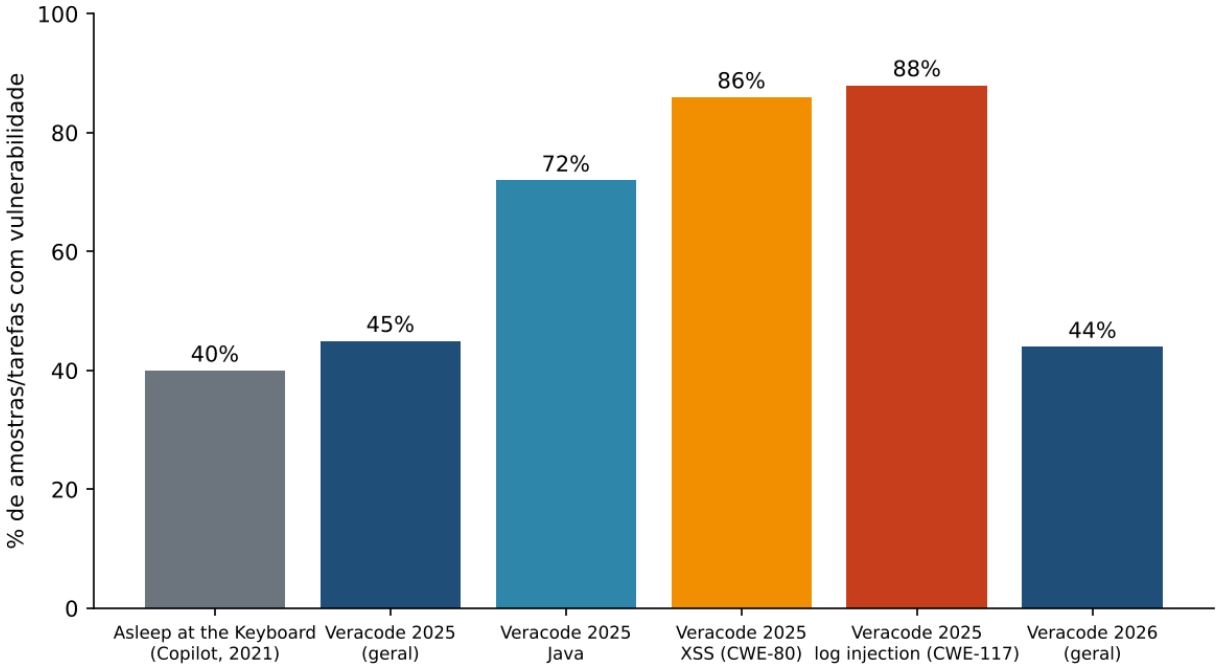
Um modelo aprende lendo bilhões de linhas de código público, muitas antigas ou inseguras, e produz o que é *estatisticamente plausível*, não o que é seguro. Além disso, surgem riscos novos: pacotes inventados, senhas copiadas para o código e agentes enganados por instruções escondidas.

Vulnerabilidades: o que os estudos mediram

Uma **vulnerabilidade** é uma fraqueza explorável por um atacante. As fraquezas são catalogadas pelo **CWE** da MITRE (CWE-79 é XSS; CWE-89, injeção de SQL); um caso concreto num produto recebe um **CVE**.

- **"Asleep at the Keyboard?" (Pearce et al., NYU, 2021)**: 89 cenários de risco, 1.689 programas gerados pelo Copilot, cerca de **40%** vulneráveis. Limitação: Copilot de 2021 e cenários feitos para provocar o erro.
- **Perry et al. (Stanford, CCS 2023)**: com 47 participantes, quem usava IA escreveu código **menos seguro** em quatro de cinco tarefas e **tinha mais confiança** de que estava seguro. Limitação: amostra pequena, maioria de estudantes.
- **Veracode 2025**: mais de 100 modelos, 80 tarefas com uma solução segura e outra insegura; em **45%** o modelo escolheu a insegura (Java acima de 70%; XSS 86%). Modelos maiores melhoraram em compilar, **não** em segurança.

- **Veracode 2026**: taxa de aprovação em segurança praticamente parada em cerca de 55% a 56%.



Porcentagem de amostras ou tarefas com vulnerabilidade no código gerado por IA, por estudo. As metodologias diferem; as barras mostram ordem de grandeza e estabilidade ao longo de cinco anos. Fonte: elaboração própria com dados de Pearce et al. (2021) e Veracode 2025 e 2026.

> **Atenção**
> "45% do código de IA é vulnerável" é uma simplificação errada. O correto: em 45% das *tarefas com armadilha de segurança*, o modelo caiu nela. O recado continua: código de IA passa pelos mesmos controles que código humano, e um pouco mais.

O vídeo Revisei código gerado por I.A..... PREOCUPANTE, de Augusto Galego, mostra isso em miniatura: um app de tarefas gerado pelo Cursor sem controle de usuário, sem paginação, expondo o objeto de domínio inteiro e com JWT sem algoritmo fixo (porta para *algorithm confusion*). Quem define padrões e arquitetura é o humano.

Dívida técnica

A GitClear analisou 211 milhões de linhas alteradas entre 2020 e 2024: linhas **movidas** (sinal de refatoração) caíram de cerca de 25% para menos de 10%, linhas **coladas** subiram de 8,3% para 12,3% e blocos duplicados cresceram cerca de 8 vezes em 2024. É correlação, de uma empresa do setor, mas combina com PRs maiores e revisões mais longas.

Pacotes alucinados e slopsquatting

Modelos às vezes recomendam bibliotecas que **não existem**. Spracklen e colegas (USENIX Security 2025) testaram 16 modelos com 576 mil amostras: **19,7%** das referências a pacotes eram alucinações, e os nomes falsos se repetem. Daí o **slopsquatting**: o atacan-

te registra com código malicioso os nomes que as IAs costumam inventar e espera alguém, ou um agente, rodar `npm install`.

> **Dica**
> Pesquise todo pacote antes de instalar (idade, downloads, mantenedores); use registro interno com lista aprovada; trave versões com lockfile; rode SCA; e não deixe agentes instalarem dependências sem aprovação humana.

Segredos vazados

Segundo a GitGuardian, cerca de 29 milhões de segredos (senhas, chaves de API, tokens) vazaram no GitHub público em 2025 (+34%). Commits feitos com ajuda de IA vazaram segredos em cerca do **dobro** da taxa média, e mais de 24 mil segredos apareceram em configurações de servidores MCP (Livro 2, Capítulo 4 (Agentes e MCP)). As regras, lembradas em O MÍNIMO que um dev precisa saber sobre segurança: `.env` fora do Git, segredos num cofre e, se uma senha foi commitada, **trocá-la**, porque apagar o commit não a "desvaza".

Prompt injection em agentes

Num LLM (modelo de linguagem), dados e instruções são o mesmo texto. **Prompt injection** é colocar, no texto que a IA vai ler, instruções que ela obedece; é o risco número 1 do OWASP Top 10 para LLMs. O vídeo What Is a Prompt Injection Attack?, da IBM Technology, separa a injeção **direta** da **indireta** (escondida numa página, PDF ou e-mail). Em Securing AI Agents: How to Prevent Hidden Prompt Injection Attacks, um agente de compras paga o dobro por causa de texto preto sobre fundo preto.

Agentes de código leem issues, comentários e READMEs, qualquer um deles com possíveis instruções maliciosas. No caso **s1ngularity** (agosto de 2025), uma versão maliciosa do pacote Nx usou as CLIs de IA instaladas na máquina da vítima para procurar credenciais, expondo 2.180 contas. Skills de terceiros também são porta de entrada, como alerta 5 Best Practices for Building AI Agent Skills.

Padrões e normas

> **Para começar**
> Software tem seus "códigos de obra", e a IA os torna mais importantes: viram o gabarito para conferir o que a máquina produziu e podem virar instruções ("siga o OWASP ASVS nível 2") no arquivo de regras do agente.

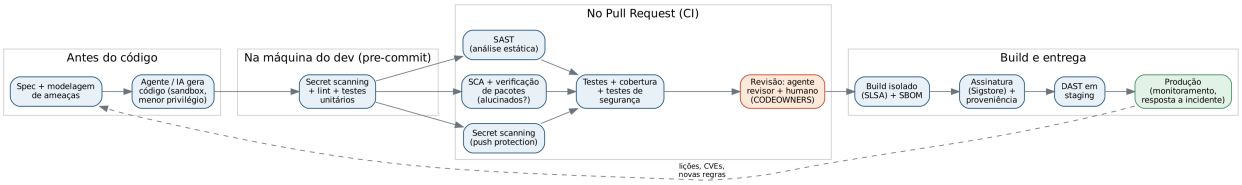
Referência	O que é, em palavras simples	Exemplo do dia a dia
OWASP Top 10:2025	Os dez riscos mais graves de sites e sistemas web	A lista das dez falhas mais comuns que um vistoriador procura numa casa
OWASP Top 10 for LLM 2025	Os principais riscos de apps com IA, como prompt injection	Um bilhete escondido que faz o assistente obedecer a um estranho
OWASP ASVS	Lista de requisitos que dá para testar, em três níveis	Uma ficha de vistoria com itens de “sim” ou “não”
NIST SSDF (SP 800-218 e 800-218A)	Boas práticas para todo o processo de desenvolvimento	O passo a passo de uma cozinha que segue as normas sanitárias
SLSA, SBOM, Sigstore	Provar a origem do software, listar seus componentes e assiná-lo	O rótulo de um alimento com ingredientes, lote e selo
CWE Top 25	As 25 fraquezas de código mais perigosas	Os 25 defeitos que mais derrubam prédios
ISO/IEC 27001	Norma certificável de gestão da segurança da informação	Um selo de qualidade que a empresa ganha após auditoria

Com IA, essas referências viram duas coisas: um checklist para revisar o código gerado e instruções no arquivo de regras do agente (por exemplo, “siga o OWASP ASVS nível 2”). Na lista de 2025, controle de acesso quebrado está em 1.º lugar, e entraram cadeia de suprimentos (as dependências de terceiros) e tratamento de erros e exceções.

No Brasil, a LGPD se soma a tudo isso: um agente que copia dados pessoais de produção para um modelo externo pode configurar tratamento indevido.

O pipeline seguro

SAST (análise estática) lê o código sem executá-lo (Semgrep, CodeQL); **DAST** (análise dinâmica) ataca a aplicação rodando (OWASP ZAP); **SCA** (análise de composição) verifica as dependências (Dependabot, OSV-Scanner); **secret scanning** bloqueia credenciais antes do push (gitleaks).



Pipeline de desenvolvimento seguro com IA: da spec e da modelagem de ameaças ao monitoramento, com verificações automáticas em cada etapa e revisão por agente e humano no PR. Fonte: elaboração própria.

O princípio é a **defesa em profundidade**: várias camadas imperfeitas e independentes. É o espírito "verificar e depois confiar" de [The Problem with Jev: Can You Trust AI Decisions?]: tratar a IA como componente não confiável. A camada mais barata é a mais cedo: um hook de pre-commit com gitleaks custa segundos.

Checklist do agente: sandbox sem credenciais de produção; token sem permissão de merge ou publicação; instalação de pacotes e rede com aprovação humana; skills e servidores MCP revisados como dependências; conteúdo externo tratado como dado, nunca como ordem.

Checklist do código: entradas validadas e consultas parametrizadas; autorização checada no servidor em cada operação; criptografia e JWT com bibliotecas consagradas e al-

goritmo fixo; saídas escapadas; toda dependência nova justificada; logs sem segredos nem dados pessoais; testes dos caminhos de falha.

O Pull Request precisa mudar

> **Para começar**
> O PR nasceu num mundo em que escrever era caro e ler era barato. A IA inverteu a equação: escrever ficou quase de graça, e ler com atenção continua caro. Quando o volume explode e os revisores não, a fila trava ou a revisão vira carimbo.

Os sintomas

Segundo o Octoverse de outubro de 2026, mais de 40% dos PRs no GitHub já começavam com rascunho de agente em setembro (eram 17% em março). O relatório da Faros AI (2025, mais de 10 mil desenvolvedores) achou nas equipes com alta adoção +98% de PRs integrados, mas **PRs 154% maiores**, **revisão 91% mais longa**, +9% de bugs e nenhum ganho de entrega no nível da empresa. O gargalo só mudou de lugar.

Em código aberto, o **AI slop** (conteúdo genérico e frequentemente errado gerado em massa, explicado em What is AI Slop? Low-Quality AI Content Causes, Signs, & Fixes) levou projetos a reagir:

Projeto	Data	Reação
Gentoo, NetBSD	2024	Proibiram ou restringiram código gerado por IA
curl	jan/ 2026	Encerrou o bug bounty: relatos válidos caíram para menos de 5%, afogados em relatos fabricados
Kernel Linux	2026	Agentes não assinam `Signed-off-by`; uso de IA declarado com `Assisted-by:`
LLVM	2026	IA proibida nas "good first issues" e em PRs sem aprovação humana
Fedora	out/ 2025	Sem proibição: responsabilidade do autor, transparência e revisão humana
GitHub	fev/ 2026	Controles para mantenedores limitarem PRs

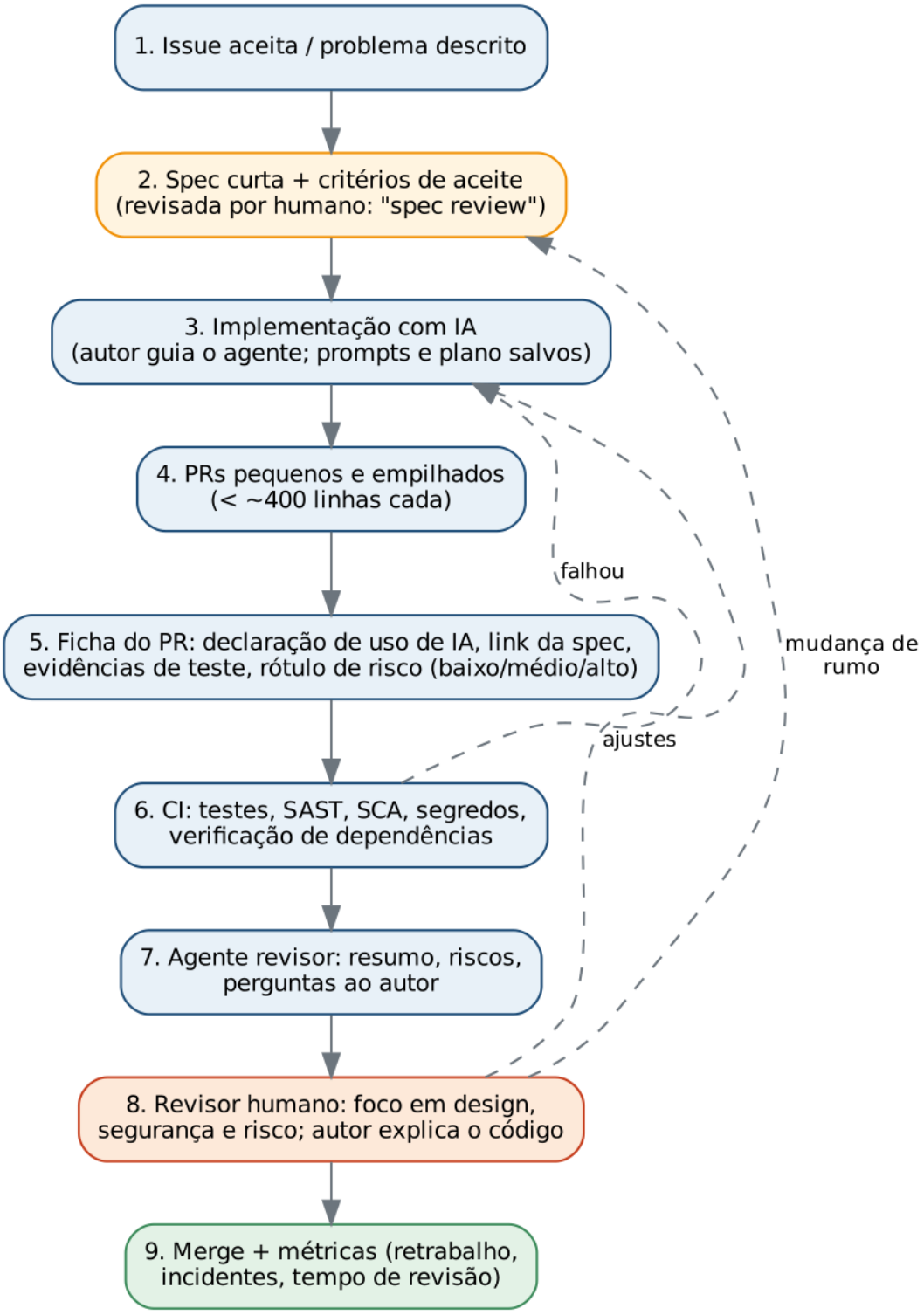
A tendência comum: **o tempo de revisão virou recurso escasso**. Como não há detector confiável de código gerado, as políticas que funcionam atacam o comportamento (PR sem issue, enorme, autor que não sabe explicar), não a ferramenta.

A proposta: um contrato de proveniência e de entendimento

O PR deve carregar **de onde veio** a mudança, **o que pretende**, a **evidência** de que funciona e a **prova de que o autor a entende**:

1. **Declaração de uso de IA** no template ou num trailer como `Assisted-by:`. É metadado, não confissão.
2. **Spec anexada e revisada antes do código.** Se gerar é barato, o erro caro é construir a coisa errada (Capítulo 3 (AI-Driven Engineer)).

3. **PRs pequenos e empilhados** (*stacked PRs*), com alerta no CI acima de cerca de 400 linhas.
4. **Evidência de testes**: testes novos, saída do CI e capturas para mudanças visuais. "A IA disse que está pronto" não conta.
5. **Rótulos de risco** (baixo, médio, alto), sugeridos pelos caminhos alterados via CODEOWNERS, que definem quem aprova.
6. **Revisão em dois papéis**: o agente faz a primeira passada (resumo, padrões inseguros, aderência à spec); o humano julga arquitetura, negócio e risco. Agente nunca aprova sozinho mudança de risco alto.
7. **O autor explica o código** com as próprias palavras nos PRs de risco médio e alto.
8. **Política clara** (`CONTRIBUTING.md`, `AI_POLICY.md`) e **métricas**: tamanho de PR, tempo até revisão, retrabalho, defeitos escapados.



Fluxo proposto para o Pull Request na era da IA: spec revisada antes do código, PRs pequenos com proveniência e evidências, triagem por CI e agente, e humano focado em design, segurança e entendimento. Fonte: elaboração própria.

```
## O que muda e por quê
Issue: #123 · Spec: docs/specs/0042-reset-senha.md
## Uso de IA
Ferramenta / partes geradas / plano ou prompts principais
## Risco
[ ] baixo  [ ] médio  [x] alto (toca autenticação)
## Evidências
Testes novos: test_reset_token_expira, test_reset_token_uso_unico
Saída do CI: (link)
## Explicação do autor
Por que esta abordagem? O que pode dar errado? O que NÃO foi testado?
```

O modelo também reduz a resistência à IA: declarar vira rotina neutra (o que tira o incentivo a esconder), a revisão volta a ser julgamento e não caça a `console.log`, e o valor do desenvolvedor (spec, arquitetura, risco, explicação) fica visível. Comece pequeno: campos "Uso de IA" e "Evidências" no template, alerta de tamanho no CI e um mês de medição.

Você sabia?

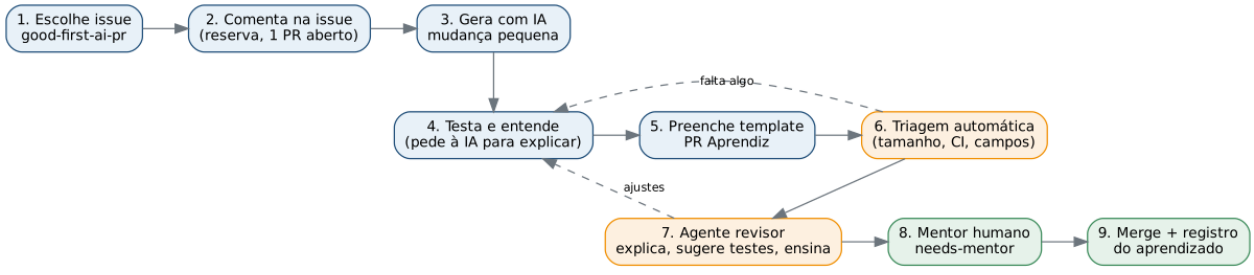
O Pull Request não é invenção do GitHub. O Git traz o comando `git request-pull`, que gera um texto pedindo a outro mantenedor que "puxe" as mudanças; muitas contribuições ao kernel Linux ainda chegam assim, por e-mail. O GitHub transformou o pedido em página com comentários em 2008.

Um padrão para iniciantes: o "PR Aprendiz"

Para começar

Numa cozinha profissional, o estagiário começa descascando legumes, com uma ficha simples e um chef que corrige com paciência. O **PR Aprendiz** é essa cozinha aplicada ao código aberto: um formato para quem programa com vibe coding contribuir sem medo, com pedido pequeno, IA declarada e algo aprendido ao final.

Princípios: uma mudança por PR; declarar a IA e o prompt sem constrangimento; saber dizer com as próprias palavras o que o código faz; registrar o que aprendeu e o que não entendeu; máquina faz a triagem e humano faz a mentoria; quotas protegem o projeto.



Fluxo do PR Aprendiz: da escolha de uma issue rotulada até o merge com registro do aprendizado. Em laranja, etapas automáticas; em verde, humanas. Fonte: elaboração própria.

```
## PR Aprendiz
### 1. O que eu queria (link da issue)
### 2. O que mudou, em palavras simples
### 3. Como testei
### 4. Qual IA usei e o prompt (e o que eu mesmo alterei)
### 5. O que eu aprendi
### 6. Dúvidas (não há pergunta boba)
- [ ] Li o CONTRIBUTING.md   - [ ] Muda uma coisa só   - [ ] Rodei os testes
```

Rótulos: `good-first-ai-pr` marca issues pequenas e de baixo risco (documentação, mensagens de erro, testes); `needs-mentor` chama um mentor quando há dúvidas. **Agente revisor:** age como tutor, não juiz: explica o diff em linguagem simples, sugere testes, aponta o conceito por trás do ajuste, faz uma ou duas perguntas de entendimento e nunca aprova sozinho.

Etiqueta: o mentor responde em até uma semana, comenta o código e não a pessoa e fecha com motivo quando não dá para aproveitar; o aprendiz mantém um PR aberto por vez e responde com as próprias palavras, não colando a IA. **Progressão:** Aprendiz (só issues rotuladas, mentoria obrigatória) → Colaborador (após 3 PRs aceitos; risco baixo e médio, spec curta própria) → Revisor (após 10 PRs; mentora outros, que é a melhor forma de fixar o que se aprendeu).

Exemplo. Uma usuária que nunca programou resolve a issue "traceback quando o arquivo não existe": a IA trata a exceção e mostra "Arquivo 'x.csv' não encontrado"; ela adiciona um teste, registra que aprendeu o que é `try/except` e pergunta se seria melhor checar a existência antes. O agente explica que tratar a exceção é preferível, porque o arquivo pode sumir entre a checagem e a abertura (uma *condição de corrida*), e a mentora aprova.

Contra o slop: quotas por nível, triagem que devolve com explicação PRs sem issue, com campos vazios, grandes demais ou com CI quebrado, e escopo restrito a issues rotuladas. O campo "o que eu aprendi" e as perguntas do agente custam pouco a quem é sincero e muito a quem faz spam. Tudo cabe em três peças: uma seção no `CONTRIBUTING.md`, o template em `.github/PULL_REQUEST_TEMPLATE/aprendiz.md` e uma workflow de triagem.

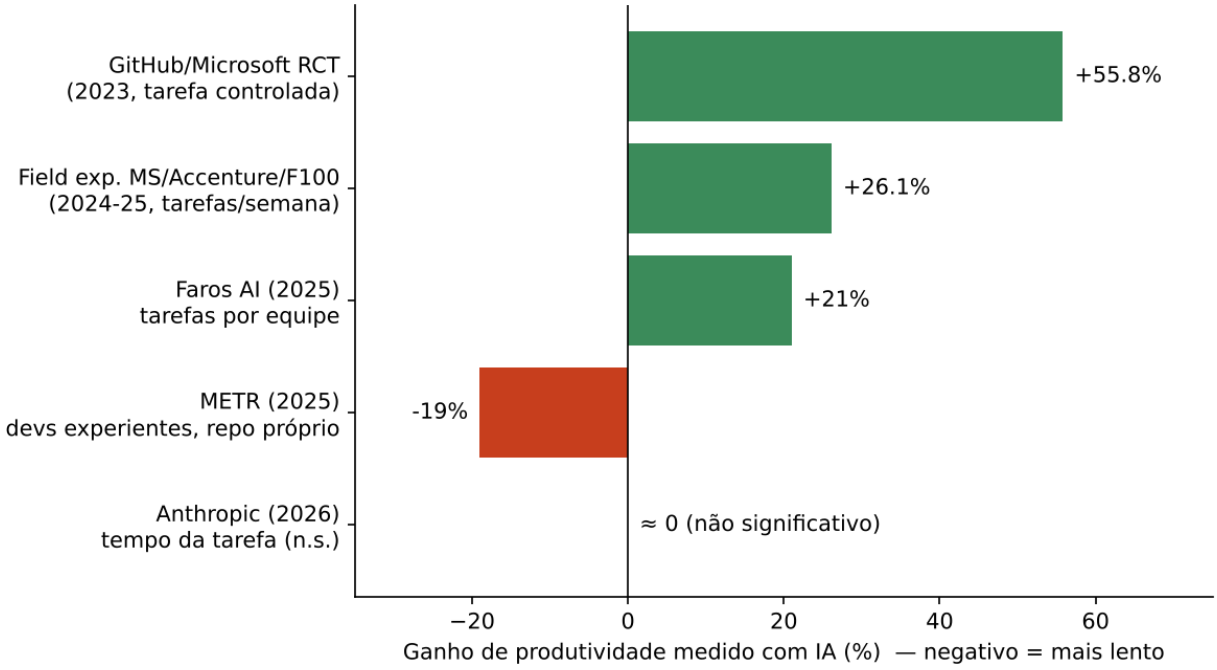
> **Você sabia?**
> Em 2020, o Hacktoberfest, que dava camisetas a quem abrisse PRs em outubro, gerou uma onda de PRs inúteis que sobrecarregou mantenedores. A regra mudou: só participam repositórios que optam pelo tópico `hacktoberfest`, e só contam PRs aceitos.

Com IA × sem IA: o que os fatos mostram

> **Para começar**
> "A IA me deixa 10 vezes mais produtivo" e "a IA só atrapalha" costumam ser impressões, e impressões enganam. Para responder com honestidade, olhamos experimentos com sorteio (**RCTs**) em três dimensões: tempo, qualidade e aprendizado.

Tempo e produtividade

- **Peng et al. (2023)**: com Copilot, um servidor HTTP saiu **55,8% mais rápido**. Tarefa única, pequena, estudo do fabricante.
- **Cui, Demirer et al. (2025)**: três experimentos de campo com 4.867 desenvolvedores; **+26%** de tarefas por semana, com ganho maior entre os menos experientes. "Tarefa concluída" não mede qualidade.
- **METR (2025)**: 16 veteranos de grandes projetos abertos, 246 issues reais sorteadas; com IA levaram **19% mais tempo**, mas acharam ter ganho 20%. Em fevereiro de 2026, o METR disse ser "provável" que as ferramentas atuais acelerem, com "evidência muito fraca" do tamanho.
- **DORA**: em 2024, mais adoção de IA vinha com vazão −1,5% e estabilidade −7,2%; em 2025 a relação com a vazão ficou positiva, mas a instabilidade seguiu subindo. A IA é um *amplificador* dos fundamentos que o time já tem.



Efeito medido do uso de IA na velocidade em estudos diferentes. Verde: ganho; vermelho: perda; cinza: não significativo. Métricas, populações e ferramentas diferem. Fonte: elaboração própria com dados de Peng et al. (2023), Cui et al. (2025), Faros AI (2025), METR (2025) e Anthropic (2026).

Aprendizado

- **Anthropic (2026)**: 52 engenheiros, maioria júnior, aprendendo a biblioteca Trio. Com IA, 50% no quiz; sem IA, 67%, com tempo praticamente igual. Quem gerou e depois pediu explicações teve de 65% a 86%; quem só delegou ficou abaixo de 40%.
- **Bastani et al. (PNAS, 2025)**: quase mil alunos de matemática; o ChatGPT comum melhorou os exercícios e piorou a prova sem IA em 17%; uma versão "tutora" eliminou a perda.
- **MIT Media Lab (2025)**: preprint com 54 estudantes sugere "dívida cognitiva" ao escrever com ChatGPT. Sinal de alerta, não prova.

Resumo dos estudos:

- **Tempo:** Peng 2023, 55,8% mais rápido (uma tarefa só); Cui 2025, +26% de tarefas; METR 2025, 19% mais lento (só 16 devs); Faros 2025, nenhum ganho na empresa; Anthropic 2026, sem diferença.
- **Qualidade e segurança:** Faros 2025, PRs 154% maiores e +9% de bugs (estudo observacional); Veracode 2025/26, falha em cerca de 45% das tarefas com armadilha.
- **Aprendizado:** Anthropic 2026, −17 pontos no quiz, mas sem perda para quem pedia explicações (52 pessoas, curto prazo); Bastani 2025, −17% na prova, perda anulada pela IA tutora (só matemática).

Três padrões emergem: a IA acelera tarefas pequenas e novas e ajuda mais quem tem menos contexto; o ganho individual some em PRs maiores e instabilidade se o processo não muda; e delegar sem entender custa aprendizado, enquanto usar a IA para entender o preserva. Os modelos de 2026 são mais capazes que os desses estudos, mas a segurança (Veracode 2026) não melhorou no mesmo ritmo.

Recomendações

- **Quem aprende:** use a IA como tutora, não como ghostwriter (quem escreve por você); nada entra no projeto sem que você saiba explicar; reserve tempo sem IA para fundamentos.
- **Profissionais:** use IA onde rende (protótipos, testes, migrações, documentação); escreva a spec antes do prompt; desconfie da sua sensação de velocidade.
- **Líderes:** adote o PR com proveniência, risco e explicação; meça com DORA e retrabalho, não com linhas; proteja as tarefas de formação dos juniores; tenha uma política de IA curta, incluindo que dados nunca vão para modelo externo.

> **Em resumo**
> A IA é um amplificador. Com specs, testes, PRs pequenos, revisão por risco e pipeline de segurança, ela acelera a entrega sem cobrar em segurança e aprendizado. Sem isso, acelera também bugs, vulnerabilidades e desenvolvedores que não entendem o que entregam.

Fontes deste capítulo

Vídeos

Título	URL	Autor ou canal	Ponto principal	
Revisei código gerado por I.A..... PREOCUPANTE	https://www.youtube.com/watch?v=u1nhcio8_iQ	Augusto Galego	Falhas de autorização e JWT em app gerado	30/10/2026
O MÍNIMO que um dev precisa saber sobre segurança	https://www.youtube.com/watch?v=aGVN6aHKkE0	Augusto Galego	Menor privilégio e segredos fora do repositório	30/10/2026
What Is a Prompt Injection Attack?	https://www.youtube.com/watch?v=jrHRe9lSqqA	IBM Technology	Injeção direta e indireta	30/10/2026
Securing AI Agents: How to Prevent Hidden Prompt Injection Attacks	https://www.youtube.com/watch?v=5ZA1lTxTH3c	IBM Technology	Agente enganado por texto oculto	30/10/2026

Título	URL	Autor ou canal	Ponto principal	
5 Best Practices for Building AI Agent Skills	https://www.youtube.com/watch?v=qYNs80FKIVc	IBM Technology	Skills de terceiros como dependências	30/10/2026
The Problem with Jev: Can You Trust AI Decisions?	https://www.youtube.com/watch?v=_yO_Q7qo0HQ	IBM Technology	Verificar e depois confiar	30/10/2026
What is AI Slop? Low-Quality AI Content Causes, Signs, & Fixes	https://www.youtube.com/watch?v=hl6mANth6oA	IBM Technology	O que é AI slop	30/10/2026

Estudos, normas e reportagens

Título	URL	Autor ou veículo	Ponto principal	
AI-Generated Code Poses Major Security Risks in Nearly Half of All Development Tasks	https://www.veracode.com/press-release/ai-generated-code-poses-major-security-risks-in-nearly-half-of-all-development-tasks-veracode-research-reveals/	Veracode	45% das tarefas com falha	30/10/2026
Veracode 2026 GenAI code security: 56 percent pass rate	https://thenextweb.com/news/veracode-2026-genai-code-security-56-percent-pass-rate	The Next Web	Aprovação em segurança estagnada	30/10/2026
Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions	https://arxiv.org/abs/2108.09293	Pearce et al. (NYU)	~40% dos programas vulneráveis	30/10/2026
Do Users Write More Insecure Code with AI Assistants?	https://arxiv.org/abs/2211.03622	Perry et al. (Stanford)	Menos segurança, mais confiança	30/10/2026
AI Copilot Code Quality: 2025 Research	https://www.gitclear.com/ai_assistant_code_quality_2025_research	GitClear	Mais duplicação, menos refatoração	30/10/2026
Hallucinated packages: how AI invents dependencies attackers exploit	https://www.endorlabs.com/learn/hallucinated-packages-how-ai-invents-dependencies-attackers-exploit	Endor Labs	19,7% de pacotes alucinados	30/10/2026
Over 29 million secrets were leaked on GitHub in 2025	https://www.techradar.com/pro/security/over-29-million-secrets-were-leaked-on-github-in-2025-and-ai-really-isnt-helping	TechRadar	Commits com IA vazam ~2× mais	30/10/2026
AI-powered malware hit 2,180 GitHub accounts in s1ngularity attack	https://bleepingcomputer.com/news/security/ai-powered-malware-hit-2-180-github-accounts-in-s1ngularity-attack	BleepingComputer	Pacote Nx malicioso usou CLIs de IA	30/10/2026
OWASP Top 10:2025	https://owasp.org/Top10/2025/	OWASP	Lista oficial de 2025	30/10/2026
OWASP Top 10 for LLM Applications 2025	https://genai.owasp.org/llm-top-10/	OWASP GenAI Security Project	Riscos de aplicações com LLM	30/10/2026
NIST SP 800-218A	https://csrc.nist.gov/pubs/sp/800/218/a/final	NIST	SSDF para IA generativa	30/10/2026
SLSA: Security levels	https://slsa.dev/spec/v1.1/levels	OpenSSF / SLSA	Níveis de garantia do build	30/10/2026
MITRE Releases 2025 List of Top 25 Most Dangerous Software Vulnerabilities	https://securityweek.com/mitre-releases-2025-list-of-top-25-most-dangerous-software-vulnerabilities/	SecurityWeek	CWE Top 25 de 2025	30/10/2026
GitHub launches Copilot Fusion as agents start 40% of pull requests	https://thenextweb.com/news/github-universe-copilot-fusion-octoverse-agents	The Next Web	Mais de 40% dos PRs começam com agente	30/10/2026
The AI Productivity Paradox Report 2025	https://www.faros.ai/blog/ai-software-engineering	Faros AI	PRs maiores, revisão mais longa	30/10/2026
How OSS Contribution Policies Changed in Response to AI Slop	https://codenote.net/en/posts/oss-ai-slop-contribution-policy-shift/	codenote	Linha do tempo das políticas	30/10/2026
Generative AI Policy Landscape in Open Source	https://redmonk.com/kholterhoff/2026/02/26/generative-ai-policy-landscape-in-open-source/	RedMonk	Mapa das políticas de IA	30/10/2026
Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity (comentário)	https://simonwillison.net/b/8833	Simon Willison	METR: 19% mais lentos	30/10/2026
An update from the study that said devs were actually slower	https://birchtree.me/blog/an-update-from-the-study-that-said-devs-were-actually-slower-with-coding-agents/	Birchtree	Atualização do METR, evidência fraca	30/10/2026

Título	URL	Autor ou veículo	Ponto principal	
The Impact of AI on Developer Productivity: Evidence from GitHub Copilot	https://www.microsoft.com/en-us/research/publication/the-impact-of-ai-on-developer-productivity-evidence-from-github-copilot	Peng et al.	55,8% mais rápido em laboratório	30/10/2026
The Effects of Generative AI on High-Skilled Work: Evidence from Three Field Experiments with Software Developers	https://papers.ssrn.com/abstract=4945566	Cui, Demirer et al.	+26% de tarefas concluídas	30/10/2026
Announcing the 2024 DORA report	https://cloud.google.com/blog/products/devops-sre/announcing-the-2024-dora-report	Google Cloud / DORA	Vazão e estabilidade em queda	30/10/2026
DORA State of AI-assisted Software Development 2025	https://www.infoq.com/news/2025/09/dora-state-of-ai-in-dev-2025	InfoQ	Vazão positiva, instabilidade em alta	30/10/2026
How AI assistance impacts the formation of coding skills	https://www.anthropic.com/research/AI-assistance-coding-skills	Anthropic	Perguntar preserva o aprendizado	30/10/2026
Without Guardrails, Generative AI Can Harm Education	https://knowledge.wharton.upenn.edu/article/without-guardrails-generative-ai-can-harm-education	Knowledge at Wharton	Tutor de IA anula a perda	30/10/2026
MIT study shows cognitive debt through ChatGPT	https://the-decoder.com/mit-study-shows-cognitive-debt-through-chatgpt-heres-what-it-means-in-real-world-practice/	The Decoder	Dívida cognitiva; preprint	30/10/2026

CAPÍTULO 5 ·

Segurança cloud-first: o app-casca e a nova camada de proteção

> *Distribuir software sempre significou entregar ao usuário uma cópia do programa e, com ela, tudo o que ele sabe fazer. Enquanto ler um binário exigia semanas de um especialista, o risco era tolerável. Com modelos de IA capazes de explicar código compilado e procurar falhas em escala, deixou de ser. Este capítulo defende uma tese de arquitetura: aplicações sensíveis, a começar pelas bancárias, devem rodar quase inteiramente na nuvem, e o que chega ao dispositivo deve ser apenas uma casca que desenha a tela, captura o toque e conversa com o servidor por um protocolo de estado.*

A tese defendida neste capítulo

Diante de uma IA que lê binários e encontra vulnerabilidades em escala, soluções sensíveis devem rodar até 99% na nuvem. A primeira camada no aparelho deve ser só uma casca, embutida ou por streaming, que se comunica por protocolo de rede e replay de estado, sem expor código nem lógica de servidor. Esta é a posição do livro, não um consenso do setor; por isso o capítulo dedica uma seção inteira aos seus custos e limites.

Para começar

Um banco não entrega a cada cliente uma cópia do cofre com o manual da fechadura. O cliente vê só o balcão e faz pedidos; o caixa, do outro lado do vidro, consulta as regras e mexe no cofre. O app-casca é esse balcão digital. Termos que voltam no capítulo: **cliente** (o programa no aparelho), **servidor** (o computador do fornecedor), **binário** (o programa compilado) e **estado** (a fotografia do que a aplicação “sabe” num instante).

O problema: tudo o que vai no pacote pode ser lido

Compilar não é esconder

Compilar só troca a representação do código. Bytecode (Java, Kotlin, C#) preserva nomes e estrutura quase intactos; código de máquina é mais difícil, mas decompiladores maduros reconstroem funções e fluxos com boa fidelidade, como mostra o Livro 4, Capítulo 4 (Eng. Reversa e Segurança). Logo, **tudo o que vai no pacote deve ser tratado como público**: regras de validação, endereços de serviços internos, chaves de API, feature flags. O guia de testes mobile da OWASP (MASTG) resume: o cliente deve ser considerado sob controle do atacante.

O que a IA mudou

A engenharia reversa sempre foi possível; o que a limitava era o custo. Modelos de linguagem explicam pseudocódigo, reconhecem bibliotecas e percorrem milhares de funções sem cansaço. O post Engenharia Reversa com IA: a Adobe morreu?, de Fabio Akita, discute esse ponto: quando a leitura do binário fica barata, a proteção baseada em "ninguém vai ter paciência de ler isto" desaparece. Na busca de vulnerabilidades, o Google anunciou em agosto de 2025 que o **Big Sleep**, agente do Google DeepMind e do Project Zero, havia encontrado 20 falhas em projetos abertos como FFmpeg e ImageMagick. As mesmas capacidades servem a defensores e atacantes.

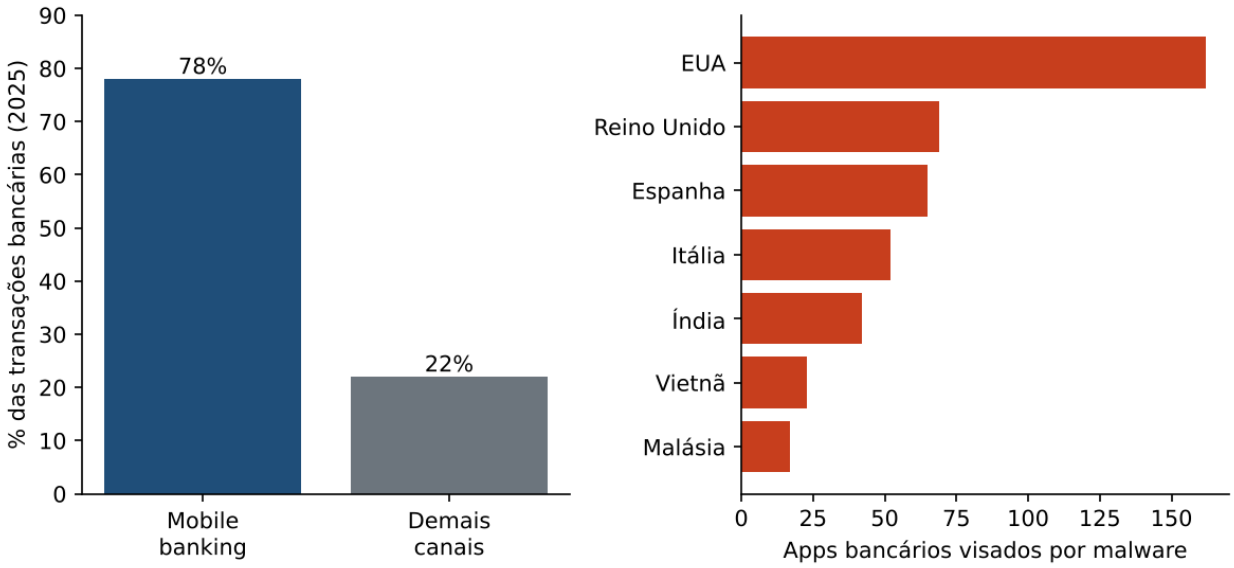
Os quatro pecados do cliente gordo

O **cliente gordo** (fat client, app que leva muita lógica no aparelho) carrega consigo e costuma cometer quatro pecados:

1. **Segredos embutidos**: se o app usa uma chave sozinho, ele a contém, e ela pode ser extraída.
2. **Lógica de negócio no cliente**: se a decisão acontece no aparelho, o atacante pode alterá-la.
3. **Superfície de API revelada**: o app entrega o mapa de endpoints (os endereços do servidor) e seus parâmetros.
4. **Adulteração**: versões modificadas do app são redistribuídas como se fossem originais.

Por que os bancos estão no centro do alvo

Segundo a Pesquisa Febraban de Tecnologia Bancária 2026, o Brasil teve cerca de 240,8 bilhões de transações bancárias em 2025, 78% delas pelo mobile banking. A Kaspersky registrou alta de 56% nos ataques de trojans bancários contra Android em 2025, e a Zimperium contou, em março de 2026, 34 famílias ativas mirando 1.243 marcas financeiras em 90 países.



À esquerda, participação do mobile banking nas transações no Brasil em 2025. À direita, países com mais apps bancários visados por malware. Fonte: elaboração própria com dados da Pesquisa Febraban 2026 (via Jornal de Brasília) e da Infosecurity Magazine.

Os trojans modernos não se limitam a roubar senhas: abusam de serviços de acessibilidade, sobrepõem telas falsas e operam *dentro da sessão verdadeira*, de modo que a fraude parece atividade normal.

Atenção: a casca não resolve tudo

Se o malware controla o aparelho, o servidor recebe pedidos que parecem legítimos. A arquitetura cloud-first reduz o que o atacante consegue aprender e alterar no app, mas precisa ser combinada com antifraude comportamental no servidor e com proteções no cliente.

Em resumo

Binários são legíveis, a IA tornou a leitura barata e apps bancários já são o alvo preferido. A pergunta de projeto deixa de ser "como esconder melhor o que está no pacote" e passa a ser "o que precisa mesmo estar no pacote".

Da proteção do código à proteção da fronteira

Para começar

Uma loja pode pôr alarme em cada produto ou deixar os produtos no estoque e mostrar só o catálogo. A indústria de software escolheu por muito tempo a primeira opção; esta seção explica por que a segunda é mais duradoura.

Ofuscação transforma código em algo funcionalmente igual, porém difícil de entender. É útil, mas tem um limite teórico: em 2001, Barak e coautores provaram que não existe ofuscador genérico de "caixa-preta virtual". Na prática, ofuscação é custo imposto ao atacante, e a IA barateia exatamente esse custo.

A resposta duradoura é mover a **fronteira de confiança**, a linha que separa o que você controla do que não controla. O aparelho do usuário está do lado de fora; o servidor, do lado de dentro. A regra da OWASP é que **nenhuma decisão de segurança pode depender do lado de fora**. O app-casca leva essa regra ao limite: se o cliente não decide nada, não há decisão a falsificar.

A ideia é antiga: terminais de mainframe, o X Window System (1984), Citrix e a web clássica eram modelos de casca. Os apps nativos e as SPAs dos anos 2010 levaram lógica ao cliente em busca de fluidez; a IA ofensiva é um motivo forte para o pêndulo voltar.

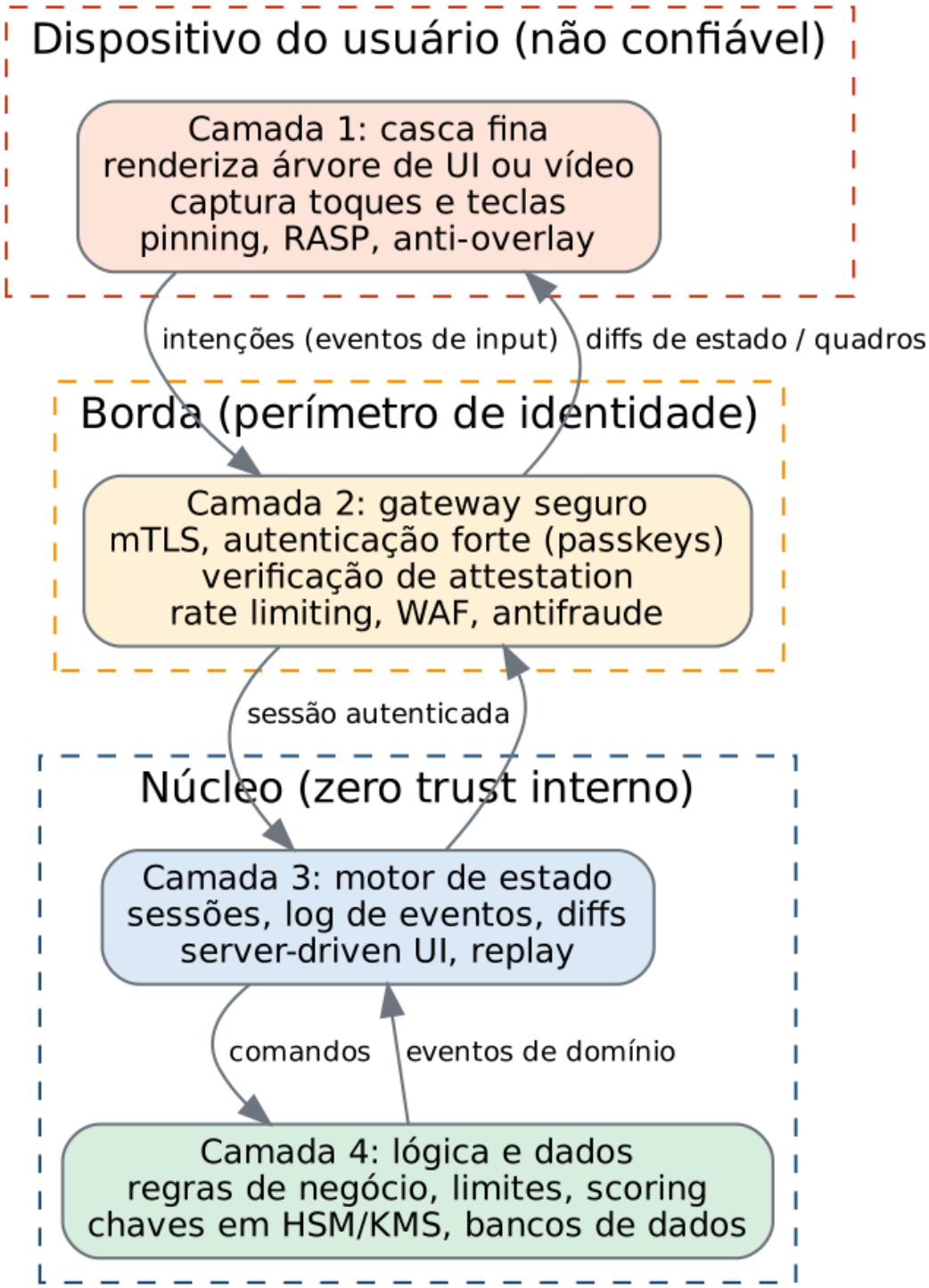
Você sabia?

Em 1995, Larry Ellison, da Oracle, defendeu o *Network Computer*, um aparelho sem disco que buscaria tudo na rede. A ideia fracassou porque as redes domésticas eram lentas; três décadas depois, Chromebooks e streaming de jogos realizaram boa parte dela.

A arquitetura proposta em quatro camadas

Para começar

Num teatro de marionetes, o público vê os bonecos, mas as mãos e o roteiro ficam atrás da cortina. A casca é o palco; o servidor, o titereiro; entre os dois há uma porta vigiada.



As quatro camadas da arquitetura cloud-first. O dispositivo fica fora da fronteira de confiança e só troca intenções e diffs de estado com o gateway. Fonte: elaboração própria.

Camada 1: a casca fina

A casca só **renderiza** e **captura input**. Há dois sabores: a **casca de componentes**, que monta telas a partir de uma árvore enviada pelo servidor (server-driven UI, Phoenix LiveView), e a **casca de pixels**, um player interativo que recebe vídeo e devolve toques (cloud gaming, desktops virtuais). Em ambos, o pacote não contém regra de negócio, endpoint interno nem segredo de serviço.

Camada 2: o gateway seguro

O gateway só deixa passar sessões que provem três coisas:

1. **Canal autêntico**: TLS 1.3 com certificate pinning (o app só aceita o certificado esperado) e, quando possível, mTLS (o cliente também se identifica com certificado), com a chave do cliente guardada em hardware seguro.
2. **Usuário autêntico**: **passkeys** (FIDO2/WebAuthn), resistentes a phishing.
3. **App e aparelho íntegros**: **attestation** assinada pelo sistema (Play Integrity no Android, App Attest no iOS), *verificada pelo servidor*.

O mTLS é natural em dispositivos corporativos gerenciados; em apps de consumo, o certificado do cliente pode ser gerado no primeiro uso e guardado no Secure Enclave (iPhone) ou no StrongBox/TEE (Android). Nas passkeys, a biometria apenas desbloqueia a chave, que nunca sai do hardware. Na attestation, o ponto crucial é que quem confere a declaração é o servidor, nunca o próprio app.

O gateway aplica ainda limitação de taxa, WAF e detecção de bots, e envia sinais ao motor de antifraude. Como o protocolo entre casca e gateway é pequeno, há poucos formatos de mensagem a validar, bem menos do que numa API REST com centenas de endpoints.

Camada 3: o motor de estado

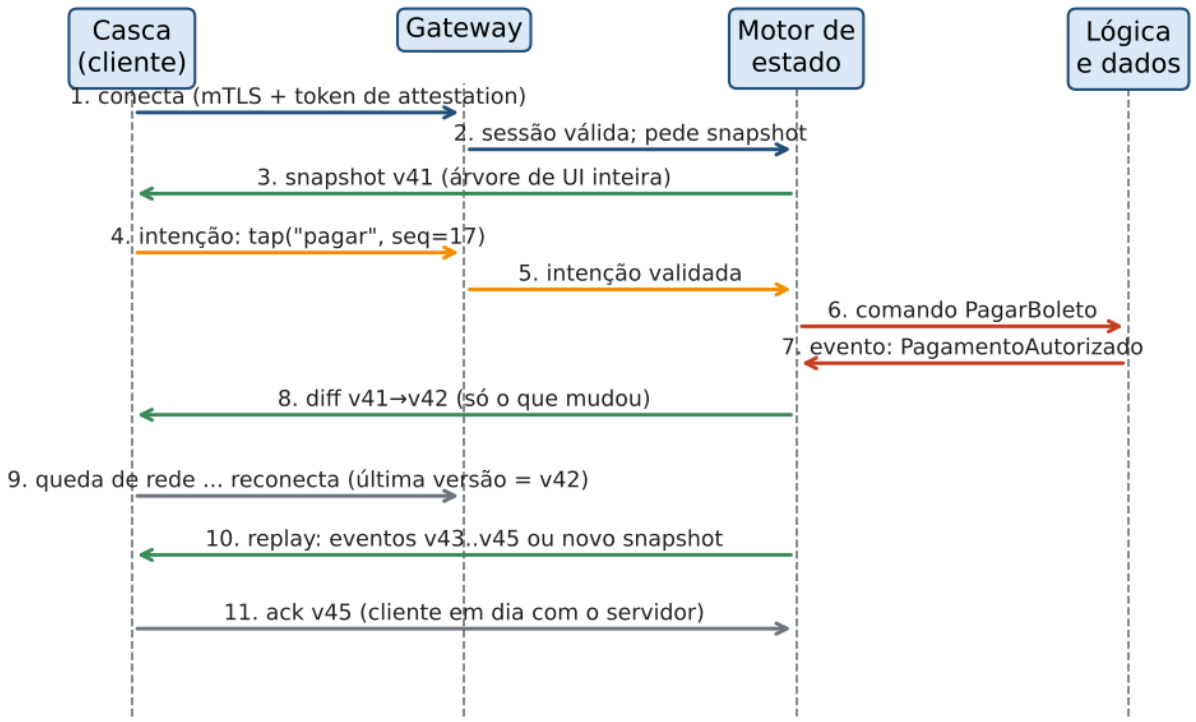
Em vez de dezenas de operações, o servidor mantém para cada sessão um **estado de tela** e um **log de eventos**. O cliente envia **intenções** (“tocou em Pagar”); o motor aplica a lógica, calcula o novo estado e devolve só o **diff** (a diferença em relação à tela anterior). Três ideias se combinam: **event sourcing** (Livro 4, Capítulo 1 (Eng. Software)), **server-driven UI** e **sincronização por versão**, que torna a reconexão trivial: “estou na versão 42; mande o que veio depois”.

Camada 4: lógica e dados só no servidor

Regras, modelos de risco, chaves (em HSM ou KMS) e bancos de dados ficam aqui. Os serviços se autenticam entre si com identidades de carga de trabalho e mTLS interno, cada um com só as permissões de que precisa. Essa camada não confia na anterior por padrão: recebe comandos com contexto de usuário e sessão e decide de novo, como se o pedido viesse de fora. É o princípio de **zero trust** aplicado ao próprio núcleo.

O protocolo de replay de estado

A figura a seguir mostra uma interação típica: conectar, receber a tela, tocar num botão, perder a rede e se recuperar.



Sequência do protocolo de replay de estado. A casca envia intenções numeradas; o motor responde com snapshots ou diffs versionados e, após uma queda, reenvia os eventos perdidos. Fonte: elaboração própria.

```
// cliente -> servidor: intenção, nunca uma regra
{"t":"intent","seq":17,"base":41,"action":"tap","target":"btn_pagar"}

// servidor -> cliente: diff sobre a versão 41 (estilo JSON Patch)
{"t":"patch","from":41,"to":42,"ops":[
  {"op":"replace","path":"/screen/title","value":"Pagamento confirmado"}]}

// após reconexão
{"t":"resume","last":42}
```

Primeiro a sessão é estabelecida e o cliente recebe um **snapshot**, a árvore completa da tela. Depois, a única coisa que o usuário “faz” é enviar intenções; cada uma atravessa o gateway, vira um comando de domínio, produz um evento e volta como diff. O número de sequência detecta duplicatas e garante ordem; após uma queda, o servidor reenvia os eventos que faltam ou um snapshot novo. O diff segue a ideia do JSON Patch (RFC 6902); em produção usam-se formatos binários e WebSocket, HTTP/2 ou, para pixels, WebRTC.

> **Dica: idempotência é obrigatória**
>
> Toda intenção que move dinheiro deve carregar uma chave de idempotência, para que uma reconexão automática não vire pagamento em dobro.

Variantes que já existem no mercado

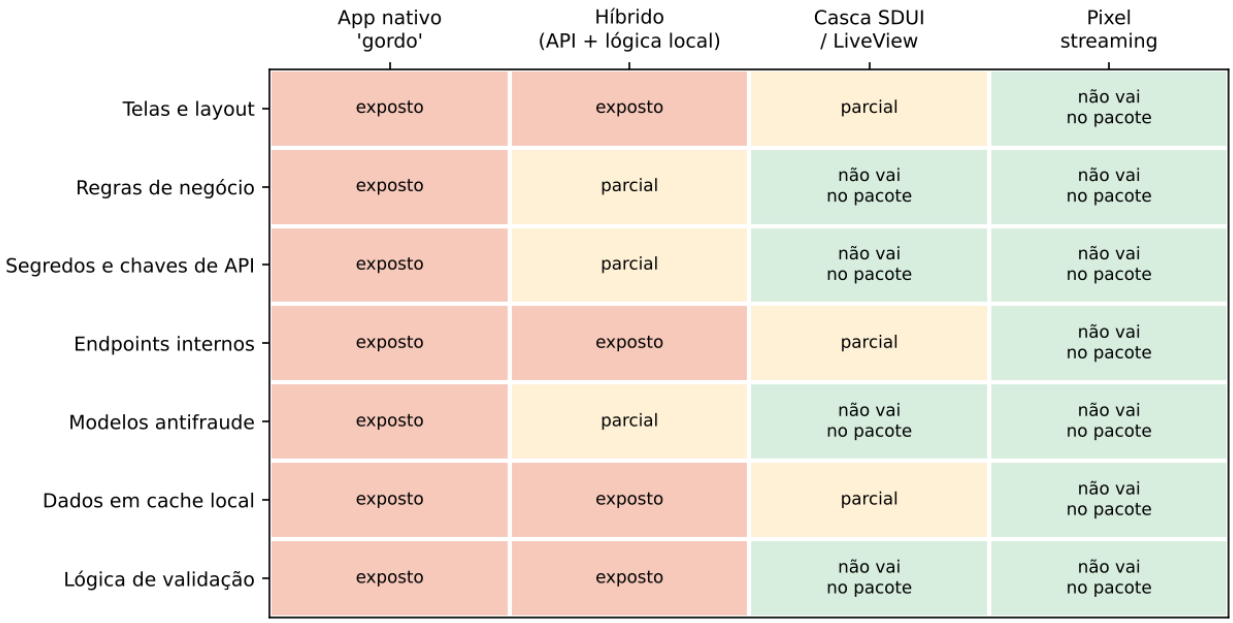
A casca não é uma tecnologia, é um espectro:

- **Renderização no servidor**: SSR e HTMX enviam HTML; o **Phoenix LiveView** (Elixir) mantém um processo leve por usuário e envia só os trechos que mudaram, e o **Blazor Server** faz o mesmo em .NET.
- **Server-driven UI nativo**: o Airbnb descreveu em 2021 sua Ghost Platform; o Nubank, em 2023, o **Backend Driven Content**, em Clojure e Flutter, usado em 43% do app. Ambos adotaram SDUI por velocidade de produto, mas o efeito colateral de segurança é real: o app carrega menos conhecimento do negócio.
- **WebView em apps bancários**: casca nativa para login e biometria, com telas web. Exige domínios fixos, pinning e nenhuma ponte JavaScript-nativo genérica.
- **Pixel streaming e VDI**: GeForce NOW, Pixel Streaming da Unreal e desktops virtuais corporativos não deixam sequer descrição de interface no cliente, ao custo de GPU, banda e latência.

Cada família tem seu cuidado. Em LiveView e HTMX, a tentação é reintroduzir lógica sensível em JavaScript "para ficar mais rápido". Em SDUI, o passo que transforma ganho de produto em política de segurança é simples de enunciar: nenhuma regra de negócio no cliente, nem mesmo de validação. No streaming de pixels, o efeito colateral é curioso: trapaças em jogos que dependem de ler a memória local deixam de funcionar, porque a memória está no data center. Para um banco, transmitir vídeo seria desperdício; para uma estação de engenharia com software caro, pode fazer todo sentido. Já VDI e DaaS são usados há anos por bancos, seguradoras e órgãos públicos em centrais de atendimento: os dados do cliente nunca tocam o notebook do operador, e bloqueios de área de transferência, impressão e captura de tela são aplicados no servidor.

> **Você sabia?**
> O LiveView só é viável em escala graças à máquina virtual BEAM, criada na Ericsson nos anos 1980 para centrais telefônicas que não podiam cair. A tecnologia que mantinha ligações de pé hoje mantém telas sincronizadas.

> **Você sabia?**
> O OnLive, primeiro grande serviço comercial de jogos por streaming, foi lançado em 2010 e fechou em 2015; o Google Stadia durou de 2019 a janeiro de 2023. Ambos tropeçaram mais no modelo de negócio e na latência das redes do que na tecnologia.



Comparação qualitativa do que vai dentro do pacote distribuído em cada arquitetura. Fonte: elaboração própria.

Variante	O que viaja pela rede	Custo principal
SSR / HTMX	Pedaços de página prontos (HTML)	Telas muito interativas dão mais trabalho
LiveView / Blazor Server	Só o que mudou na tela (diffs)	Memória no servidor para cada usuário
SDUI nativo	A “planta” da tela, peça por peça	Manter o catálogo de peças
Pixel streaming / VDI	Vídeo da tela e os toques	Placa de vídeo, internet boa e atraso

O que a casca ainda precisa proteger

> **Para começar**
>
> O balcão do banco não guarda dinheiro, mas precisa de vidro blindado e câmera. A casca não tem segredos de negócio, mas é onde o usuário digita a senha e confirma o pagamento.

- **Certificate pinning**: fixar a chave pública do servidor, com uma chave reserva, usando os mecanismos da plataforma.
- **Higiene do pacote**: remover símbolos e logs de depuração e nenhum segredo de longa duração. O teste: se o pacote fosse publicado amanhã, algo grave aconteceria? A resposta deve ser “não”.
- **RASP** (autoproteção do app em execução): verificações em tempo de execução (root, depurador, emulador) que **informam, não decidem**; o servidor combina os sinais com o risco da operação.
- **Attestation**: veredito verificado no servidor e vinculado a um nonce. É um sinal forte, não uma garantia.
- **Proteção da interface**: bloquear captura de tela e sobreposições em telas sensíveis e pedir confirmação por passkey nas operações de alto valor, sem bloquear quem depende de leitores de tela.

- **Antifraude no servidor**: como recebe todas as intenções, o servidor vê o comportamento da sessão (ritmo, sequência de telas, beneficiário novo) e decide quando exigir uma barreira extra.

Dois princípios atravessam essa lista. Primeiro, **um controle que decide localmente pode ser desligado localmente**: por isso RASP e attestation alimentam o servidor em vez de bloquear sozinhos. Segundo, a attestation tem limites conhecidos (aparelhos com root bem disfarçado existem, e o Google endureceu as regras da Play Integrity em 2025), então ela entra como um sinal entre vários. Um trojan que opera dentro da sessão costuma deixar padrões, como toques sem pausa humana e navegação direta para a tela de transferência; a casca não impede o malware de existir, mas o servidor, com visão total, pode reagir.

> **Em resumo**
> Na arquitetura casca, as proteções do cliente deixam de esconder segredos e passam a proteger o canal, atestar integridade, coletar sinais e proteger a interface. Quem decide é sempre o servidor.

Limites e trade-offs

> **Para começar**
> Centralizar tudo é como guardar o dinheiro da cidade num único cofre: fica mais fácil vigiar, mas, se a estrada fechar, ninguém saca. Cada custo abaixo é real e tem resposta de engenharia, nem sempre barata.

- **Latência**: uma ida e volta pode passar de 50 ms, e centenas em redes móveis ruins. Respostas: deixar local o que é cosmético (animações, máscaras), computação de borda e HTTP/3.
- **Uso offline**: casca pura não funciona sem rede. Resposta: cache de leitura cifrado e sem permitir ação; apps de campo pedem modelo híbrido.
- **Custo de servidor**: estado por sessão consome memória. Respostas: estados pequenos, hibernação de sessões e a variante certa. Em troca, correções chegam a todos com um deploy.
- **Privacidade**: o fornecedor vê tudo, em tensão com a minimização da LGPD. Resposta: finalidade e retenção definidas para a telemetria.
- **Servidor como alvo único**: o contra-argumento mais sério. O núcleo precisa operar em **zero trust** (NIST SP 800-207), com segmentação, HSM e logs imutáveis. A diferença é que o defensor controla o servidor e pode auditá-lo com a mesma IA.
- **Disponibilidade e acessibilidade**: múltiplas regiões e modo somente leitura; pixel streaming quebra leitores de tela, enquanto SDUI e LiveView podem ser plenamente acessíveis.

Para o **uso offline**, o cache de leitura deve guardar só o último snapshot de telas informativas (o extrato, por exemplo), cifrado com chave protegida por hardware e marcado com data. Operações offline genuínas, como pagamento por aproximação sem conexão,

exigem outro desenho, com credenciais tokenizadas de uso limitado, que é o que as carteiras digitais já fazem. No **custo**, o outro lado da balança inclui a fraude evitada, o fim de manter várias versões antigas do app com lógica divergente e a correção de vulnerabilidades por deploy, em vez de atualização forçada em milhões de aparelhos. Na **privacidade**, convém separar telemetria de segurança de dados de produto e cifrar dados sensíveis por usuário: a casca não pode virar desculpa para vigilância.

Vale detalhar dois desses pontos. Na **latência**, a física impõe um piso: a luz na fibra percorre cerca de 200 km por milissegundo, e a percepção humana tolera até uns 100 ms como "instantâneo". Além de deixar local o que é cosmético, o servidor pode enviar junto com a tela pequenas regras de apresentação ("ao tocar, mostre carregando"), sem lógica de negócio. No **servidor como alvo único**, zero trust significa na prática que nenhum serviço confia em outro por estar na mesma rede, que cada chamada é autenticada e autorizada e que o comprometimento de um serviço não abre os demais. Bancos já convivem com a dependência do servidor: o Pix não funciona sem os sistemas do Banco Central e, mesmo assim, virou o meio de pagamento mais usado do país.

No Brasil, segundo a TIC Domicílios 2025, 65% dos usuários acessam a internet só pelo celular (87% na classe DE). Isso reforça a tese e a desafia: a casca precisa ser leve em bytes e tolerar quedas. Uma tela SDUI trafega poucos kilobytes; um minuto de vídeo, megabytes.

> **Atenção**
> Uma casca lenta ou que não abre sem 4G empurra o usuário para canais menos seguros ou para a concorrência. A tese só vence se a experiência for, no mínimo, equivalente à do app gordo nos casos de uso principais.

Quando o modelo é ideal e quando não é

	Valor de ataque	Exigência regulatória	Tolera latência	Conectividade confiável	Adequação geral
Bancos e fintechs	alto	alto	alto	médio-alto	**alto**
Governo e saúde	alto	alto	alto	médio-alto	**alto**
Corporativo interno	médio-alto	médio-alto	alto	alto	**alto**
Varejo/e-commerce	médio-alto	médio-baixo	médio-alto	médio-alto	**médio-alto**
Jogos competitivos	médio-alto	médio-baixo	médio-baixo	médio-alto	**médio-alto**
Apps de campo offline	médio-baixo	médio-baixo	médio-alto	baixo	**baixo**
Edição de mídia/criação	médio-baixo	baixo	baixo	médio-alto	**médio-baixo**
Ferramentas de dev	médio-baixo	baixo	médio-baixo	médio-alto	**médio-baixo**

Matriz de adequação do modelo cloud-first por setor, em avaliação qualitativa. Fonte: elaboração própria.

É ideal em bancos e fintechs (alto valor de ataque, regulação forte, liquidação que já exige servidor), governo e saúde, sistemas corporativos internos e software de alto valor de propriedade intelectual (Livro 5, Capítulo 3 (Propriedade intelectual)). **Não é, ou não totalmente**, em apps de campo com uso offline prolongado, edição de mídia de baixa latência, jogos competitivos (onde o servidor autoritativo já é uma casca parcial) e software livre, cujo valor está na transparência (Livro 6, Capítulo 1 (Comunidades e Linux)).

Os critérios por trás da matriz são quatro: quanto vale atacar o sistema, quanto a regulação exige, quanta latência o uso tolera e quanto se pode contar com conexão. Bancos combinam valor de ataque máximo, normas do Banco Central sobre segurança cibernética e operações que de qualquer forma exigem servidor para liquidar; algumas centenas de milissegundos numa confirmação de pagamento são aceitáveis. Em sistemas corporativos internos, a rede é controlada e os aparelhos podem ser gerenciados, o que permite mTLS completo. Já para apps de campo, a saída é híbrida: casca para operações sensíveis, captura offline com sincronização e validação posterior no servidor.

> **Dica: a regra dos 99%**
> "Até 99% na nuvem" é direção, não dogma. O 1% local deve ser só o que é seguro expor e indispensável à experiência. Se uma linha de código local responde à pergunta "o usuário pode fazer isto?", ela está do lado errado da fronteira.

Roadmap de adoção

1. **Inventário e modelagem de ameaças**: auditar o próprio pacote para listar segredos, regras e endpoints expostos.

2. **Tirar segredos e decisões do cliente**: rotacionar chaves e mover para o servidor toda validação com efeito financeiro ou de segurança.
3. **Fortalecer a borda**: pinning, attestation, passkeys e gateway com antifraude.
4. **Migrar telas para SDUI ou LiveView**, começando pelas mais sensíveis, medindo latência e consumo de dados.
5. **Consolidar o núcleo em zero trust**, com auditoria contínua assistida por IA.

Área	Pergunta de verificação
Pacote	Há segredo, endpoint interno ou regra de negócio no binário?
Canal e identidade	Há TLS 1.3, pinning com chave reserva e passkeys?
Integridade	A attestation é verificada no servidor, com nonce?
Protocolo	O cliente envia só intenções, com sequência e idempotência?
Núcleo	Os serviços internos se autenticam entre si (zero trust)?
Estado	Snapshot, diffs e replay foram testados com quedas de rede reais?
Interface	Telas sensíveis bloqueiam captura e overlays?
Antifraude	O servidor combina RASP, attestation e comportamento para decidir risco?
Experiência	Latência e consumo foram medidos em 4G fraco e aparelho de entrada?
Acessibilidade	Os componentes funcionam com leitores de tela?
Privacidade	A telemetria de intenções tem finalidade e retenção definidas?

Relação com o desenvolvimento seguro

O Capítulo 4 (Desenvolvimento seguro e Pull Request) tratou de *como* construir software seguro com IA; este trata de *onde* o código deve rodar. Vários requisitos do OWASP MASVS, como armazenamento seguro e resiliência contra engenharia reversa, encolhem quando o app não guarda nada valioso, e o esforço de verificação migra para o servidor, onde a equipe controla as ferramentas. A mesma IA que barateia a engenharia reversa barateia a auditoria do servidor: o defensor troca um campo de batalha que não controla, milhões de aparelhos nas mãos de qualquer pessoa, por um que controla.

Há também uma ligação com o novo Pull Request. Com a lógica no servidor, cada mudança de regra passa por um único pipeline de revisão, testes e implantação, e uma correção de segurança chega a todos em minutos, sem depender da loja de aplicativos, cuja aprovação pode levar dias. Em contrapartida, o servidor concentra o risco, e é ali que revisão assistida por IA, análise estática e verificação de dependências precisam do máximo rigor.

> **Em resumo**
>
> O app-casca aplica um princípio antigo, não confiar no cliente, num momento em que a IA o tornou urgente. A casca desenha e escuta; o gateway verifica; o motor de estado sincroniza; o núcleo decide em zero trust. Os custos são desafios de engenharia com respostas conhecidas, e para bancos, governo e sistemas corporativos o saldo é favorável.

Fontes deste capítulo

Dados atuais verificados em 31/10/2026.

Título	URL	Autor ou veículo	Ponto principal
Engenharia Reversa com IA: a Adobe morreu?	https://akitaonrails.com/2026/10/13/engenharia-reversa-com-ia-a-adobe-morreu/	Fabio Akita (AkitaOnRails)	IA barateia engenharia reversa de software comercial
Google says its AI-based bug hunter found 20 security vulnerabilities	https://techcrunch.com/2025/08/04/google-says-its-ai-based-bug-hunter-found-20-security-vulnerabilities/	TechCrunch	Big Sleep encontrou 20 falhas
Mobile banking cyberattacks rise by 56pc in 2025: report	https://www.dawn.com/news/1978843	Dawn (dados da Kaspersky)	Trojans bancários Android +56% em 2025
Financial Brands Targeted in Global Mobile Banking Malware Surge	https://www.infosecurity-magazine.com/news/financial-brands-mobile-banking/	Infosecurity Magazine (dados da Zimperium)	34 famílias, 1.243 marcas, 90 países
Volume de transações bancárias foi de 240,8 bilhões em 2025	https://jornaldebrasilia.com.br/noticias/economia/volume-de-transacoes-bancarias-foi-de-2408-bilhoes-em-2025-alta-anual-de-11-diz-febraban/	Jornal de Brasília (Pesquisa Febraban 2026)	78% das transações via mobile banking
TIC Domicílios 2025	https://www.mobiletime.com.br/noticias/09/12/2025/tic-domicilios-2025/	Mobile Time (dados do Cetic.br)	65% acessam a internet só pelo celular
Backend Driven Content: why we developed a Server Driven UI framework at Nubank	https://building.nu.com/server-driven-ui-framework-at-nubank/	Rodrigo Sampaio (Building Nubank)	BDC em Clojure e Flutter; 43% do app
A Deep Dive into Airbnb's Server-Driven UI System	https://medium.com/airbnb-engineering/a-deep-dive-into-airbnbs-server-driven-ui-system-842244c5f5	Airbnb Engineering	Ghost Platform: SDUI para web e mobile
OWASP Mobile Application Security (MASVS/MASTG)	https://mas.owasp.org/	OWASP	Padrões de verificação de apps móveis
SP 800-207: Zero Trust Architecture	https://csrc.nist.gov/pubs/sp/800/207/final	NIST	Definição de referência de zero trust
Play Integrity API	https://developer.android.com/google/play/integrity	Android Developers	Attestation de app e dispositivo no Android
Establishing your app's integrity (App Attest)	https://developer.apple.com/documentation/devicecheck/establishing-your-app-s-integrity	Apple Developer	Attestation com chave no Secure Enclave
RFC 6902: JavaScript Object Notation (JSON) Patch	https://www.rfc-editor.org/rfc/rfc6902	IETF	Formato padrão de diffs em JSON
Phoenix LiveView	https://hexdocs.pm/phoenix_live_view/	Documentação oficial	Estado no servidor e diffs por WebSocket
Pixel Streaming in Unreal Engine	https://dev.epicgames.com/documentation/en-us/unreal-engine/pixel-streaming-in-unreal-engine	Epic Games	Aplicação 3D transmitida por WebRTC
On the (Im)possibility of Obfuscating Programs	https://eprint.iacr.org/2001/069	Barak et al. (2001)	Não existe ofuscação genérica de caixa-preta